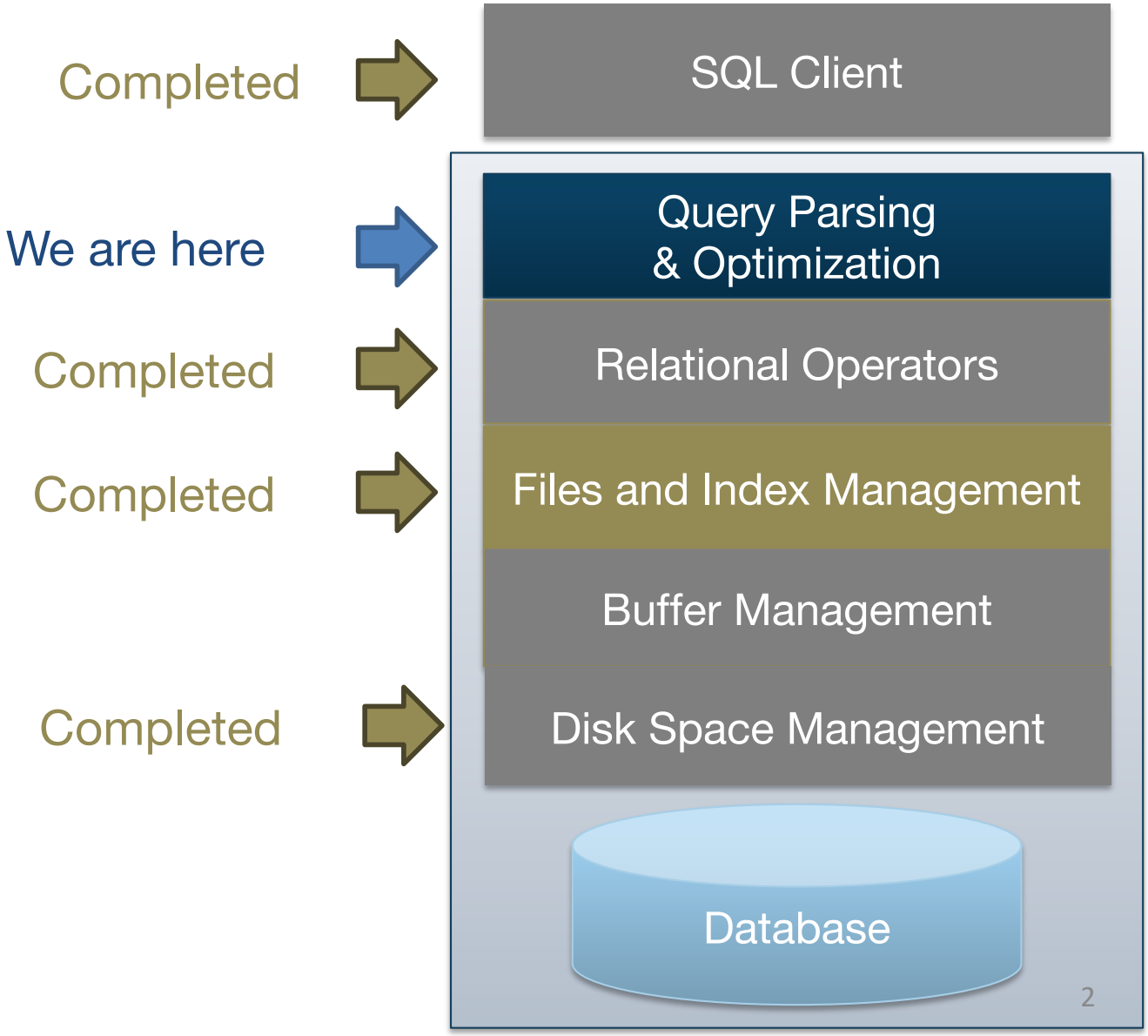
Cow book (3rd edition): Chapter 15

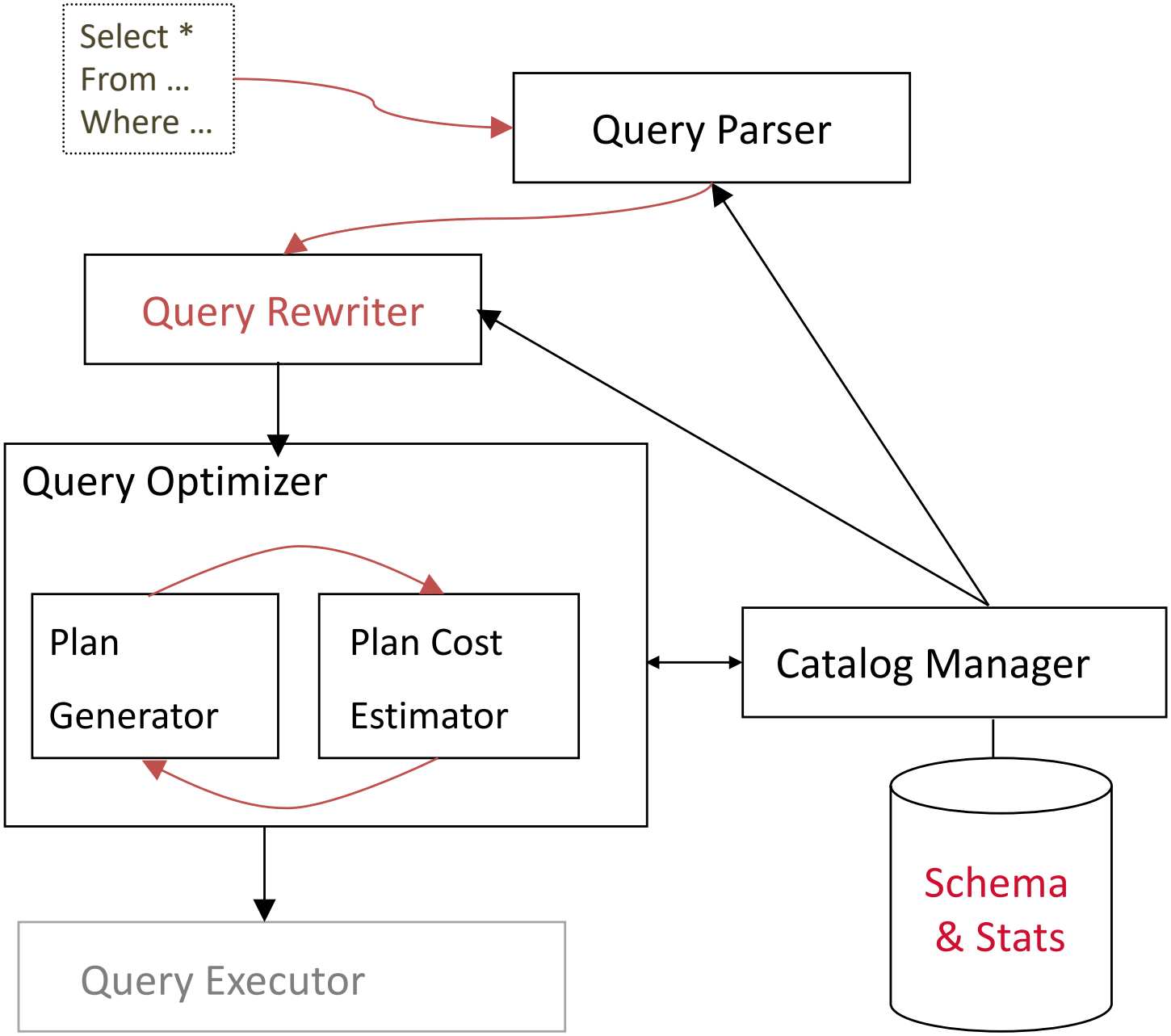
Chapter 1 – DBMS

RELATIONAL QUERY OPTIMIZATION: THE PLAN SPACE

Architecture of an RDBMS

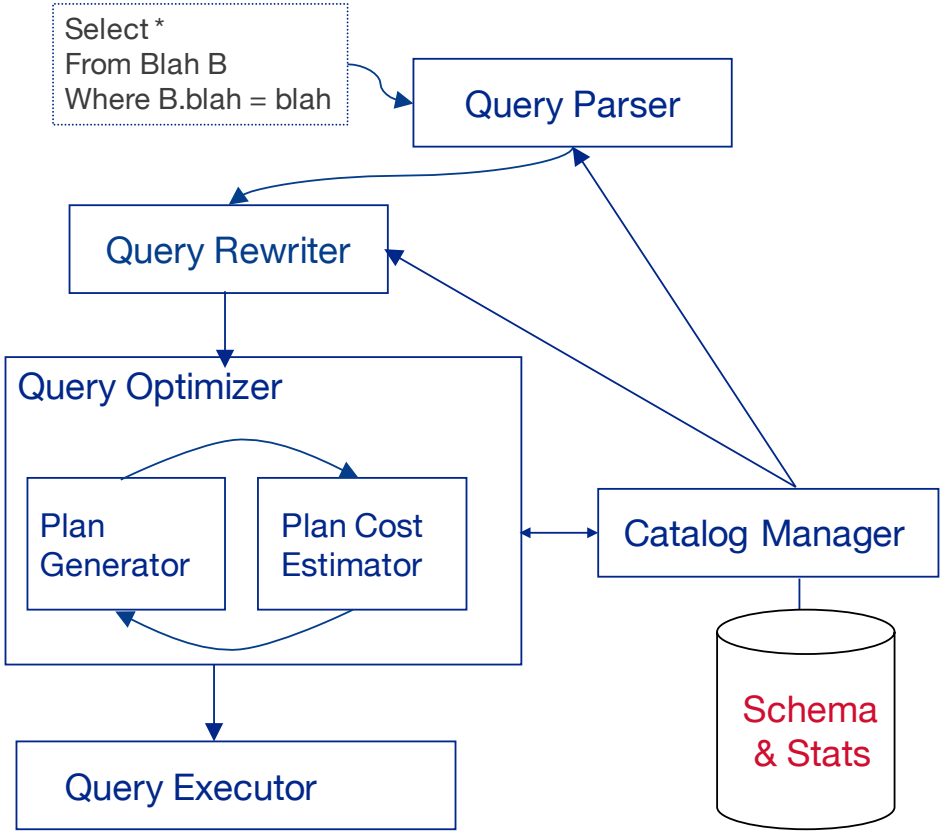


Query parsing & optimization



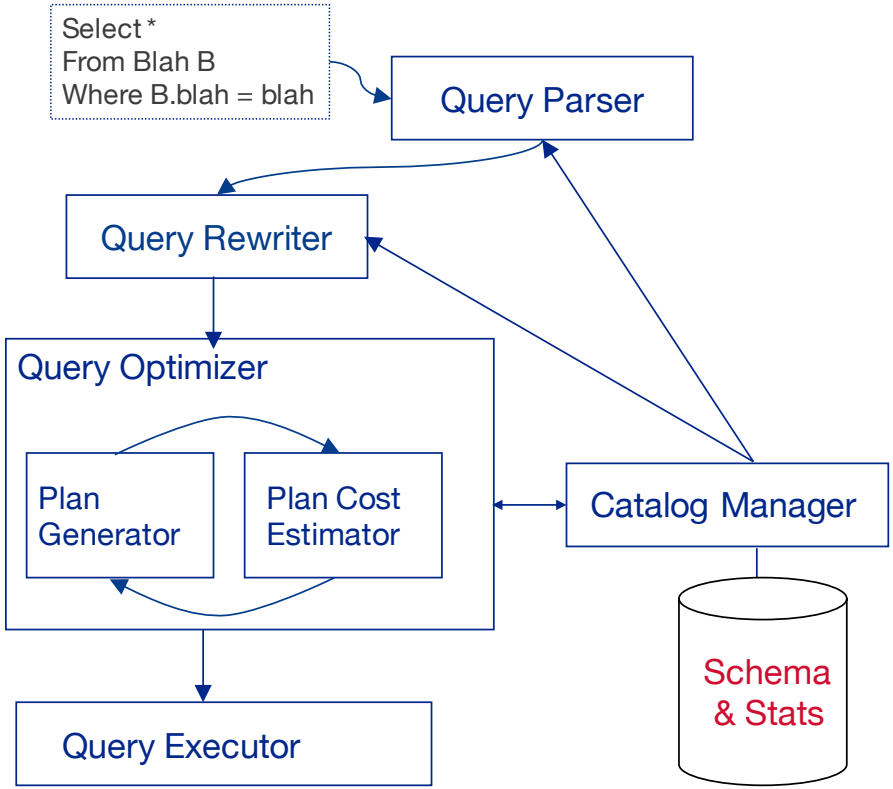
Query parsing & optimization

- Query parser
  - Checks correctness, authorization
  - Generates a parse tree
  - Simple / boring



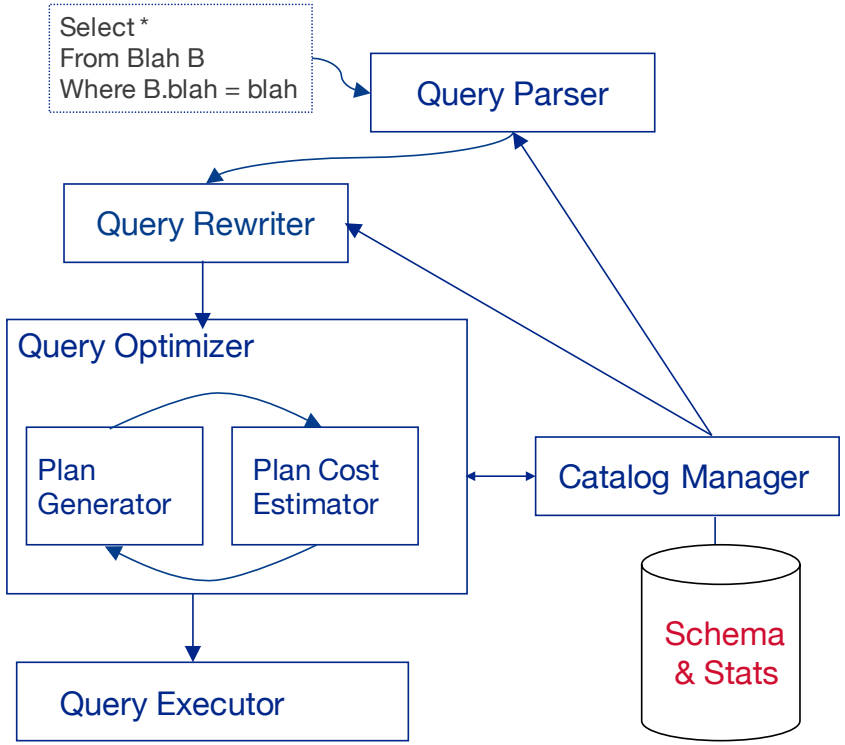
Query parsing & optimization

- Query parser
  - Checks correctness, authorization
  - Generates a parse tree
  - Simple / boring
- Query rewriter
  - Converts queries to canonical form
  - E.g. flatten views, subqueries into fewer query blocks
  - Weak spot in many open-source DBMSs



Query parsing & optimization

- Query parser
  - Checks correctness, authorization
  - Generates a parse tree
  - Simple / boring
- Query rewriter
  - Converts queries to canonical form
  - E.g. flatten views, subqueries into fewer query blocks
  - Weak spot in many open-source DBMSs
- "Cost-based" Query Optimizer
  - Optimizes 1 query block at a time
    - Select, Project, Join
    - GroupBy/Agg
    - Order By (if top-most block)
  - Uses catalog stats to find least-"cost" plan per query block
  - Most critical component of every RDBMS (both OLTP and OLAP; OLAP harder!
    - Sometimes not truly "optimal"



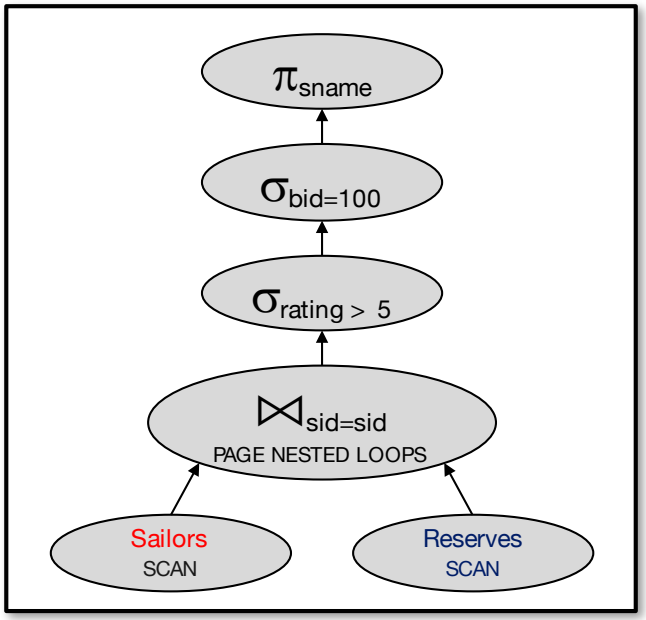
Query optimization overview

- Query block can be converted to relational algebra
- Relational algebra converts to tree
- Each operator has implementation choices
- Operators can also be applied in different orders!

```
SELECT S.sname
  FROM Reserves R, Sailors S
 WHERE R.sid=S.sid
   AND R.bid=100
   AND S.rating>5
```

$\pi_{(sname)}\sigma_{(bid=100 \wedge rating > 5)}$

(Reserves ⋈ Sailors)



Query optimizer types

- Rule-based: predefined rules, no/few cost computations
- Heuristics-based: rules and cost estimates, but no (few) statistics on data
- Cost-based: rules and statistics-based cost estimates
  - Adaptive cost-based: while plan is executed, if estimate errors > threshold, change and reoptimize

Query optimization overview

- *Plan*: Tree of RA operators (and some others) with choice of algorithm for each operator

Query optimization overview

- *Plan*: Tree of RA operators (and some others) with choice of algorithm for each operator

- Three *independent* concerns:
  - Plan space:
    - for a given query, what plans are considered?
  - Cost estimation:
    - how is the cost of a plan estimated?
  - Search strategy:
    - how do we "search" in the "plan space"?

Query optimization overview

- *Plan*: Tree of RA operators (and some others) with choice of algorithm for each operator

- Three *independent* concerns:
  - Plan space:
    - for a given query, what plans are considered?
  - Cost estimation:
    - how is the cost of a plan estimated?
  - Search strategy:
    - how do we "search" in the "plan space"?

- Optimization goal:
  - Ideally: Find the plan with least actual cost.
  - Reality: Find the plan with least estimated cost.
    - And try to avoid really bad actual plans!

Outline

- An introduction to the plan space
- Explore one simple example query
- Cost estimation and search strategy

Relational algebra equivalences

- Selections:
  - $\sigma_{c1 \wedge \ldots \wedge cn}(R) \equiv \sigma_{c1}(\ldots(\sigma_{cn}(R))\ldots)$ (*cascade*)
  - $\sigma_{c1}(\sigma_{c2}(R)) \equiv \sigma_{c2}(\sigma_{c1}(R))$ (*commute*)

Relational algebra equivalences

- Selections:
  - $\sigma_{c1 \wedge \ldots \wedge cn}(R) \equiv \sigma_{c1}(\ldots(\sigma_{cn}(R))\ldots)$ (*cascade*)
  - $\sigma_{c1}(\sigma_{c2}(R)) \equiv \sigma_{c2}(\sigma_{c1}(R))$ (*commute*)
- Projections:
  - $\pi_{a1}(R) \equiv \pi_{a1}(\ldots(\pi_{a1, \ldots, an-1}(R))\ldots)$ (*cascade*)

Relational algebra equivalences

- Selections:
  - $\sigma_{c1 \wedge \ldots \wedge cn}(R) \equiv \sigma_{c1}(\ldots(\sigma_{cn}(R))\ldots)$ (*cascade*)
  - $\sigma_{c1}(\sigma_{c2}(R)) \equiv \sigma_{c2}(\sigma_{c1}(R))$ (*commute*)
- Projections:
  - $\pi_{a1}(R) \equiv \pi_{a1}(\ldots(\pi_{a1, \ldots, an-1}(R))\ldots)$ (*cascade*)
- Cartesian Product
  - $R \times (S \times T) \equiv (R \times S) \times T$ (associative)
  - $R \times S \equiv S \times R$ (commutative)

Relational algebra equivalences

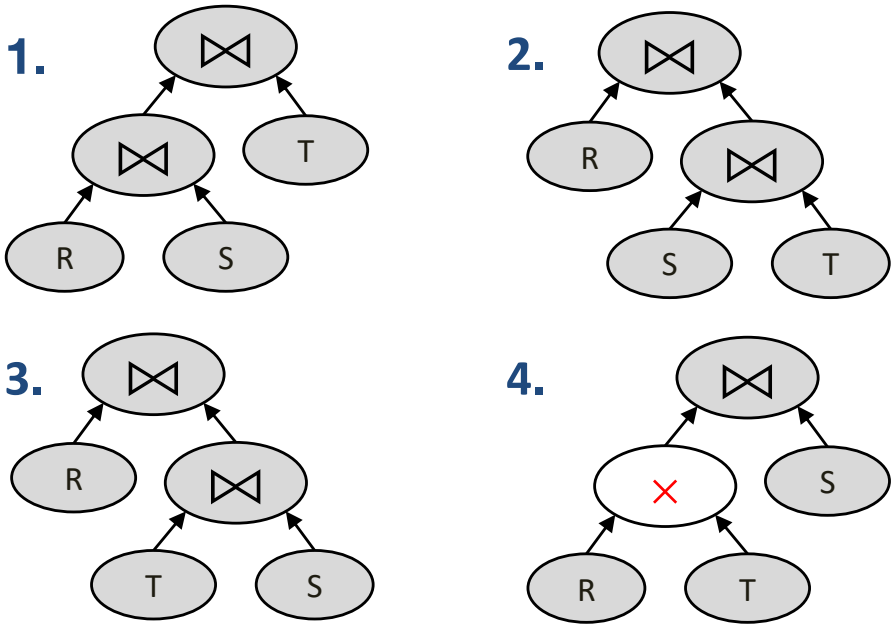
- Selections:
  - $\sigma_{c1 \wedge \ldots \wedge cn}(R) \equiv \sigma_{c1}(\ldots(\sigma_{cn}(R))\ldots)$ (*cascade*)
  - $\sigma_{c1}(\sigma_{c2}(R)) \equiv \sigma_{c2}(\sigma_{c1}(R))$ (*commute*)
- Projections:
  - $\pi_{a1}(R) \equiv \pi_{a1}(\ldots(\pi_{a1, \ldots, an-1}(R))\ldots)$ (*cascade*)
- Cartesian Product
  - $R \times (S \times T) \equiv (R \times S) \times T$ (associative)
  - $R \times S \equiv S \times R$ (commutative)
  - *This means we can do joins in any order.*
    - But… beware of join turning into cross-product!
    - Consider R(a,b), S(b,c), T(d,e)
    - **$(R \times T) \bowtie S \not\equiv R \times (T \bowtie S)$**
    - Said differently, $\sigma_{b=b}((R \times T) \times S) \not\equiv R \times \sigma_{True}(T \times S)$

Join ordering, again

```
SELECT *
  FROM R, S, T
 WHERE R.a = S.a
   AND S.b = T.b;
```

- Similarly, note that some join orders have cross products, some don't
- Equivalent for the query above:

1. $(R \bowtie_{a=a} S) \bowtie_{b=b} T$
2. $R \bowtie_{a=a} (S \bowtie_{b=b} T)$
3. $R \bowtie_{a=a} (T \bowtie_{b=b} S)$
4. $(R \times T) \bowtie_{a=a \wedge b=b} S$



Some common heuristics

$\pi_{sname}\sigma_{(bid=100 \wedge rating > 5)}$ (Reserves $\bowtie_{sid=sid}$ Sailors)

- Selection cascade and pushdown
  - Apply selections as soon as you have the relevant columns

π_{sname} ($\sigma_{(bid=100)}$ (Reserves) $\bowtie_{sid=sid}$ $\sigma_{rating > 5}$ (Sailors))

Some common heuristics

$$\pi_{\text{sname}}\sigma_{(\text{bid}=100 \wedge \text{rating} > 5)}\ (\text{Reserves} \bowtie_{\text{sid}=\text{sid}} \text{Sailors})$$

- Selection cascade and pushdown
  - Apply selections as soon as you have the relevant columns

$$\pi_{\text{sname}}\ (\sigma_{(\text{bid}=100)}\ (\text{Reserves}) \bowtie_{\text{sid}=\text{sid}} \sigma_{\text{rating} > 5}\ (\text{Sailors}))$$

- Projection cascade and pushdown
  - Keep only the columns you need to evaluate downstream operators

$$\pi_{\text{sname}}\ (\pi_{\text{sid}}(\sigma_{\text{bid}=100}\ (\text{Reserves})) \bowtie_{\text{sid}=\text{sid}} \pi_{\text{sname,sid}}\ (\sigma_{\text{rating} > 5}\ (\text{Sailors})))$$

Some common heuristics

$$\pi_{\text{sname}}\sigma_{(\text{bid}=100 \wedge \text{rating} > 5)}\ (\text{Reserves} \bowtie_{\text{sid}=\text{sid}} \text{Sailors})$$

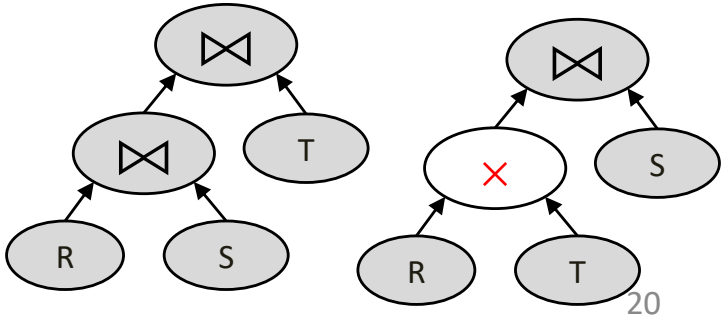
- Selection cascade and pushdown
  - Apply selections as soon as you have the relevant columns

$$\pi_{\text{sname}}\ (\sigma_{(\text{bid}=100)}\ (\text{Reserves}) \bowtie_{\text{sid}=\text{sid}} \sigma_{\text{rating} > 5}\ (\text{Sailors}))$$

- Projection cascade and pushdown
  - Keep only the columns you need to evaluate downstream operators

$$\pi_{\text{sname}}\ (\pi_{\text{sid}}(\sigma_{\text{bid}=100}\ (\text{Reserves})) \bowtie_{\text{sid}=\text{sid}} \pi_{\text{sname,sid}}\ (\sigma_{\text{rating} > 5}\ (\text{Sailors})))$$

- Avoid Cartesian products
  - Do theta-joins before cross-products
  - Consider R(a,b), S(b,c), T(c,d)
  - Favor (R ⋈ S) ⋈ T over (R × T) ⋈ S



Physical equivalences

- Base table access, with single-table selections and projections
  - Heap scan
  - Index scan (if available on referenced columns)
  - Bitmap scan (when multiple indexes on same table)
  - Binary search (if clustered index)
- Equi-joins
  - Chunk Nested Loops: simple, exploits extra memory
  - Index Nested Loops: if 1 relation small and the other indexed properly
  - Sort-Merge Join: good with small memory, equal-size tables
  - Grace Hash Join: good with 1 small table
    - Or Hybrid if implemented
- Non-Equijoins
  - Chunk Nested Loops

Quick quizz

- Pushing $\sigma_{p(x)}$ below $\bowtie$ is a bad idea in which unusual cases:
  A. p(x) always returns True on our data
  B. p(x) takes a very long time to check
  C. p(x) does not return a Boolean value

Quick quizz

Pushing project down as far as you can is a good heuristic because

A. Unneeded columns don’t take space in memory buffers
B. Unneeded columns don’t spill into disk partitions
C. Copy costs are lower during query processing
D. All of the above

Quick quizz

For an SQL query, we explore a plan space including

A. A set of relational algebra expressions, all equivalent to SQL ones
B. Multiple physical implementations per relational algebra operator
C. All of the above

Schema for examples

Sailors(*sid*:integer, *sname*:text, *rating*:integer,*age*: real)
Reserves(*sid*: integer,*bid*:integer,*day*:date, *rname*: text)

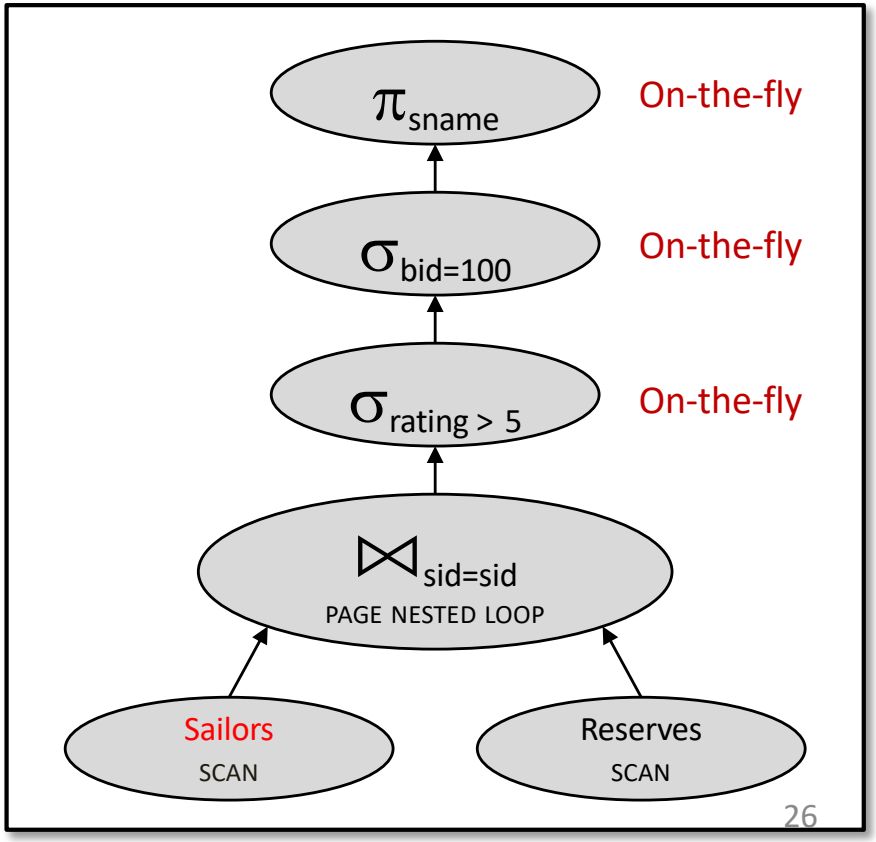
- Reserves:
  - Each tuple is 40 bytes long, 100 tuples per page, 1000 pages.
  - Assume there are 100 boats
- Sailors:
  - Each tuple is 50 bytes long, 80 tuples per page, 500 pages.
  - Assume there are 10 different ratings

- Assume we have 5 pages to read into for joins (5 + 1 page for output, so 6 buffer pages in total)

Motivating example

```
SELECT S.sname
  FROM Reserves R, Sailors S
 WHERE R.sid=S.sid
   AND R.bid=100
   AND S.rating>5
```

Here's a reasonable query plan:



Motivating example

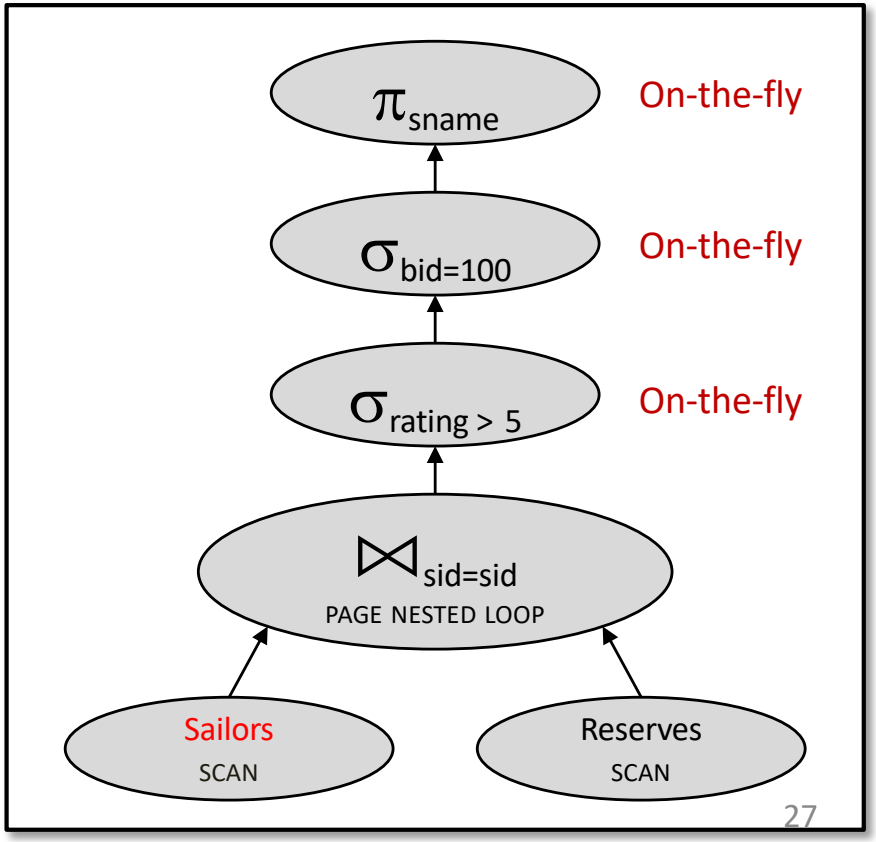
```
SELECT S.sname
  FROM Reserves R, Sailors S
 WHERE R.sid=S.sid
   AND R.bid=100
   AND S.rating>5
```

Let's estimate the cost:

- Scan Sailors (500 IOs)
- For each page of Sailors,
  Scan Reserves (1000 IOs)

- Total: 500 + 500*1000

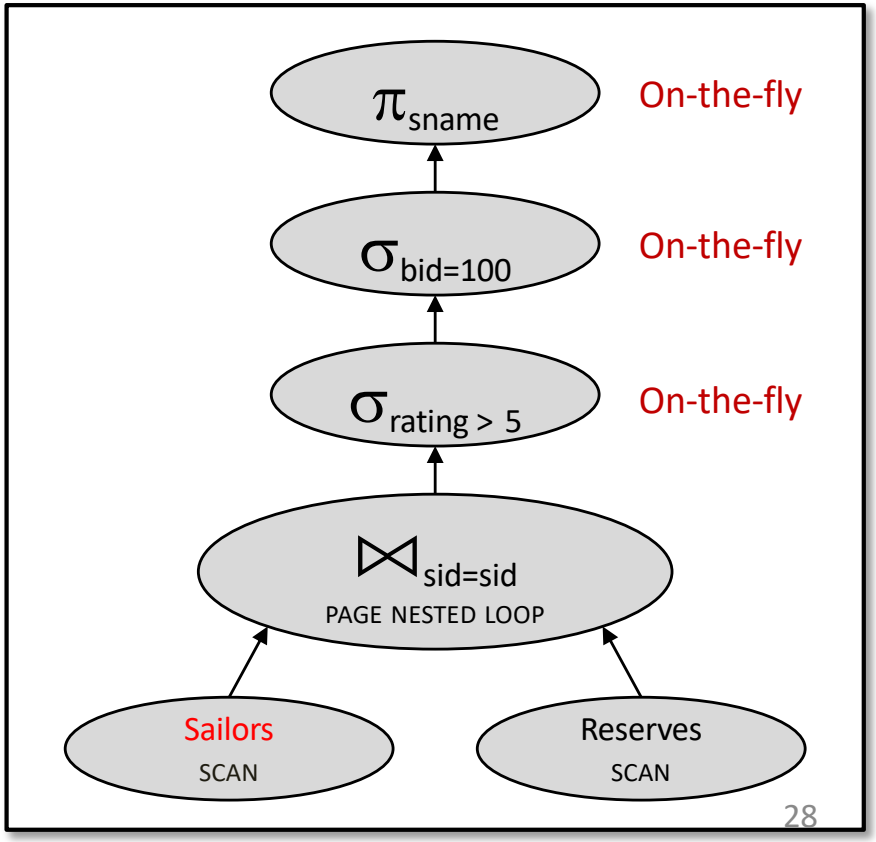
500,500 IOs



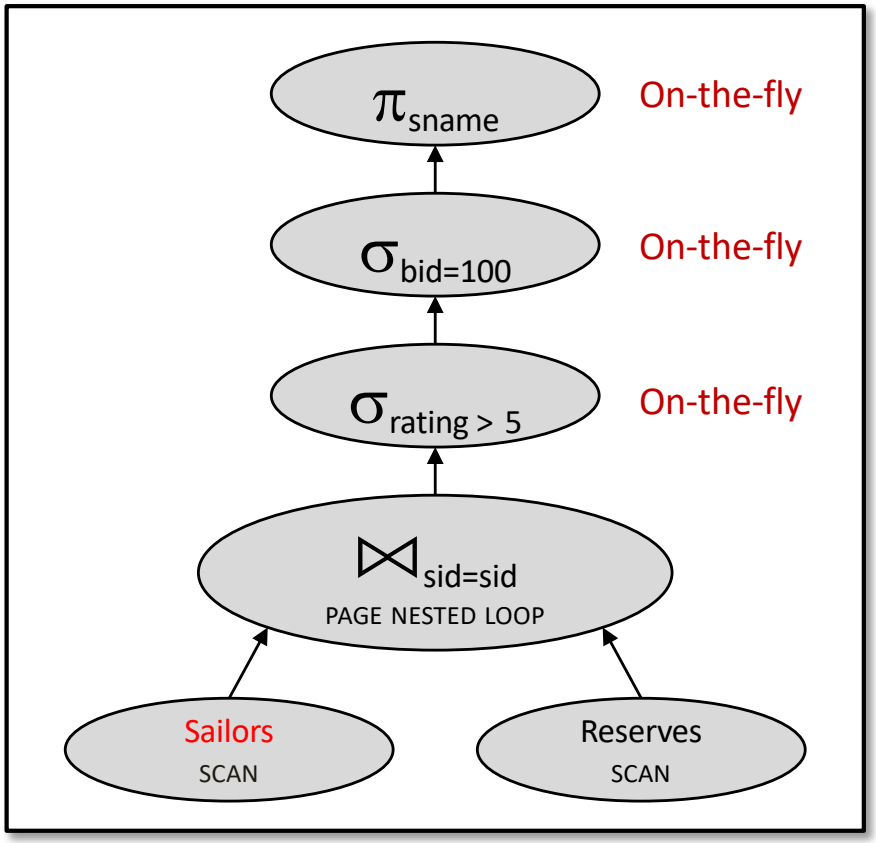
Motivating example

```
SELECT S.sname
  FROM Reserves R, Sailors S
 WHERE R.sid=S.sid
   AND R.bid=100
   AND S.rating>5
```

- Cost: 500+500*1000 I/Os
- By no means the worst plan!
- Misses several opportunities:
  - selections could be 'pushed' down
  - no use made of indexes
- *Goal of optimization:* Find faster plans that compute the same answer.

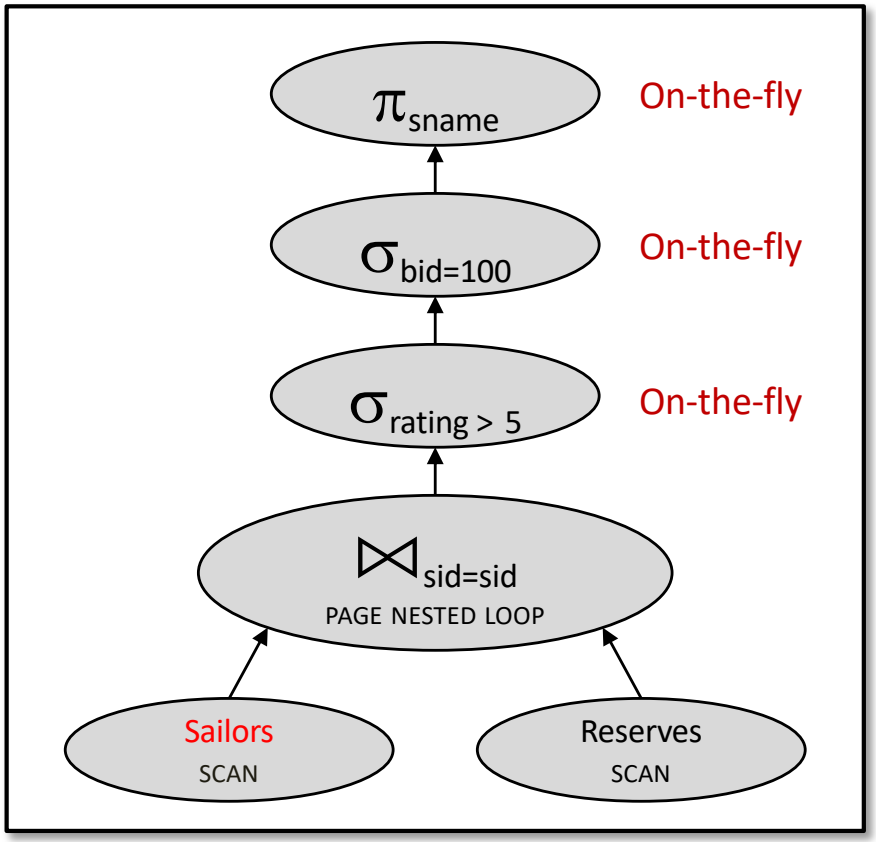


Selection pushdown

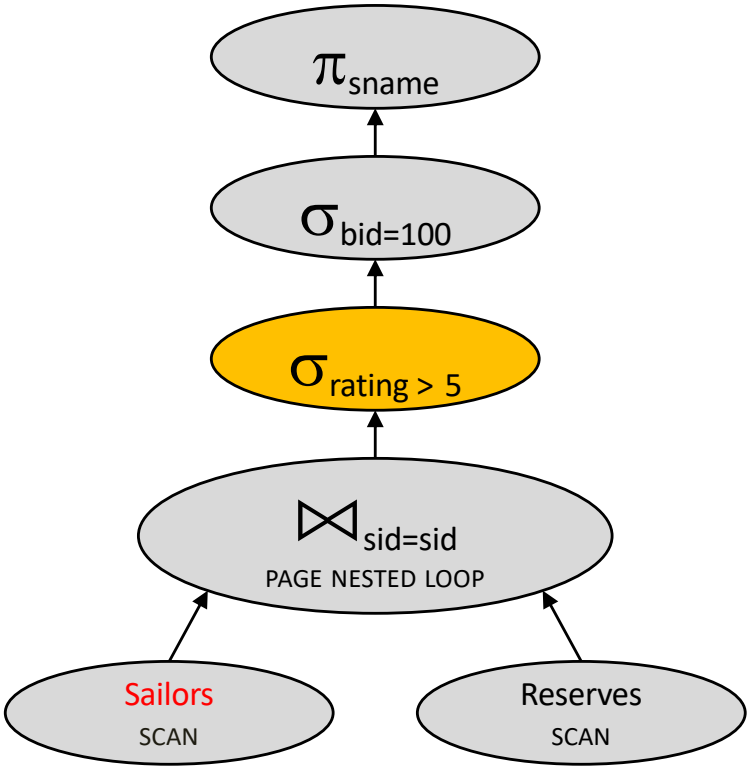


500,500 IOs

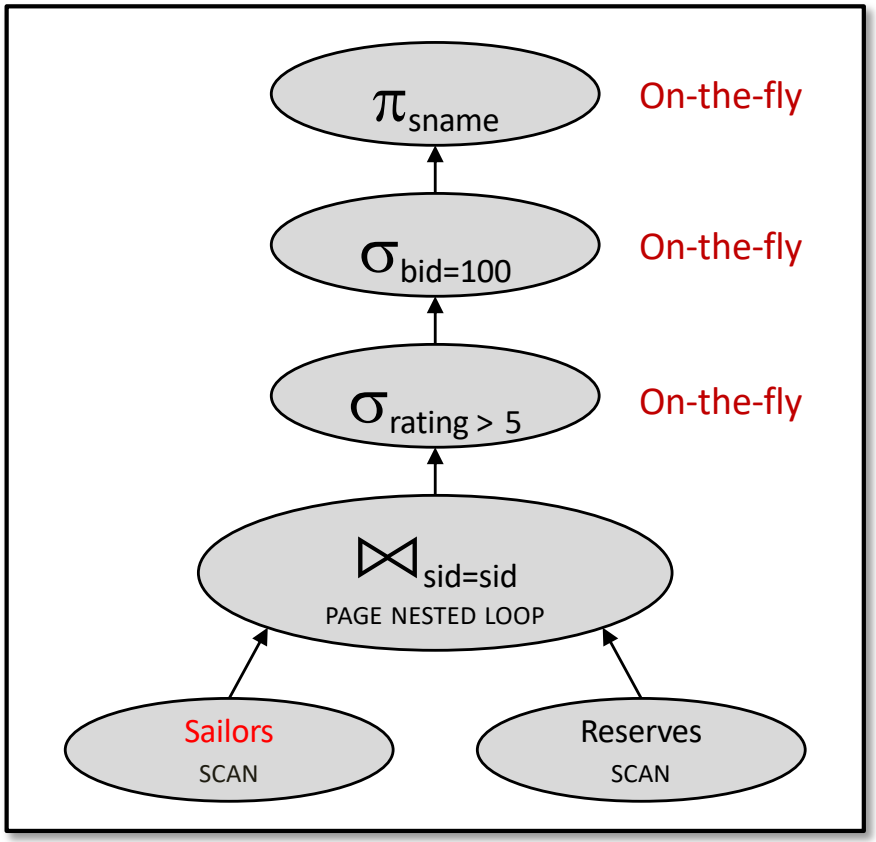
Selection pushdown



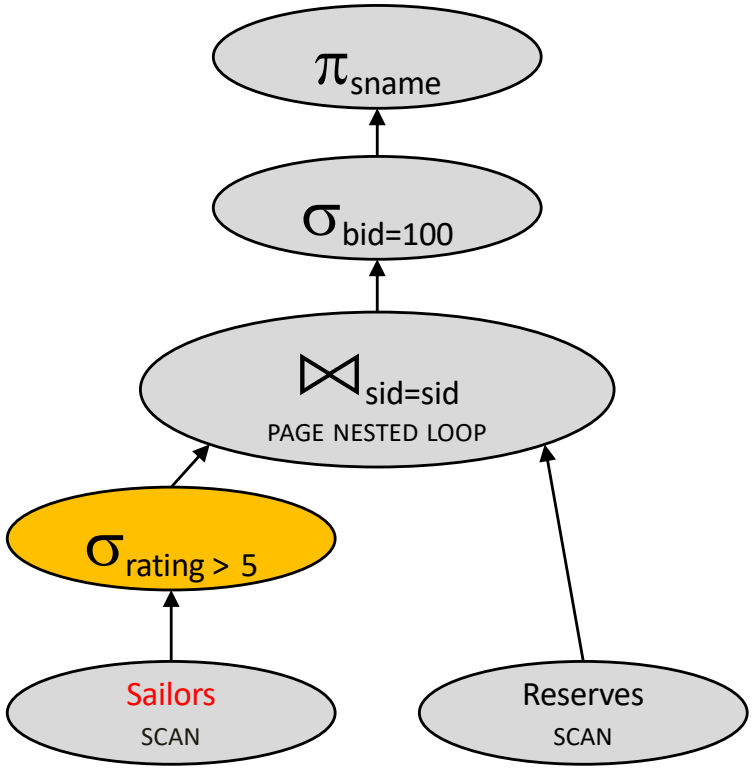
500,500 IOs



Selection pushdown



500,500 IOs



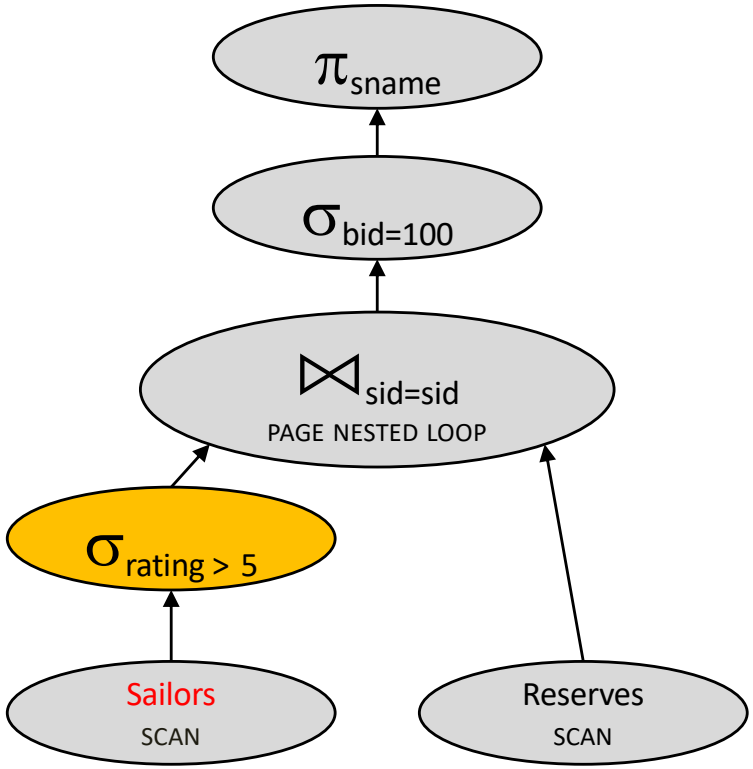
Cost?

Quick quizz

Let's estimate the cost:

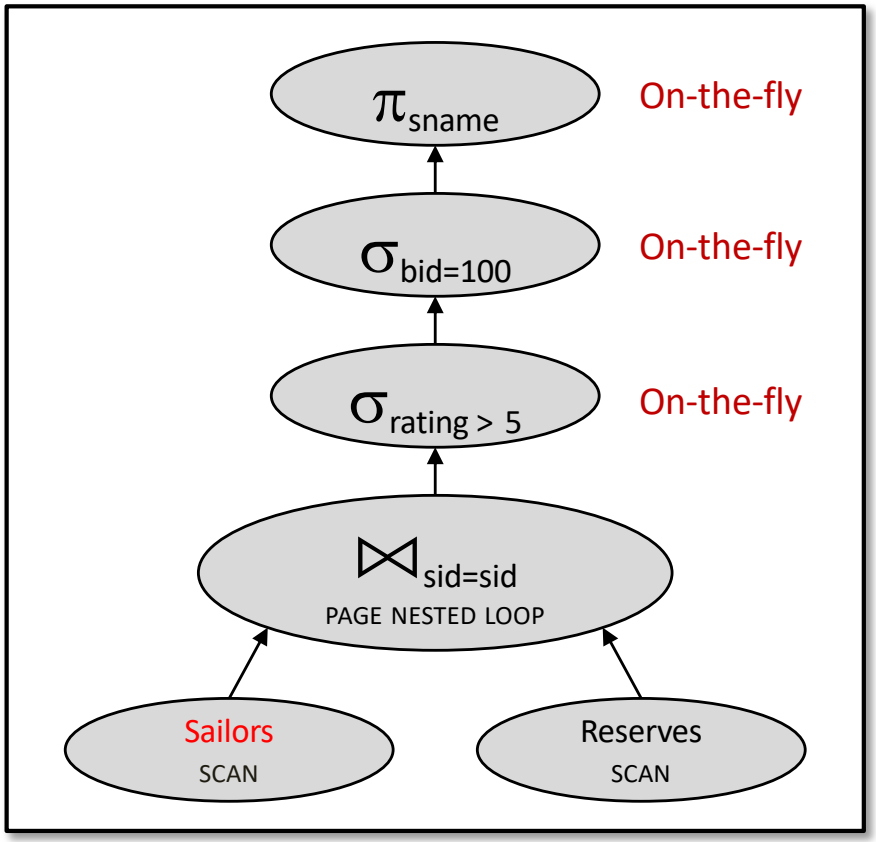
- Scan Sailors (500 IOs)
- For each pageful of high-rated Sailors,
  Scan Reserves (1000 IOs)

- Total: 500 + ???*1000

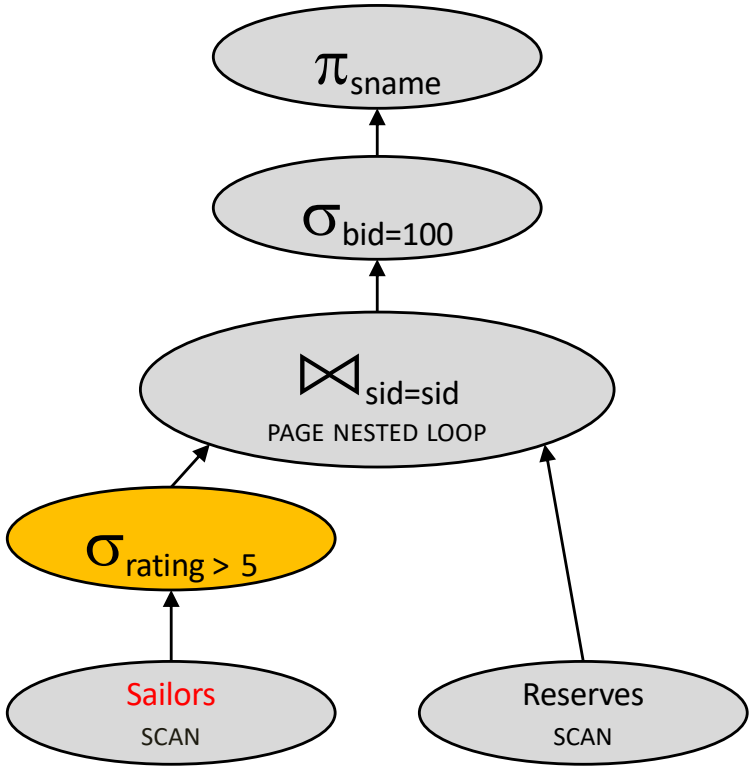


Cost?

Selection pushdown



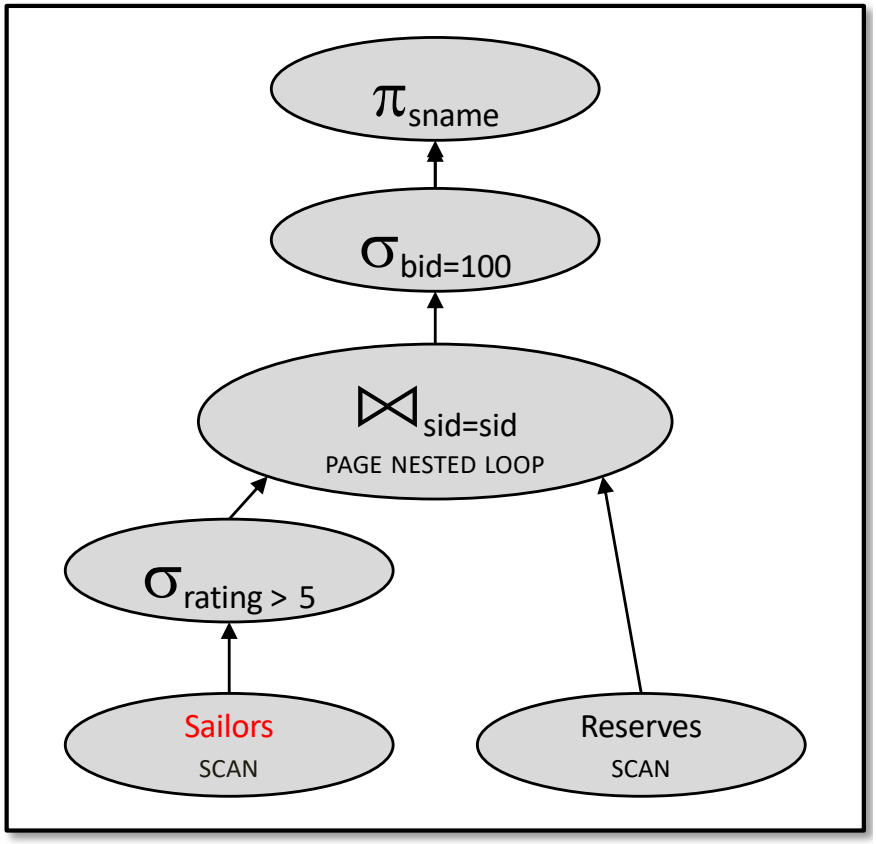
500,500 IOs



250,500 IOs

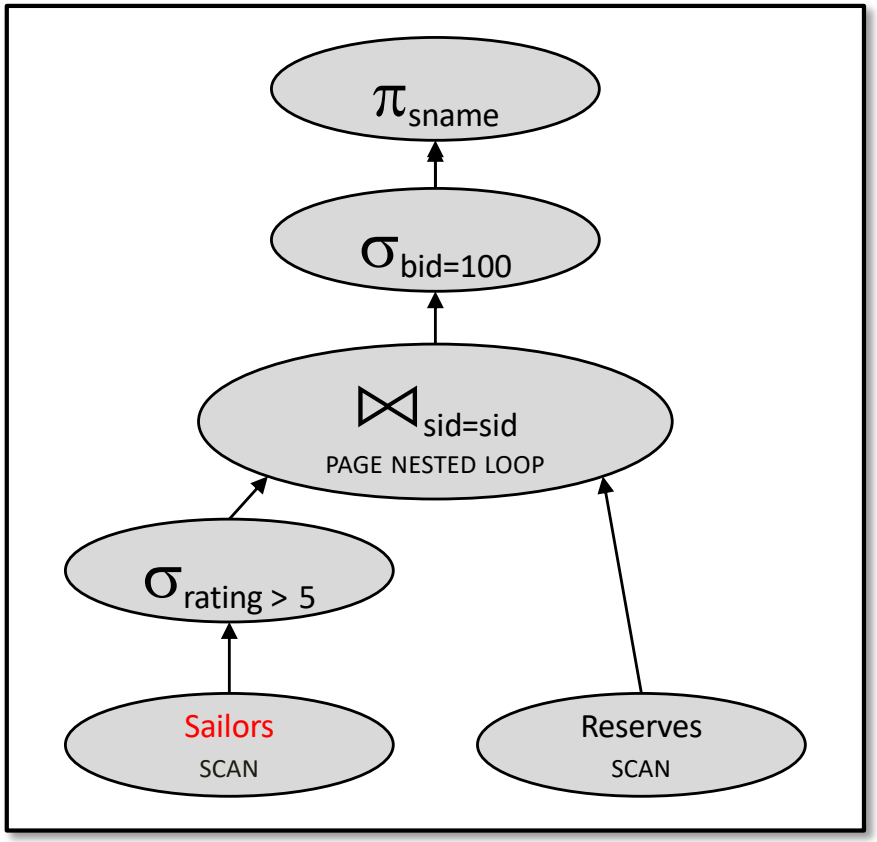
Selection pushdown

500,500 IOs



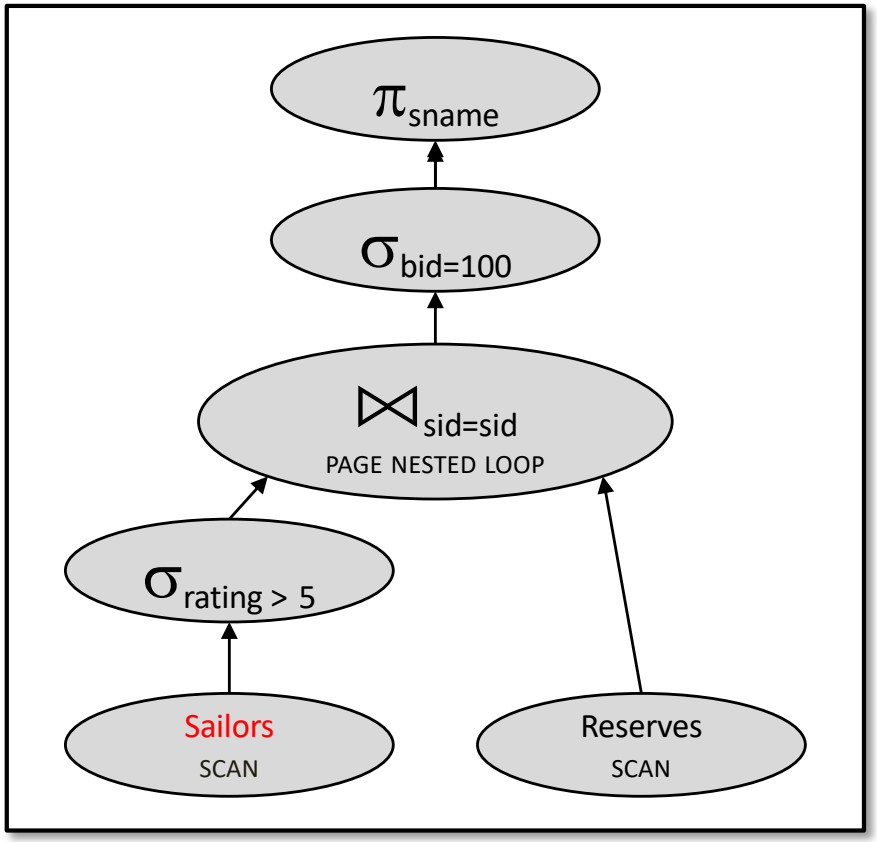
250,500 IOs

More selection pushdown

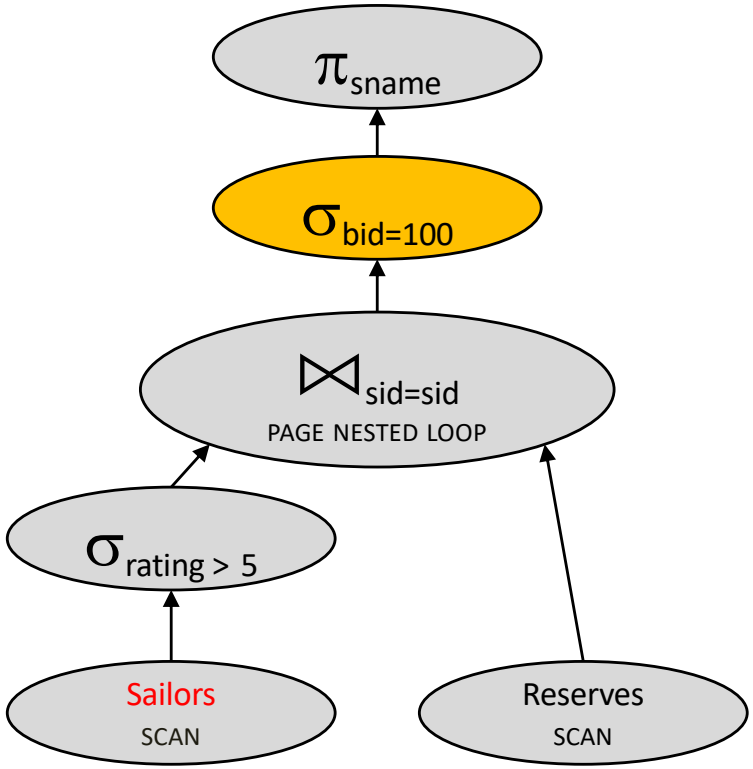


250,500 IOs

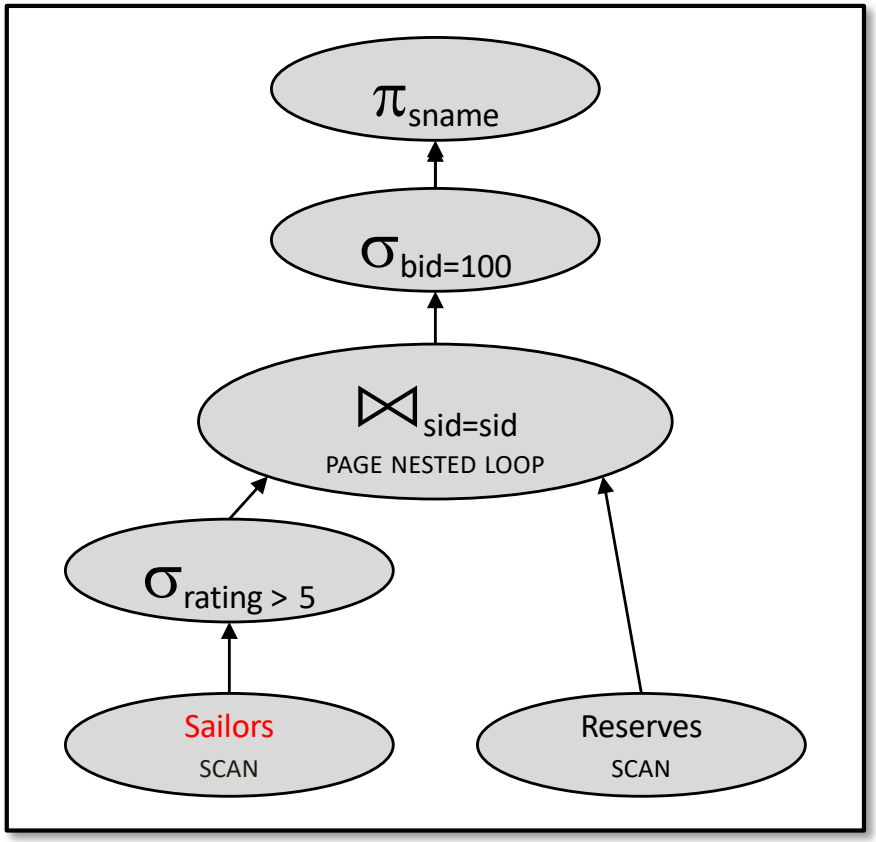
More selection pushdown



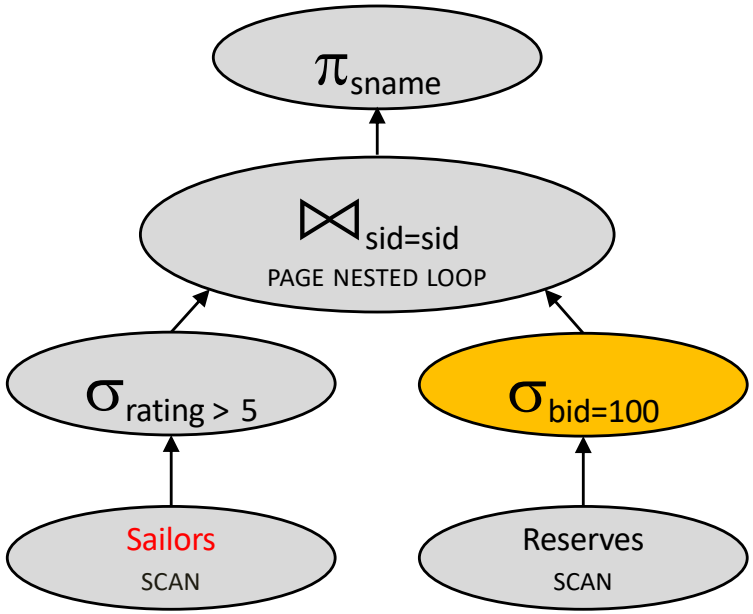
250,500 IOs



More selection pushdown



250,500 IOs



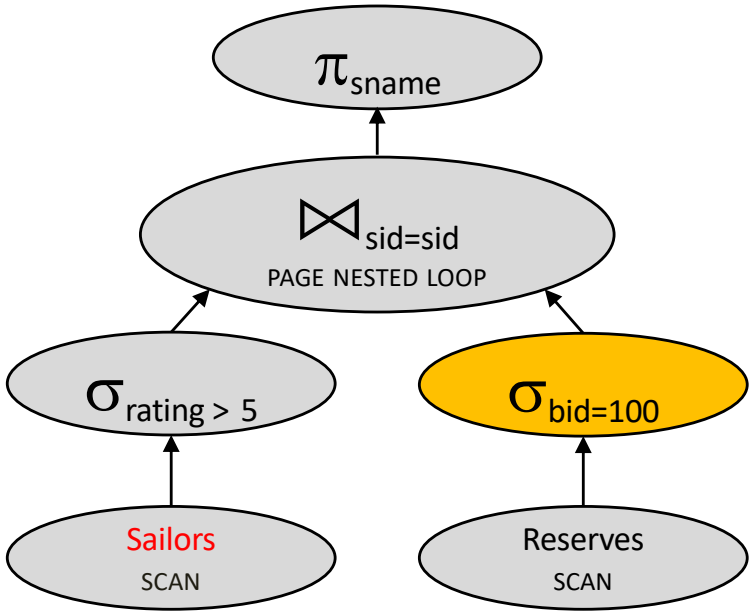
Cost???

Quick quizz

Let's estimate the cost:

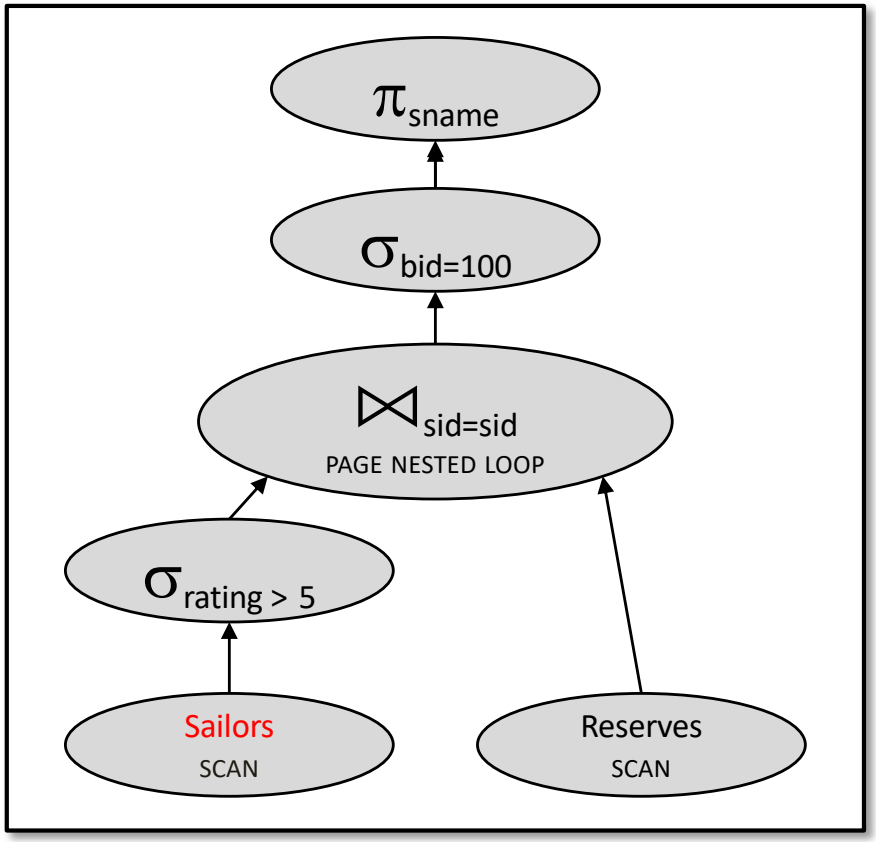
- Scan Sailors (500 IOs)
- For each pageful of high-rated Sailors,
  Do what? (??? IOs)

- Total: 500 + 250*???



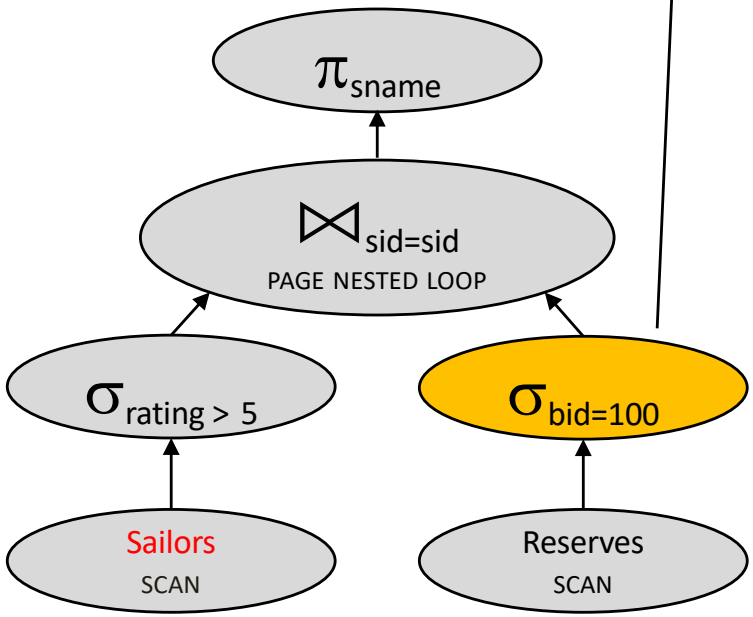
Cost?

More selection pushdown



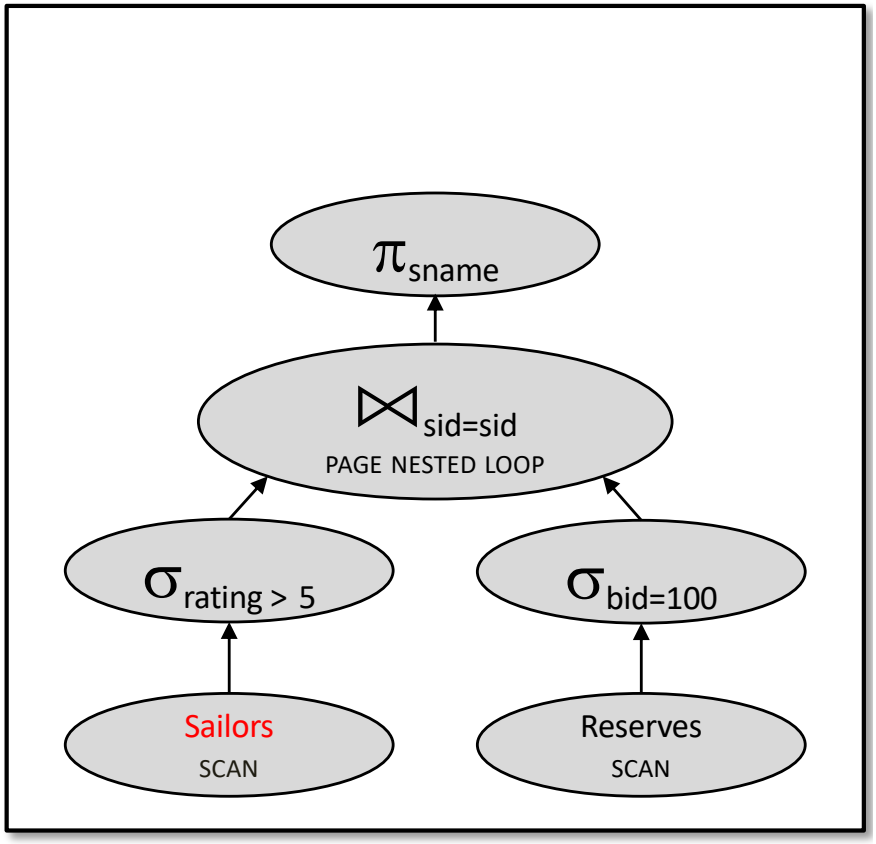
250,500 IOs

Pushing a selection into the inner loop of a nested loop join doesn't save I/Os! Essentially equivalent to having the selection above.



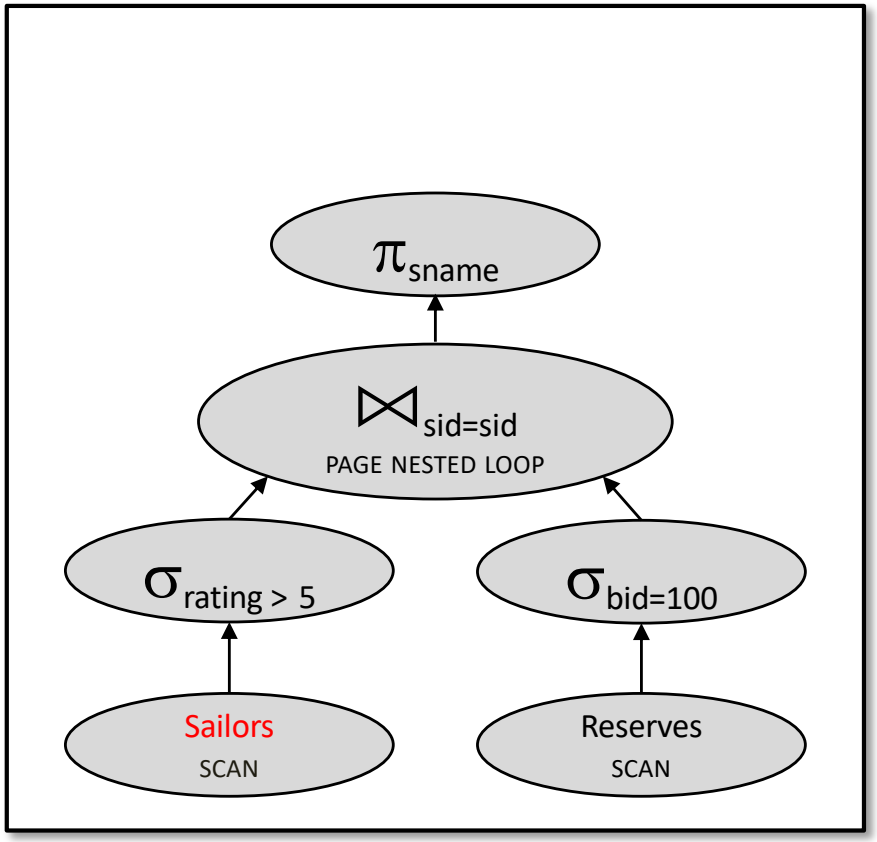
250,500 IOs

More selection pushdown



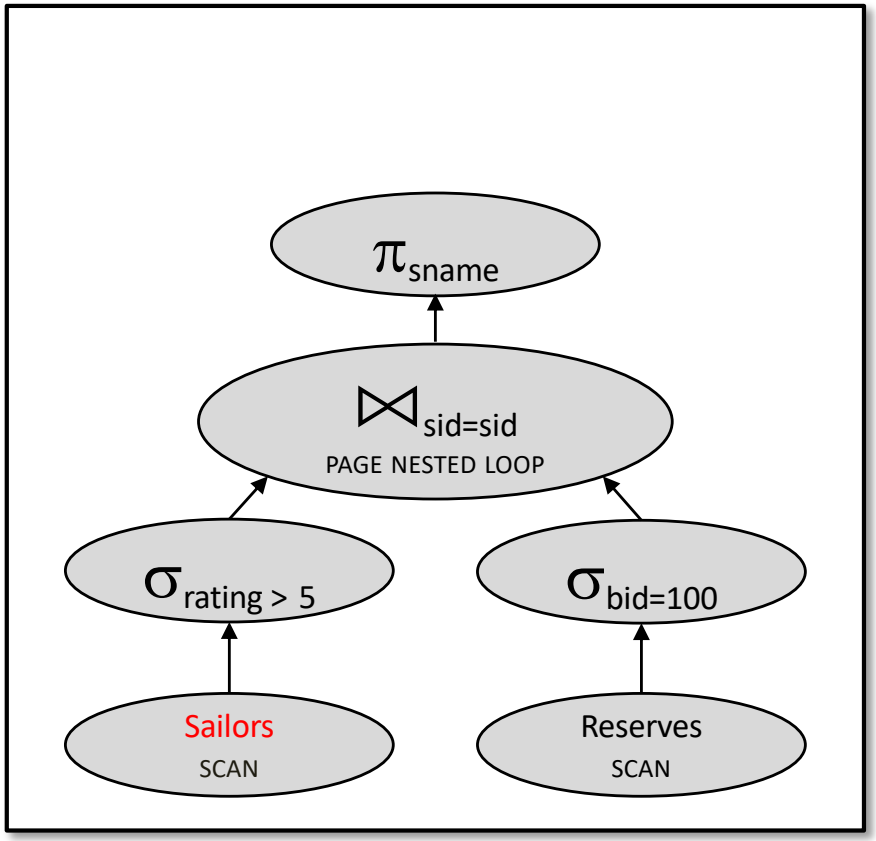
250,500 IOs

Join ordering

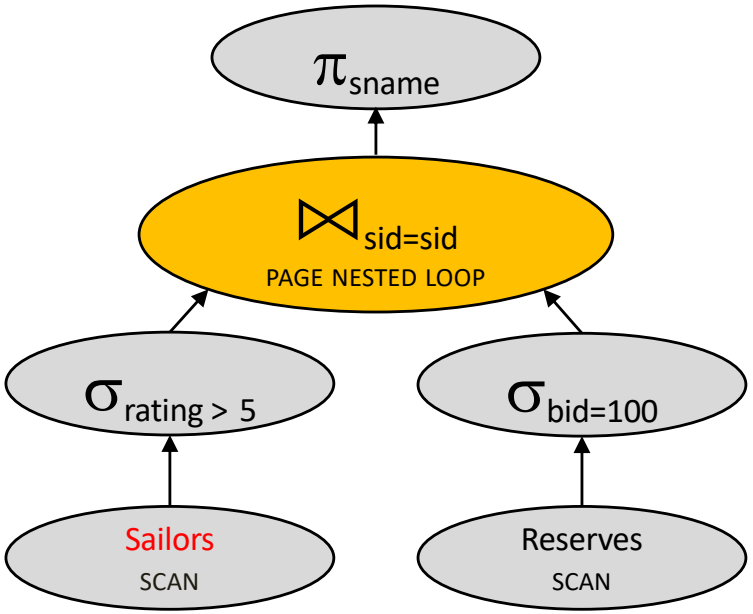


250,500 IOs

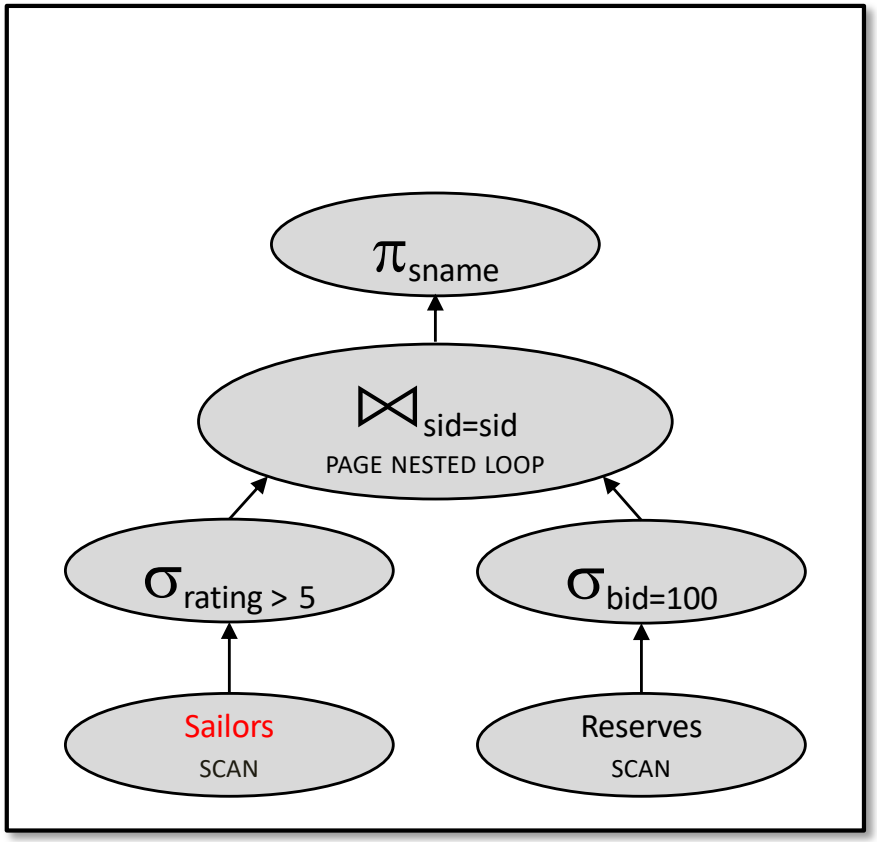
Join ordering



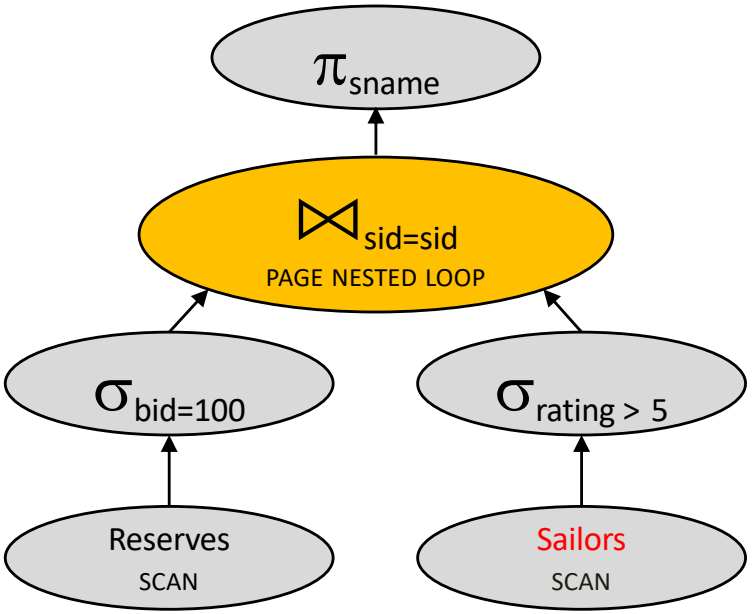
250,500 IOs



Join ordering



250,500 IOs



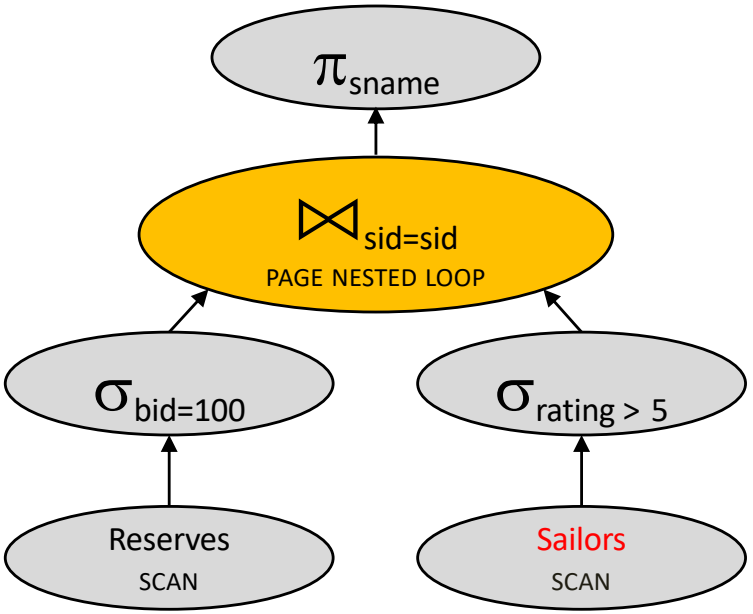
Cost???

Quick quizz

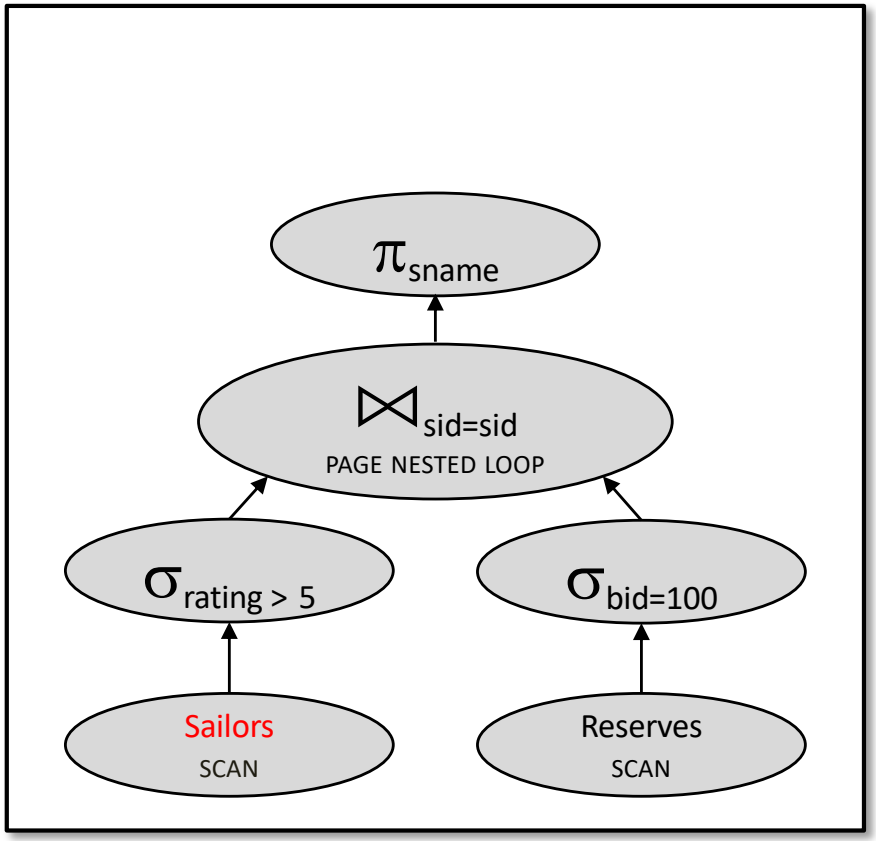
Let's estimate the cost:

- Scan Reserves (1000 IOs)
- For each pageful of Reserves for bid 100,
  Scan Sailors (500 IOs)

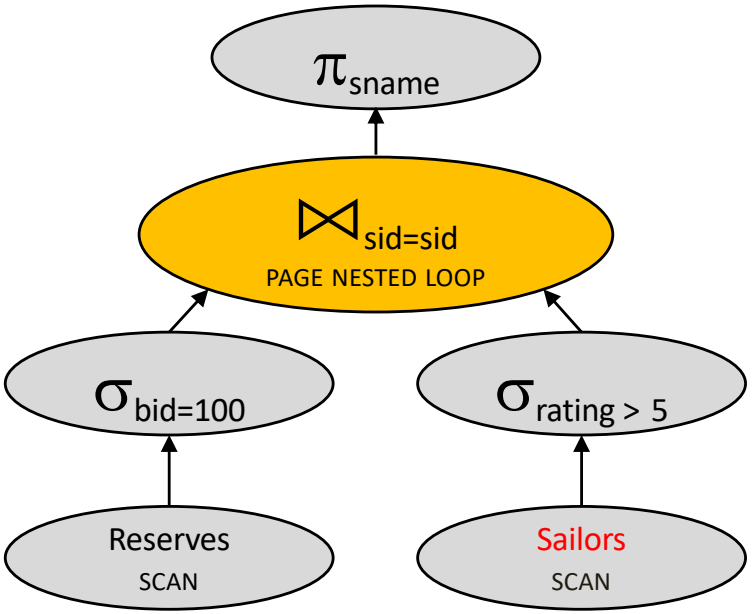
- Total: 1000 +???*500



Join ordering



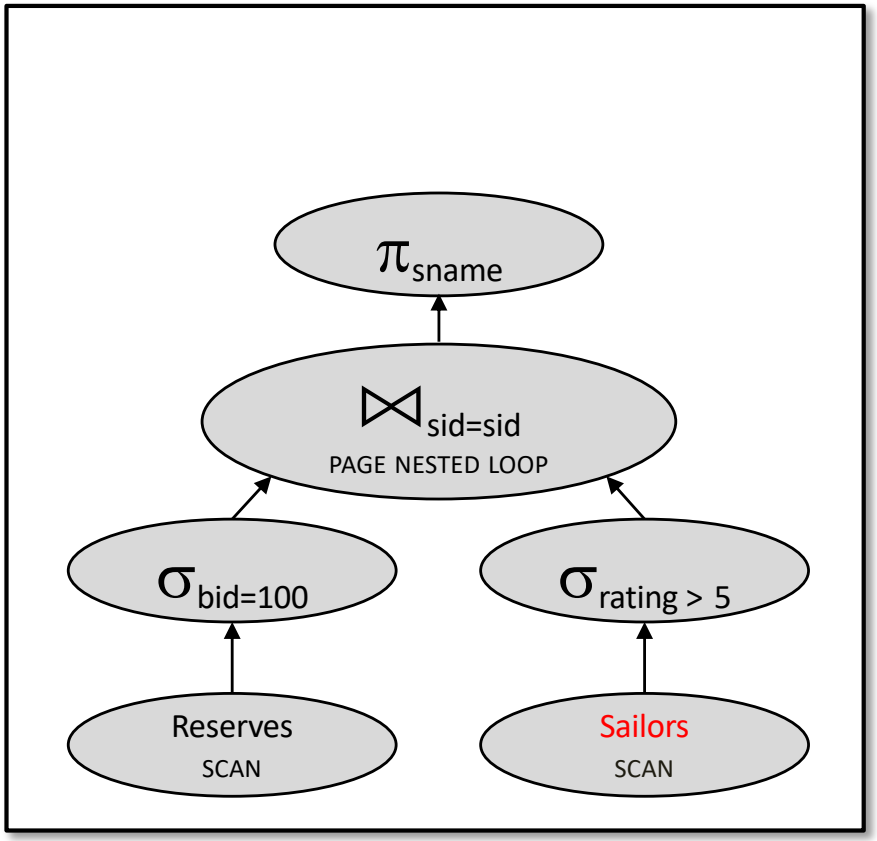
250,500 IOs



6000 IOs

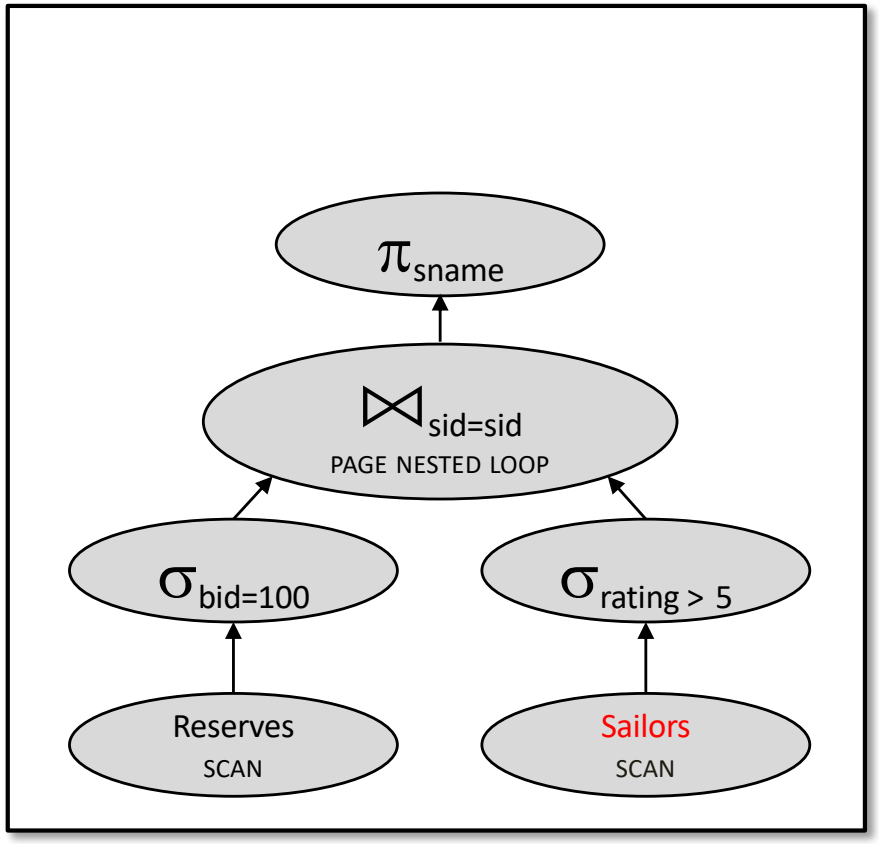
Join ordering

250,500 IOs



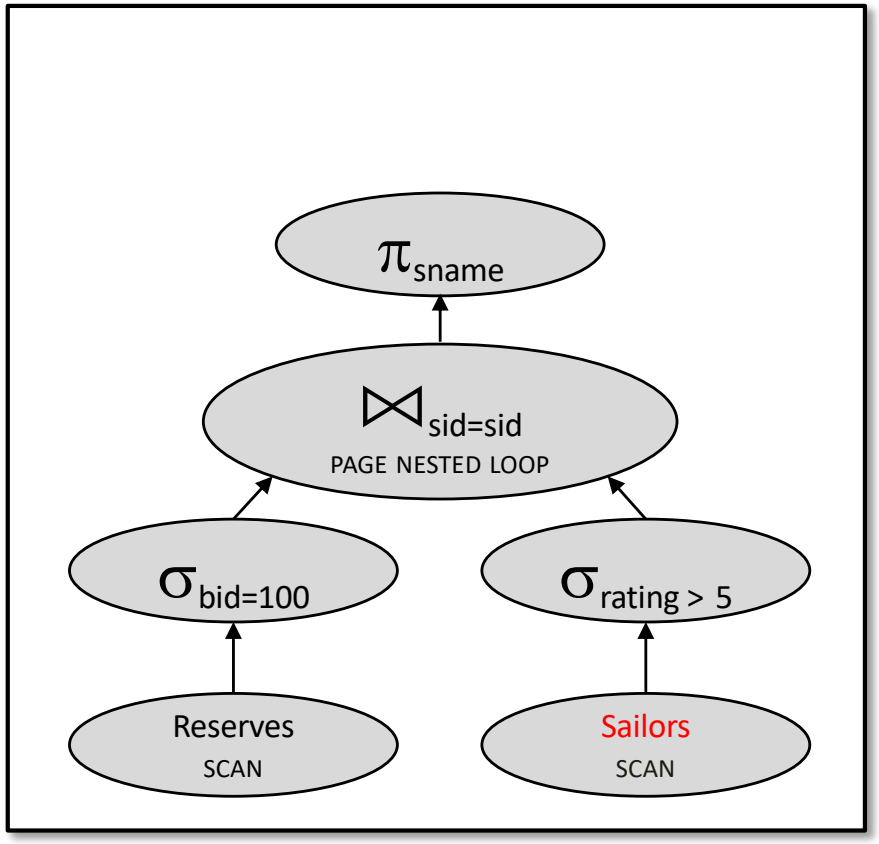
6000 IOs

Materializing inner loops

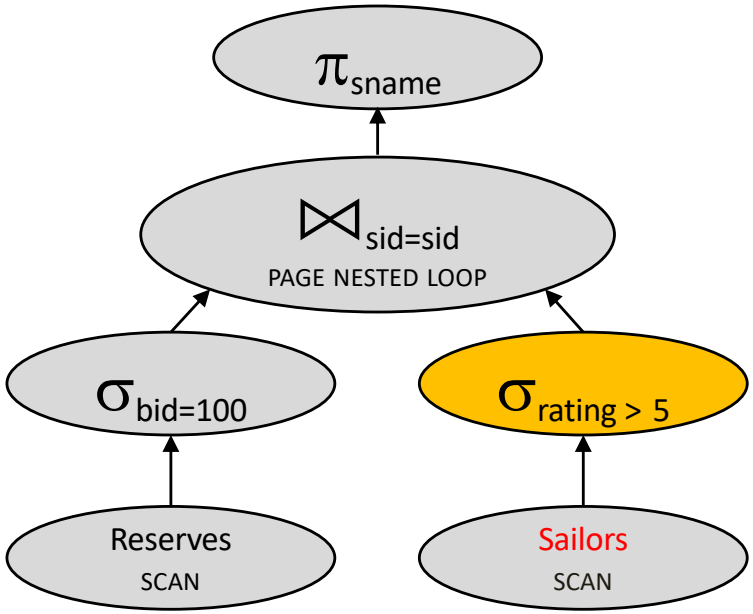


6000 IOs

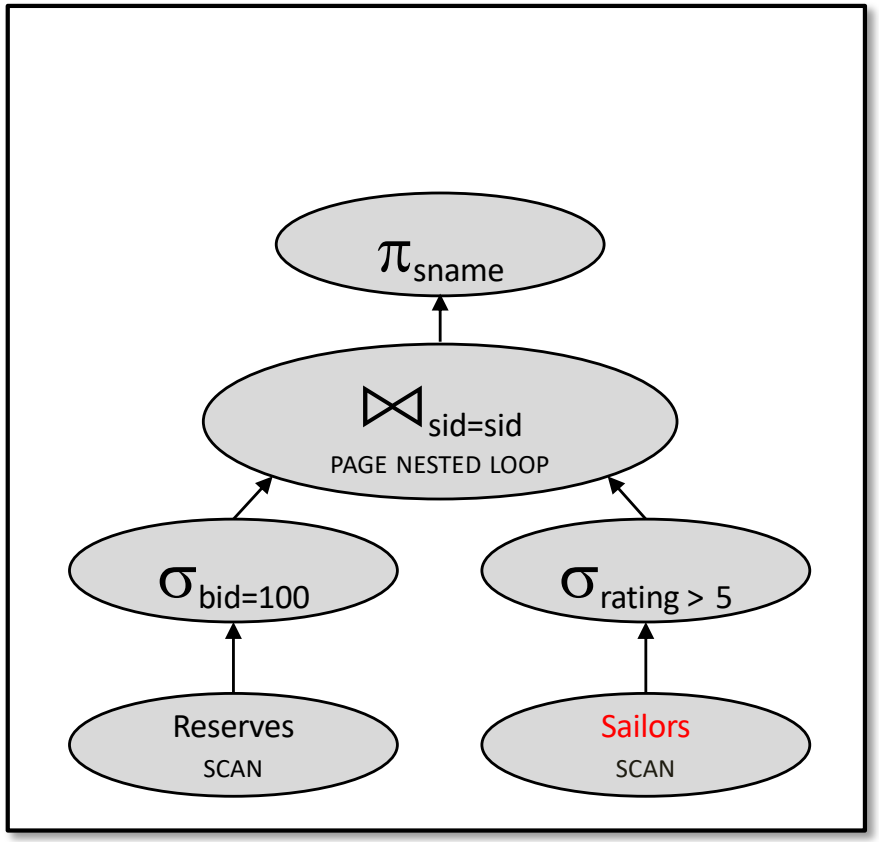
Materializing inner loops



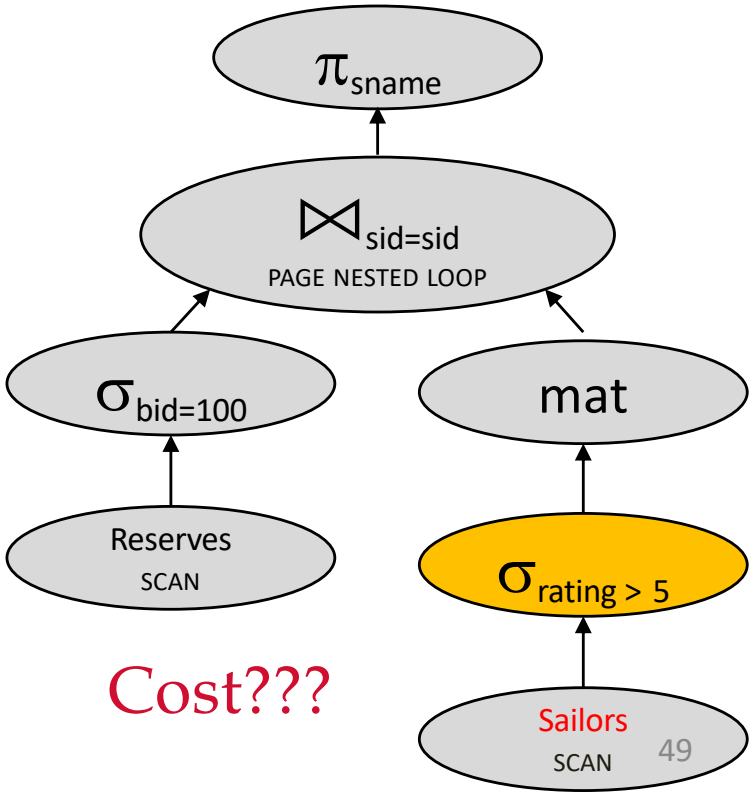
6000 IOs



Materializing inner loops



6000 IOs



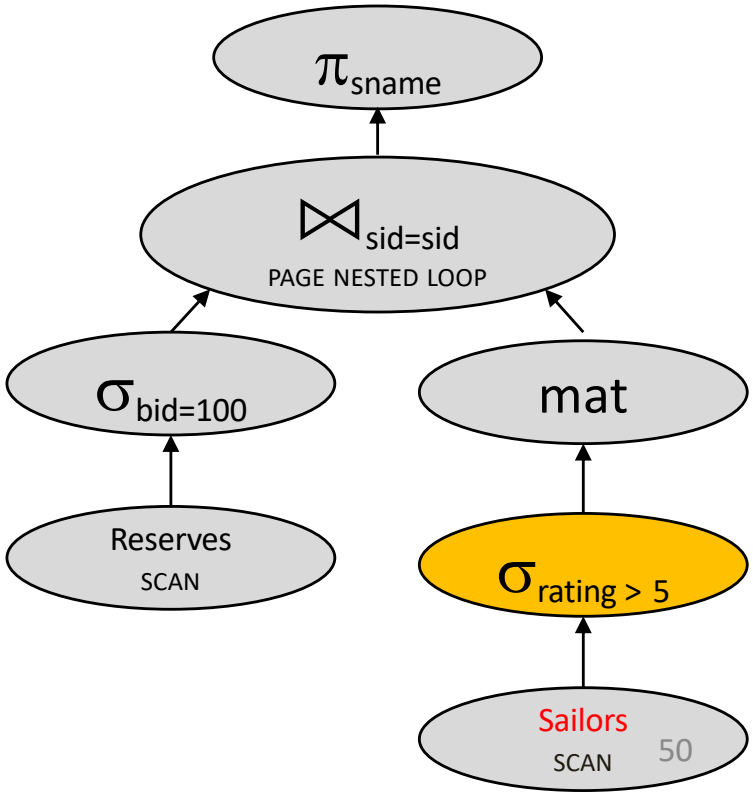
Cost???

Quick quizz

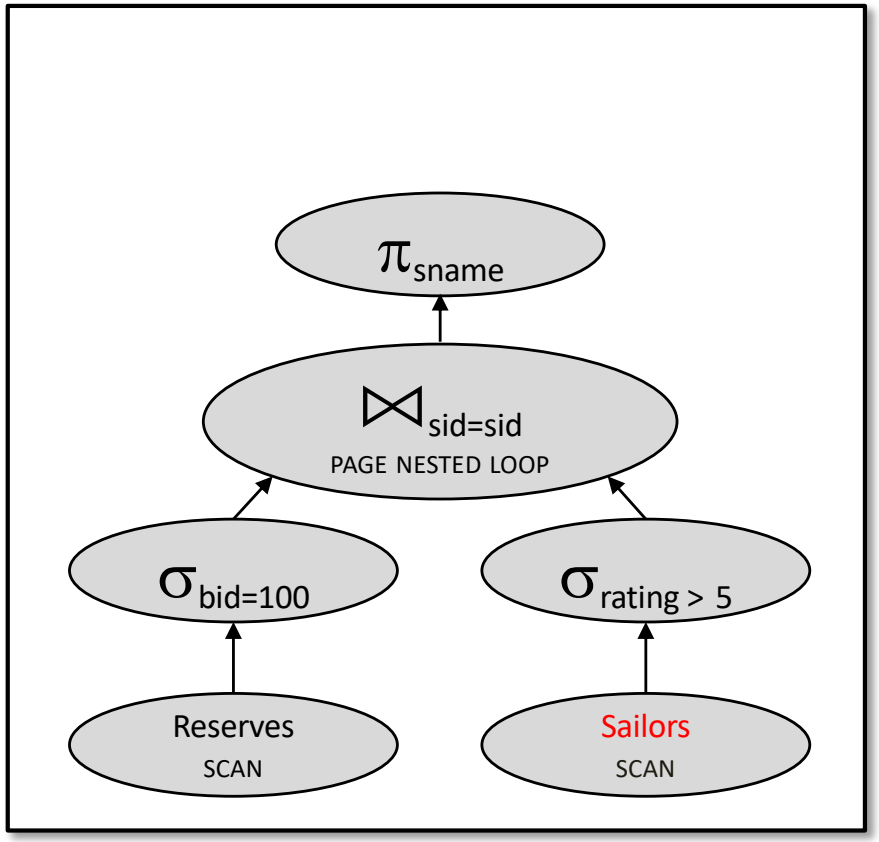
Let's estimate the cost:

- Scan Reserves (1000 IOs)
- Scan Sailors (500 IOs)
- Materialize Temp table T1 (??? IOs)
- For each pageful of Reserves for bid 100,
  Scan T1 (??? IOs)

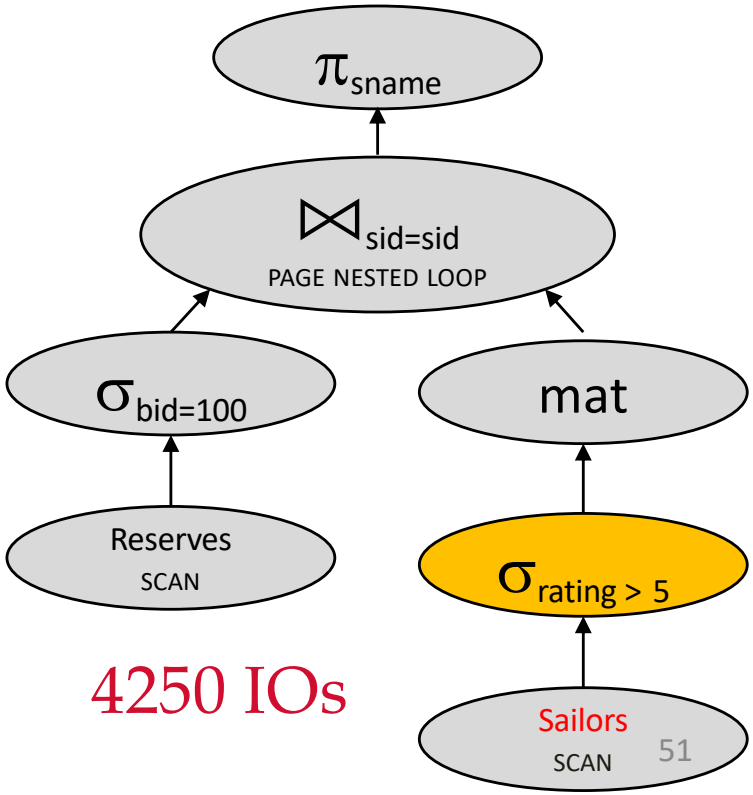
- Total: 1000 + 500 + ??? + 10*???



Materializing inner loops

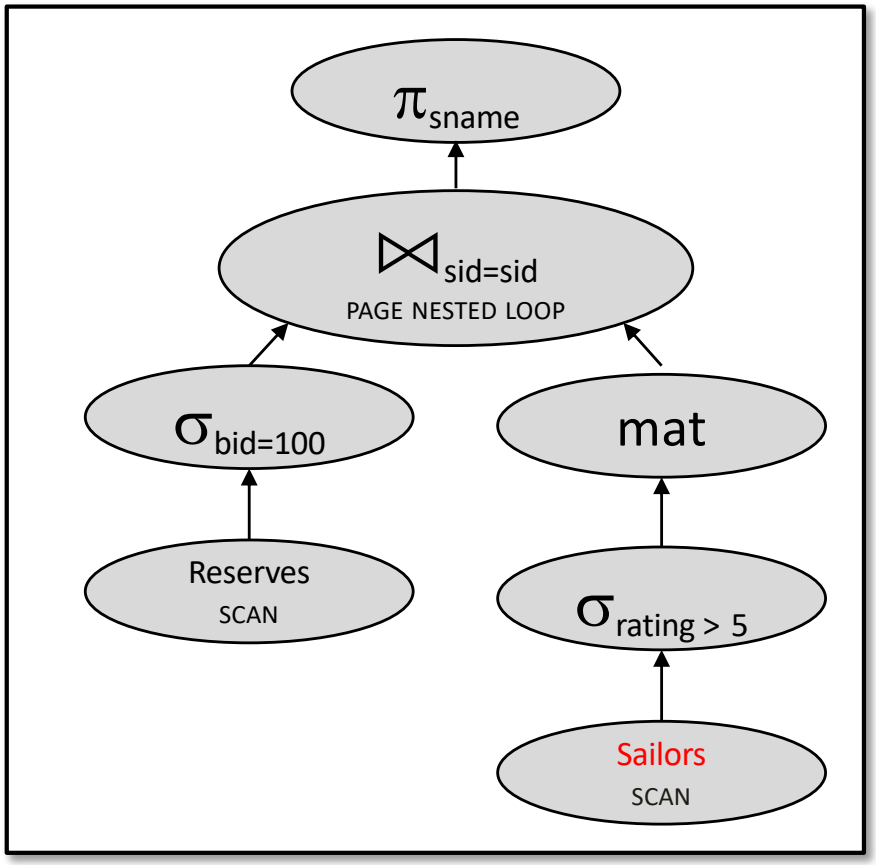


6000 IOs



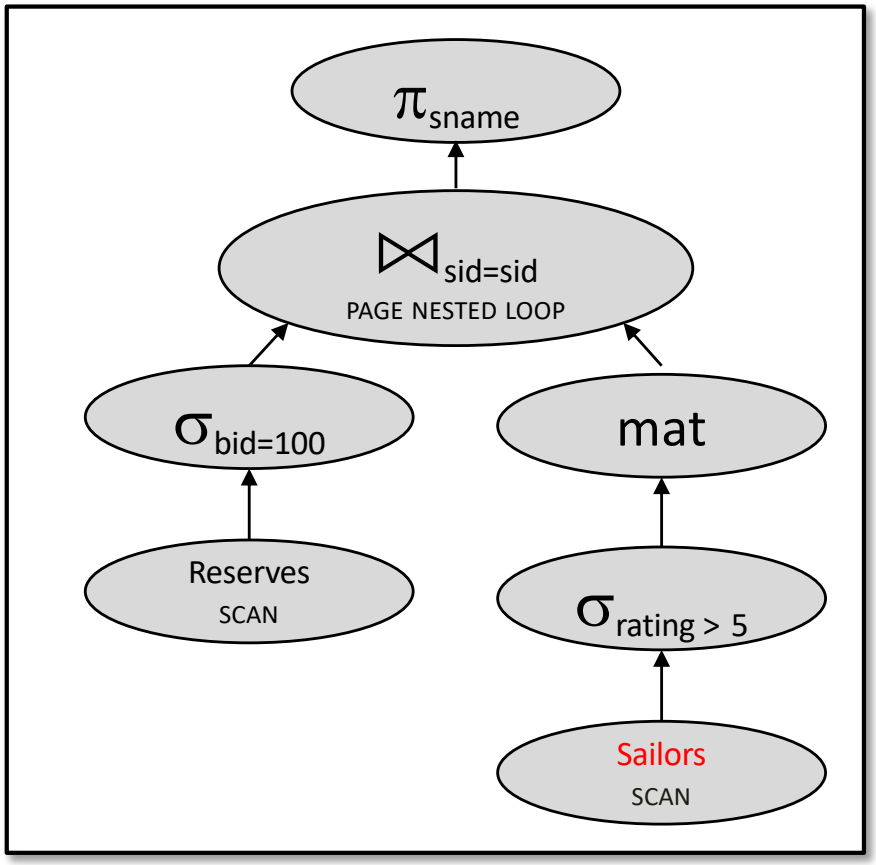
4250 IOs

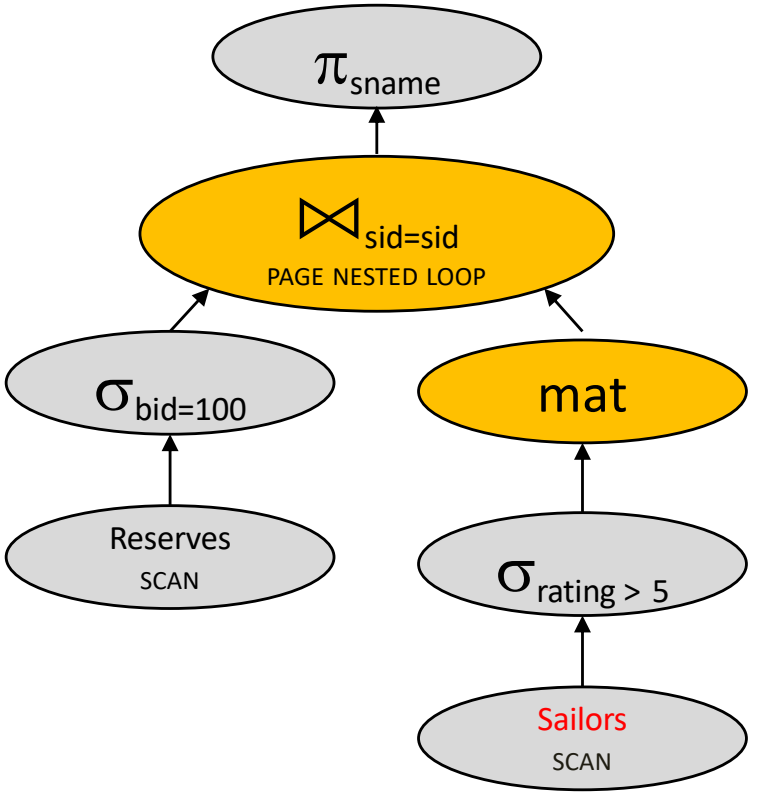
Join ordering again



4250 IOs

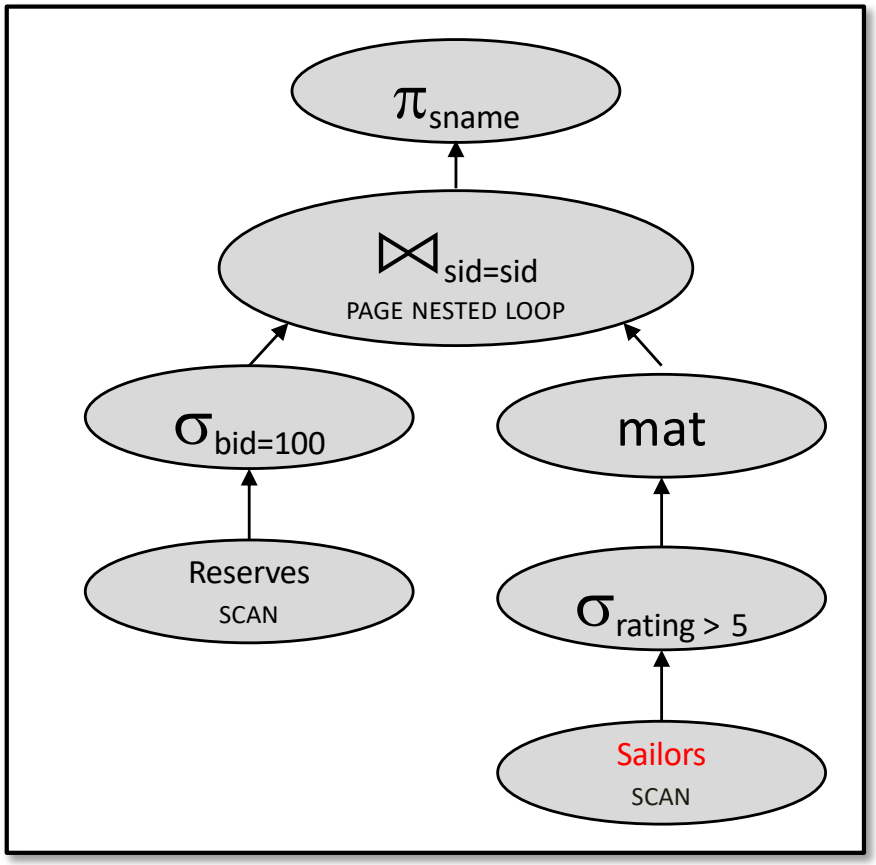
Join ordering again



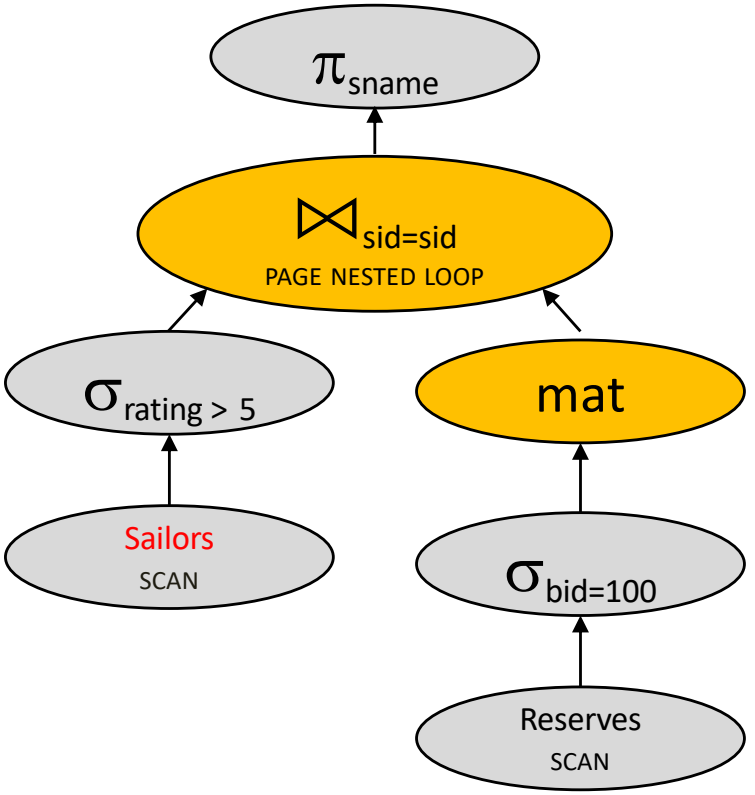


4250 IOs

Join ordering again



4250 IOs



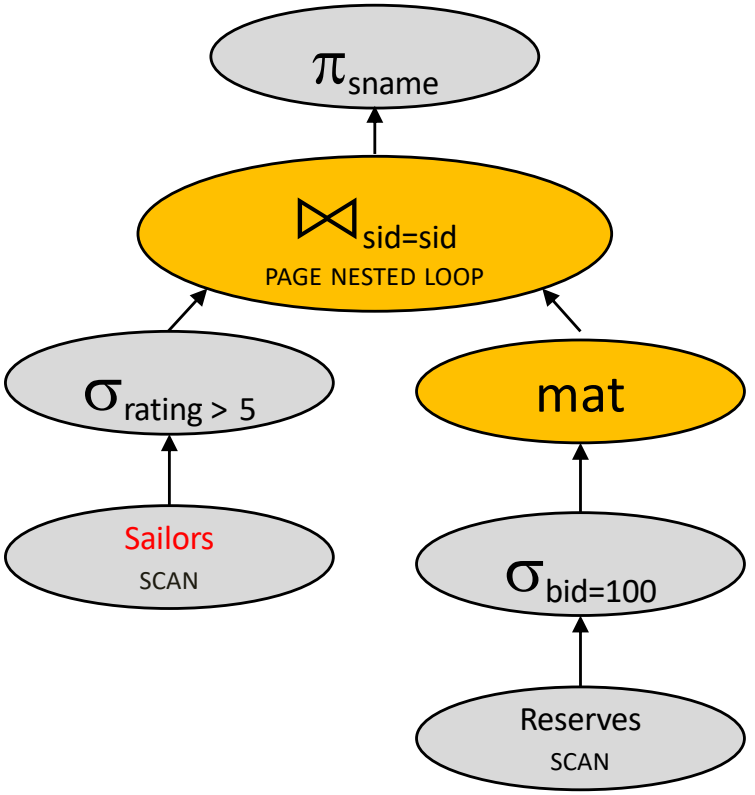
Cost???

Quick quizz

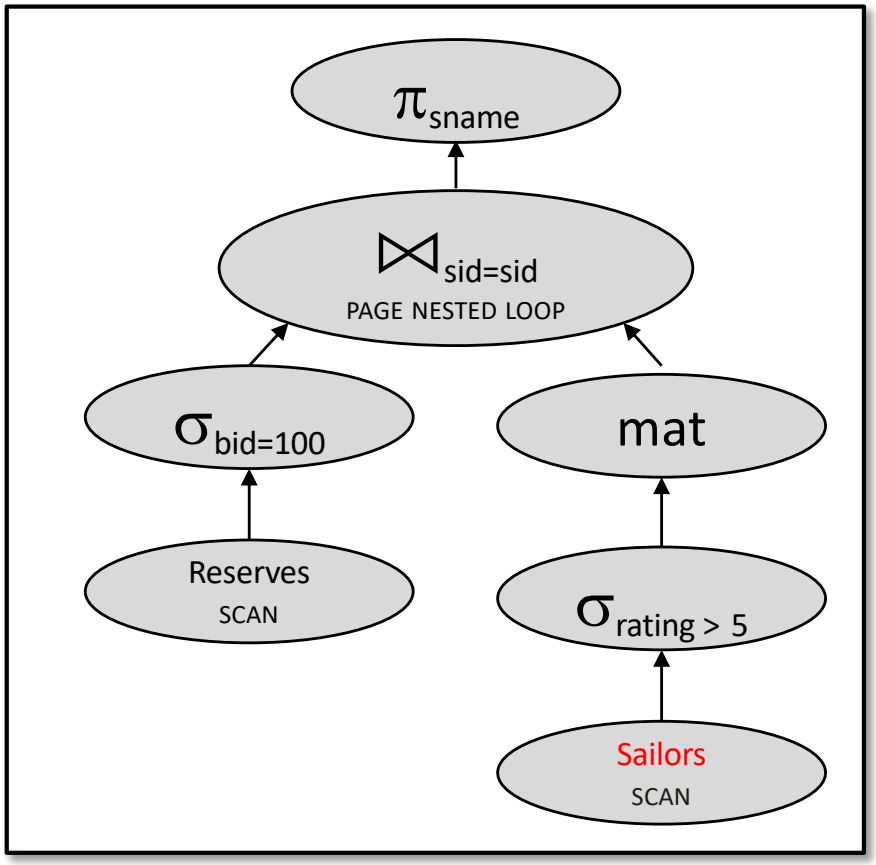
Let's estimate the cost:

- Scan Sailors (500 IOs)
- Scan Reserves (1000 IOs)
- Materialize Temp table T1 (??? IOs)
- For each pageful of high-rated Sailors,
  Scan T1 (??? IOs)

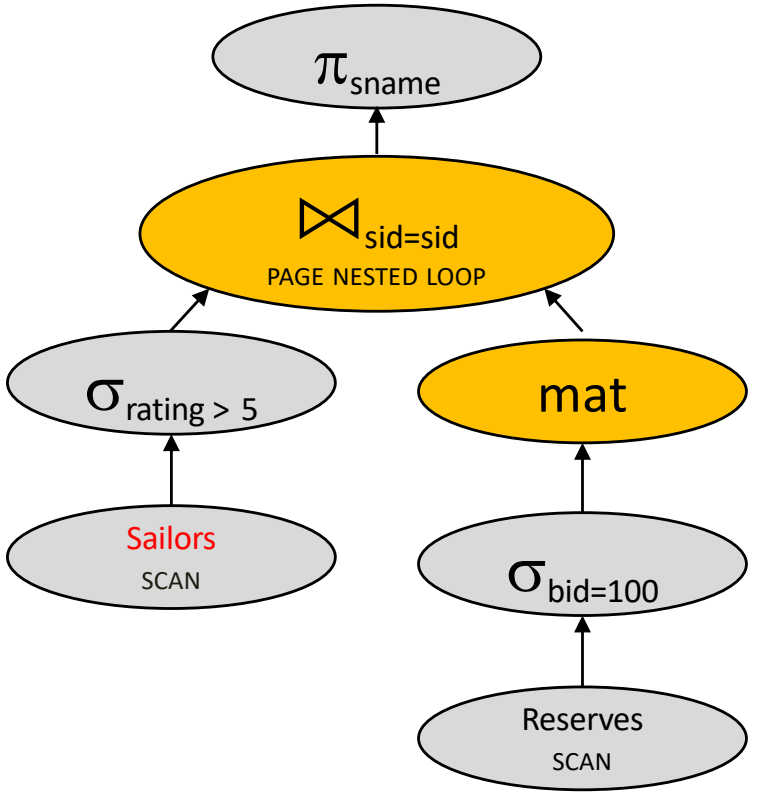
- Total: 500 + 1000 + ??? + 250*???



Join ordering again

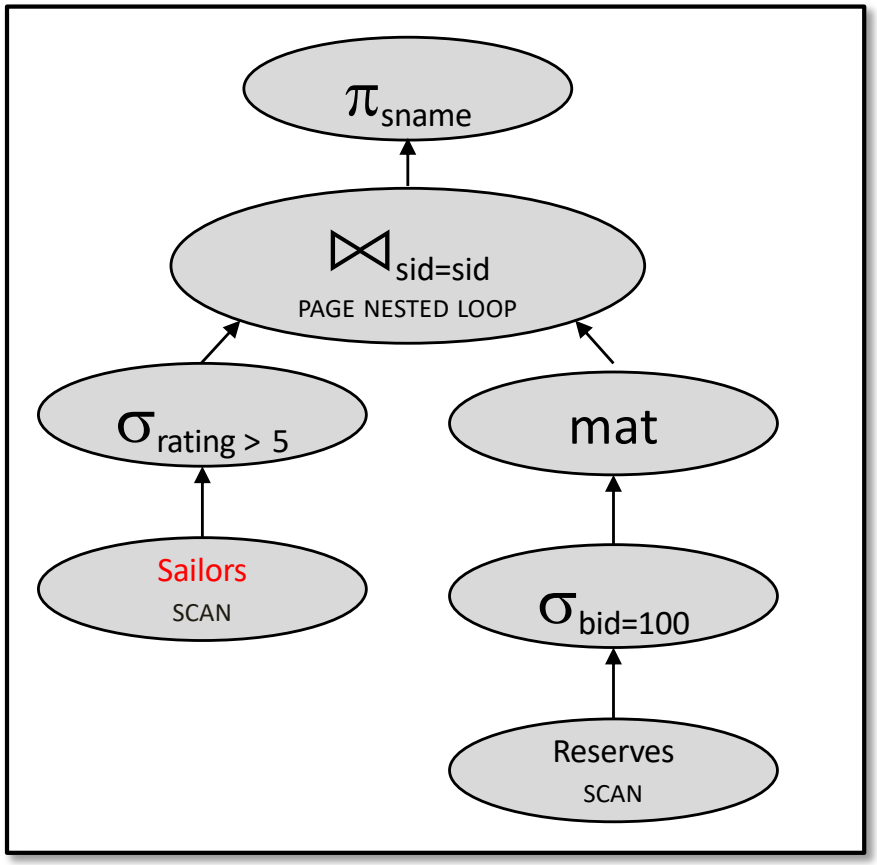


4250 IOs



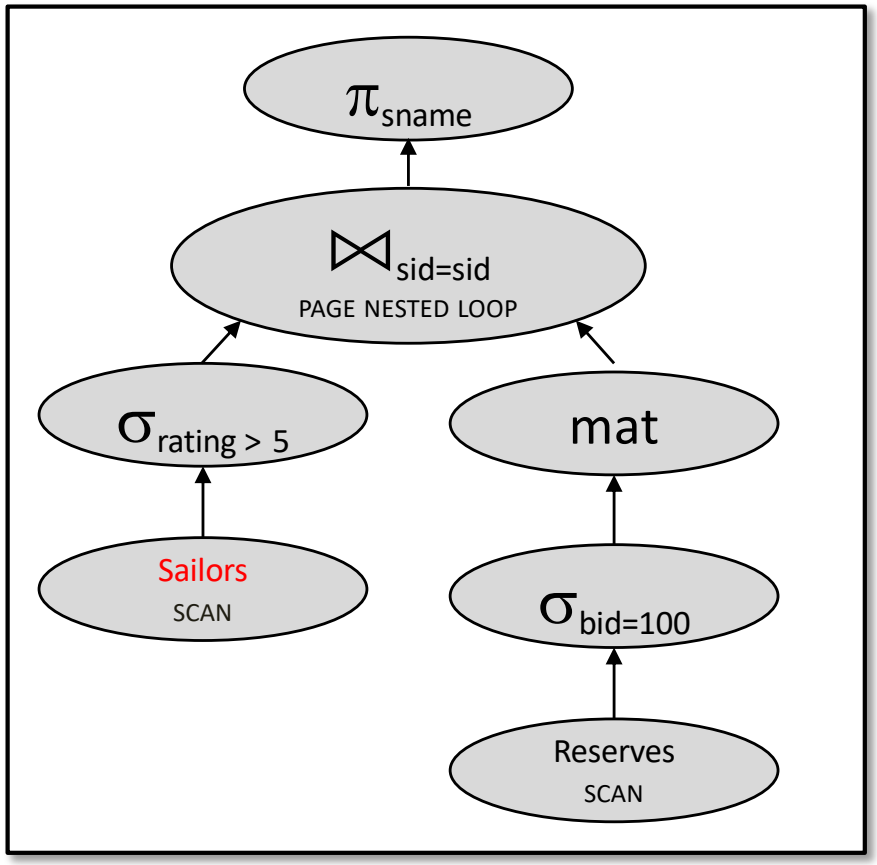
4010 IOs

Join ordering again



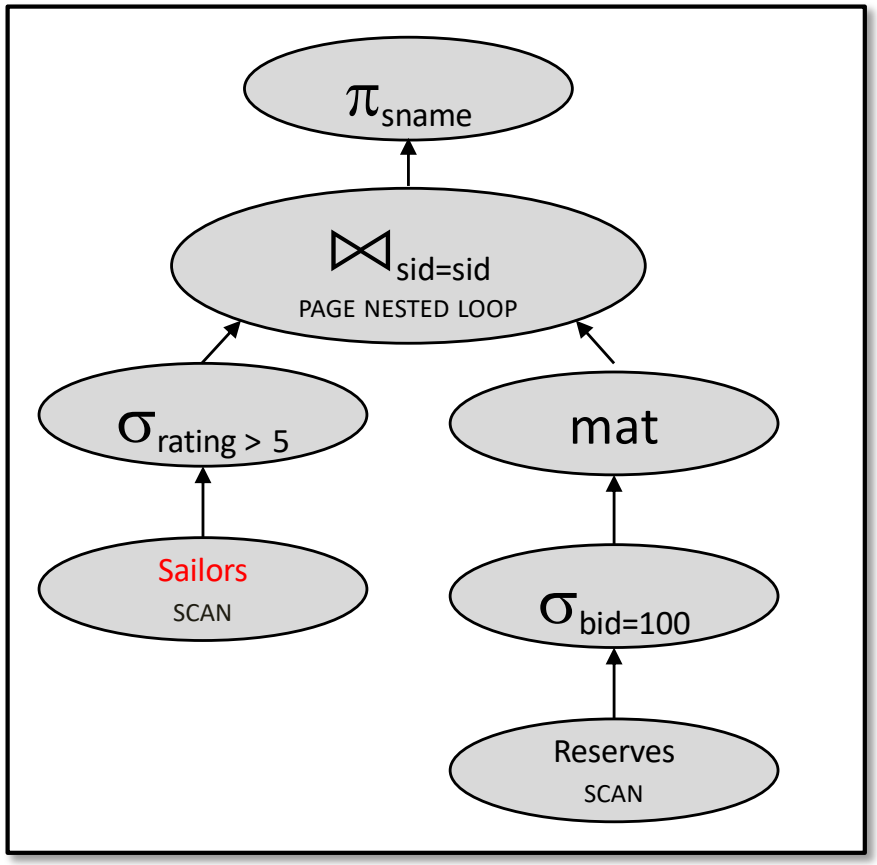
4010 IOs

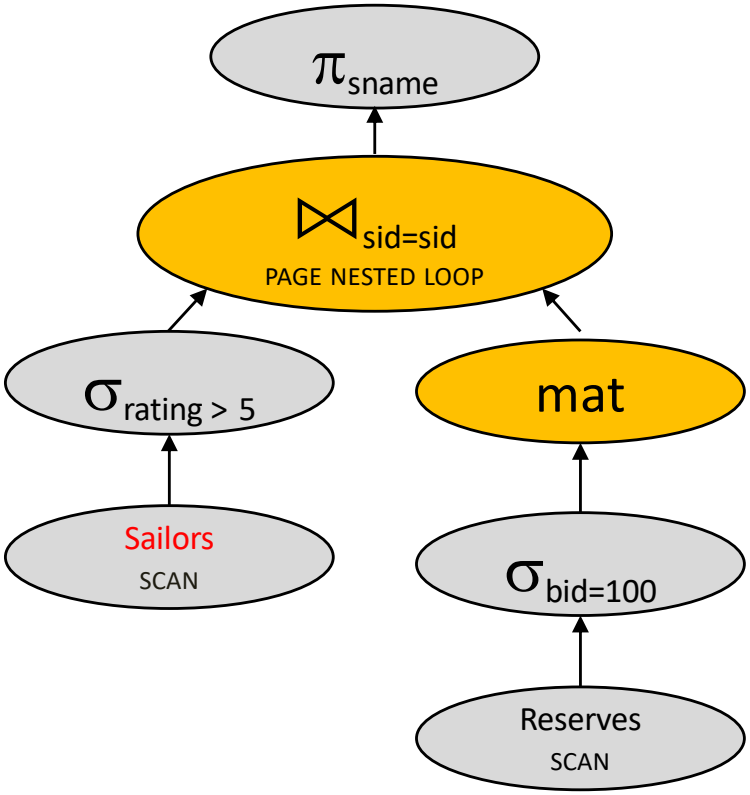
Join algorithm



4010 IOs

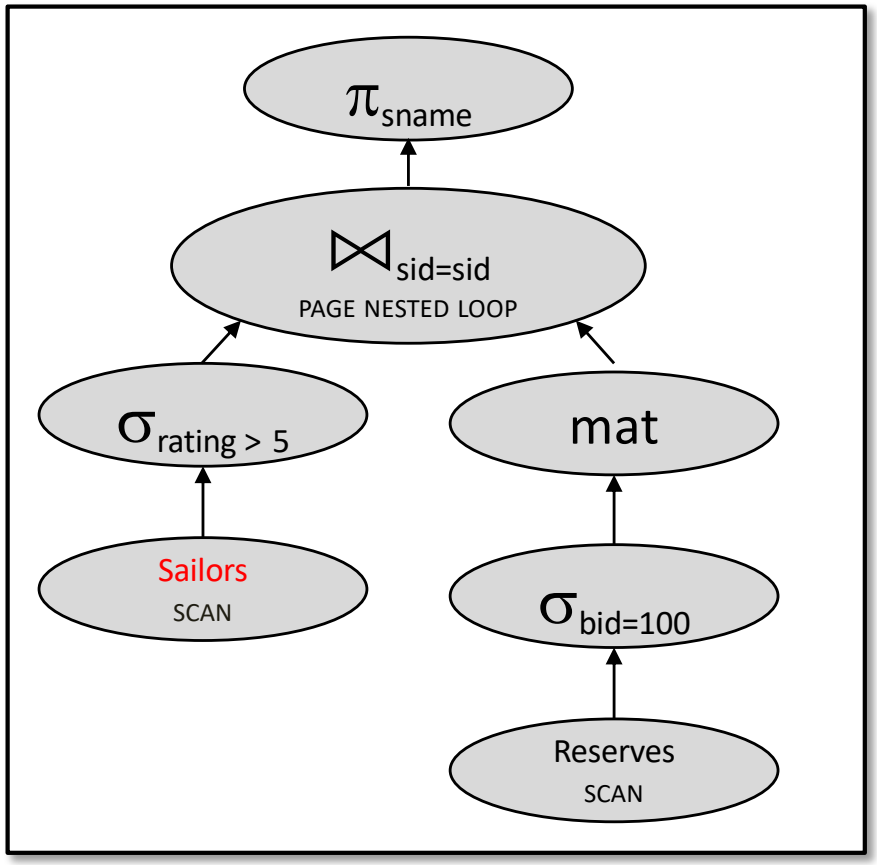
Join algorithm



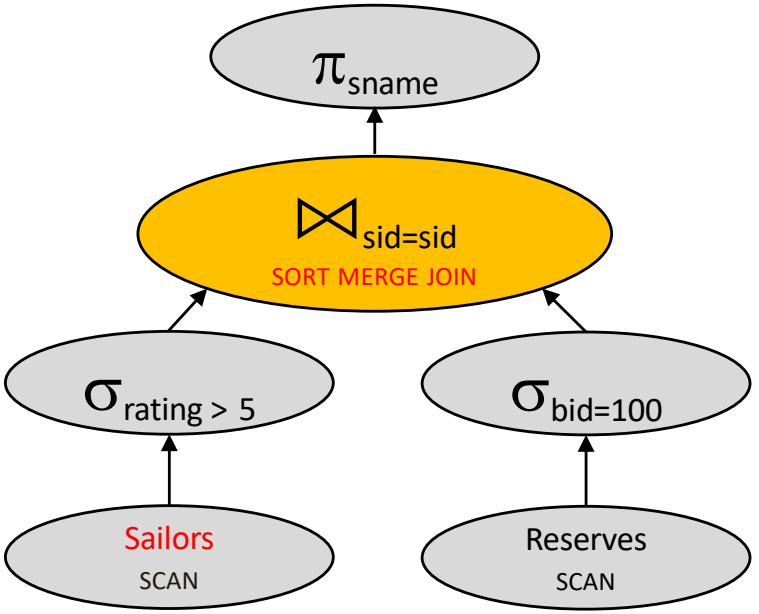


4010 IOs

Join algorithm



4010 IOs



Cost???

2 passes for reserves (pass 0 = 10 to write, pass 1 = 2*10 to read/write)

4 passes for sailors (pass 0 = 250 to write, pass 1,2,3 = 2*250 to read/write)

1000 + 500 + sort reserves(10 + 2*10) + sort sailors (250 + 3*2*250) + merge (10+250) = **3630**

With 5 buffers, cost of plan:

Scan Reserves (1000)

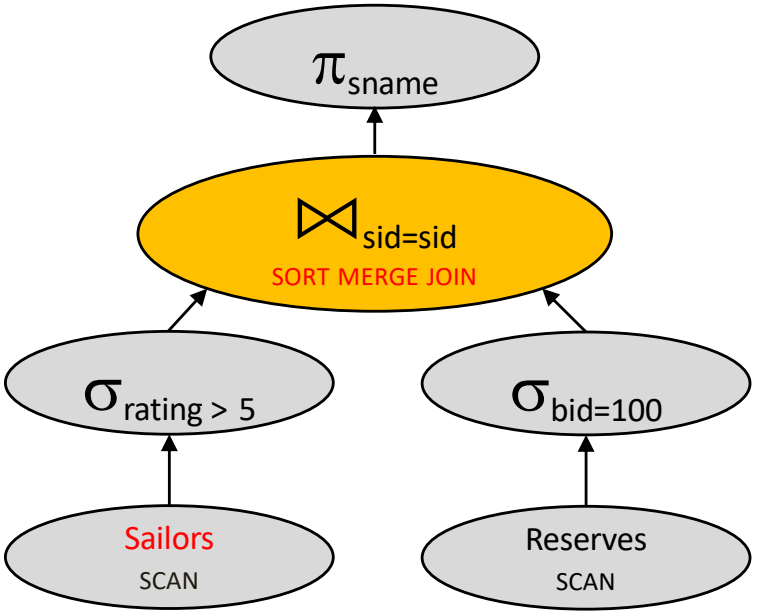
Scan Sailors (500)

Sort high-rated sailors (???)
Note: pass 0 doesn't do read I/O, just gets input from select.

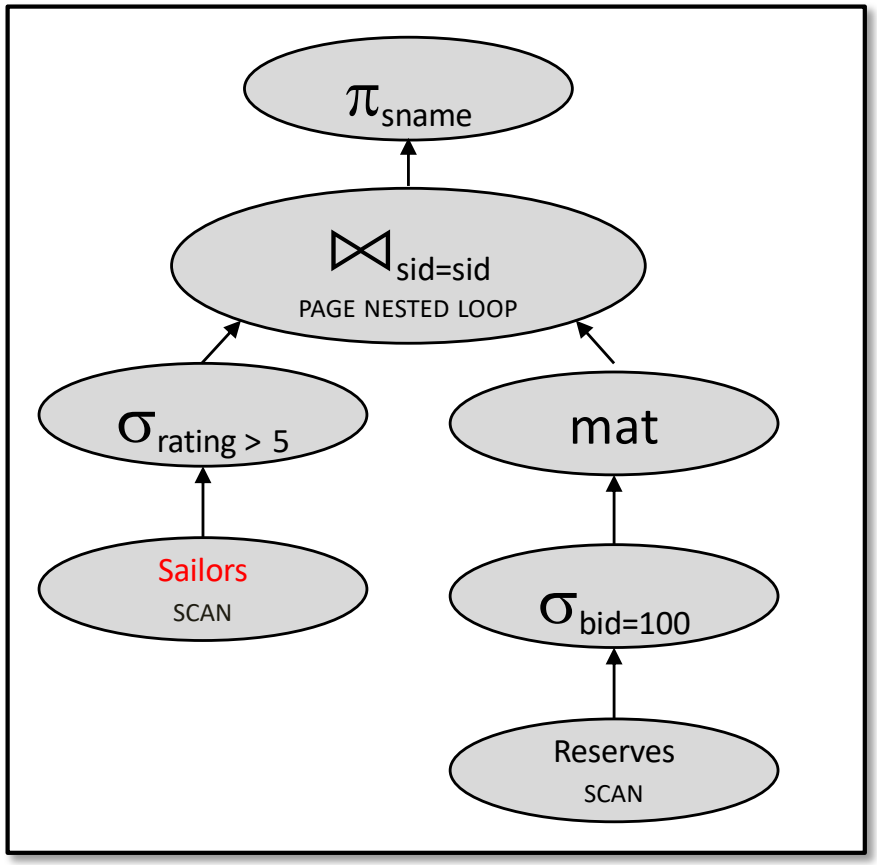
Sort reservations for boat 100 (???)
Note: pass 0 doesn't do read I/O, just gets input from select.
How many passes for each sort?

Merge (10+250) = 260

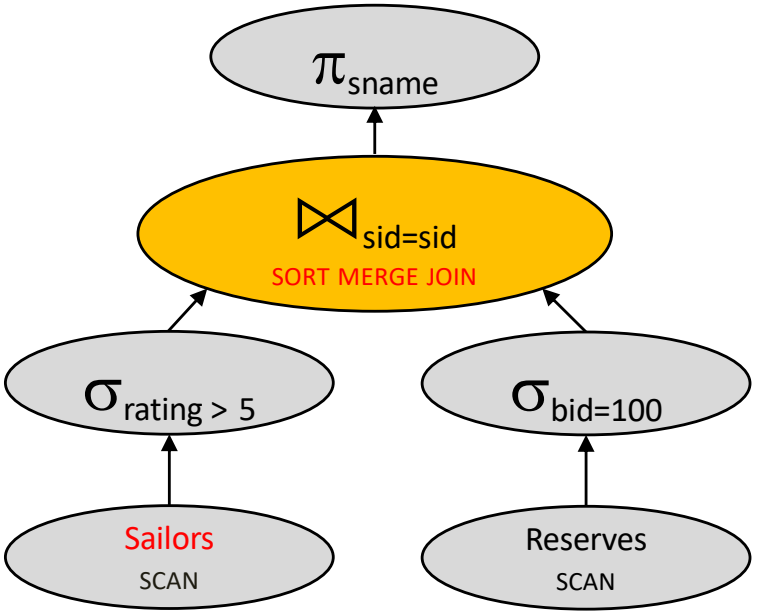
Total: 3630



Join algorithm

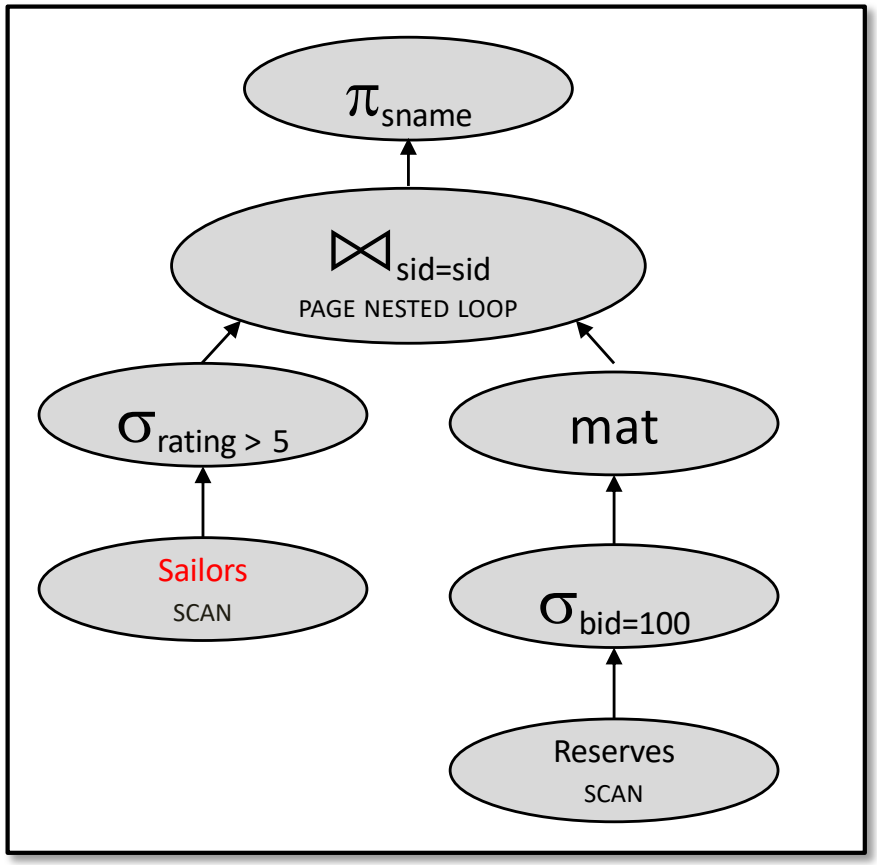


4010 IOs

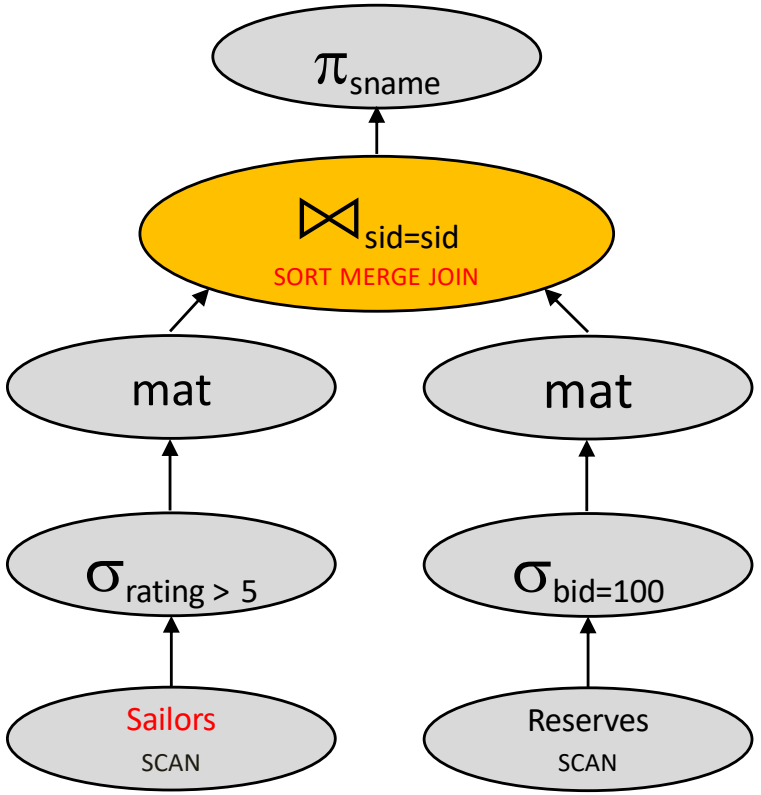


3630 IOs

Textbook considers this:



4010 IOs



2 passes for reserves (2*2*10 to read/write)

4 passes for sailors (4*2*250 to read/write)

1000 + 10 + 500 + 250 + 2*2*10 + 4*2*250 + merge (10+250) = **4060**

With 5 buffers, cost of plan:

Scan Sailors (500), write T1 (250)

Scan Reserves (1000), write T2 (10)

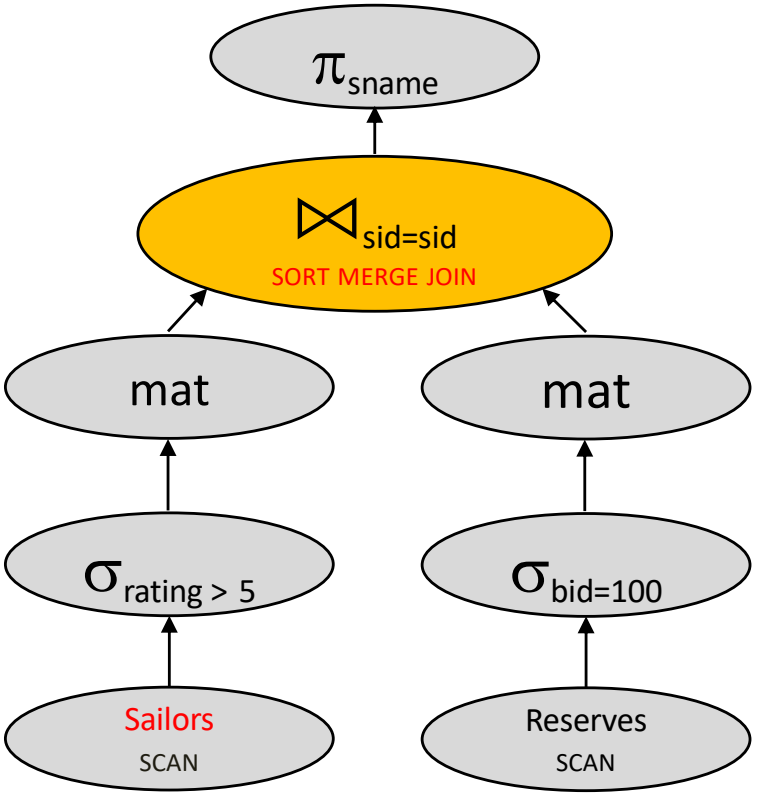
Sort T1 (???)

Sort T2 (???)

How many passes for each sort?

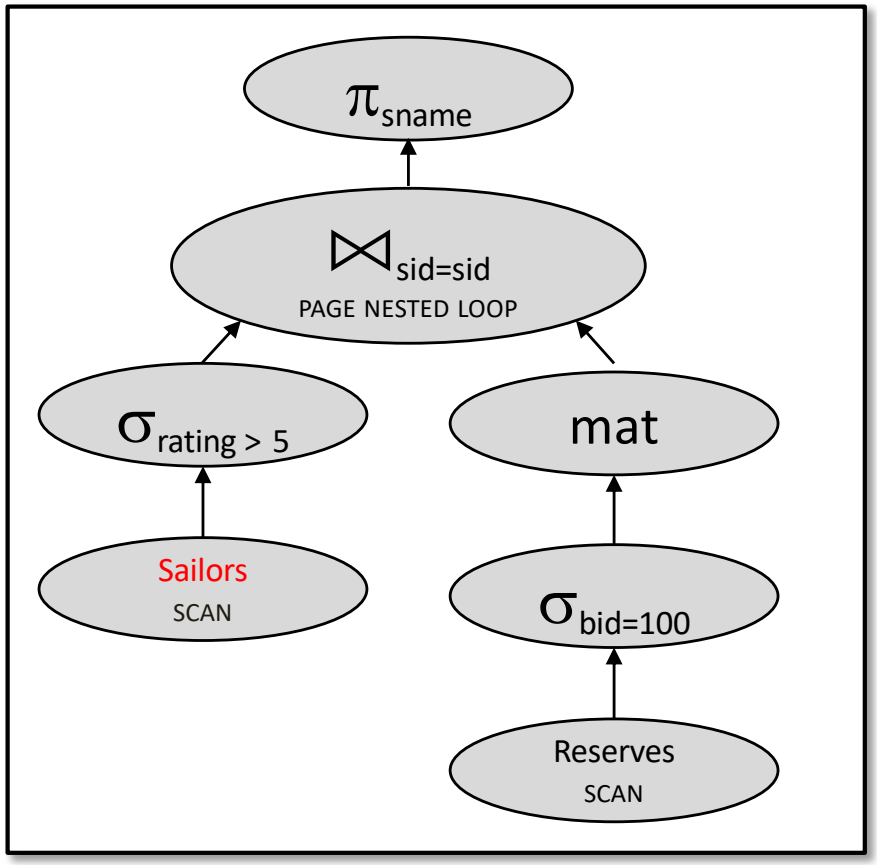
Merge (10+250) = 260

Total:



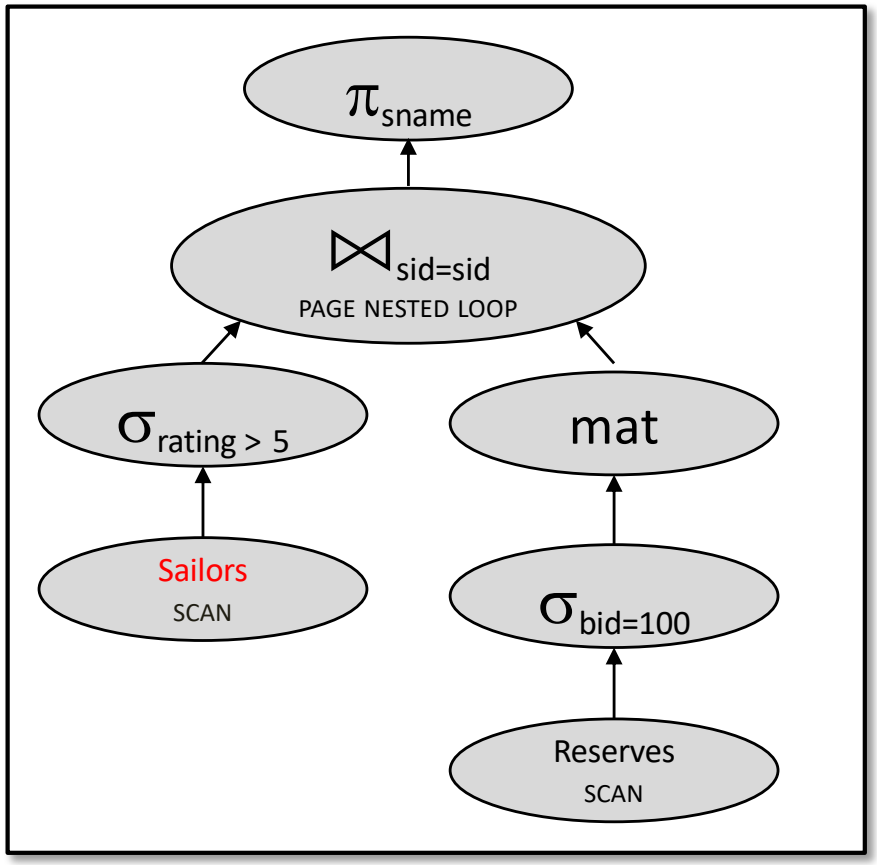
4060 IOs

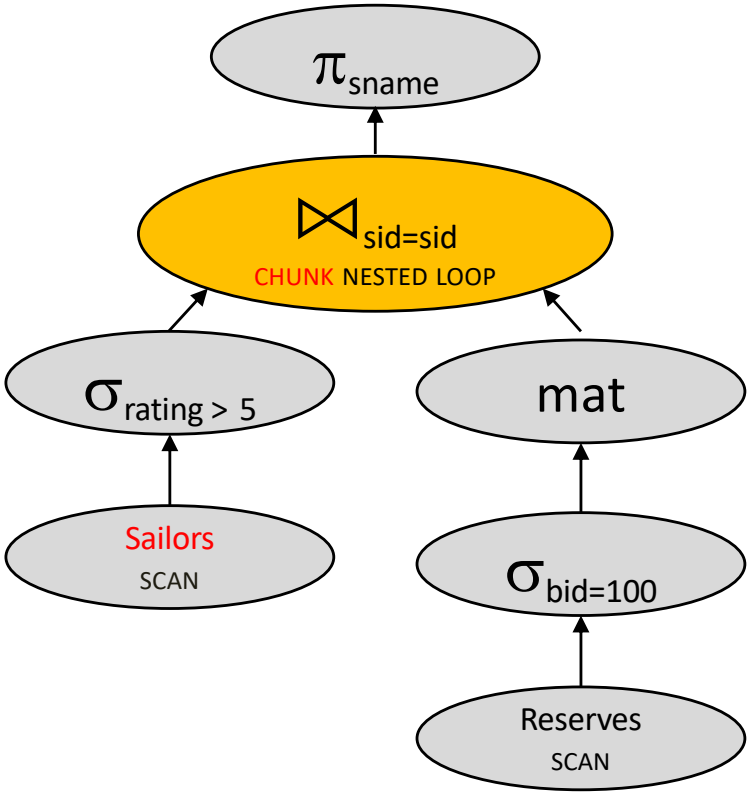
Join algorithm



4010 IOs

Join algorithm again





4010 IOs

Quick quizz

With **5 buffers**, cost of plan:

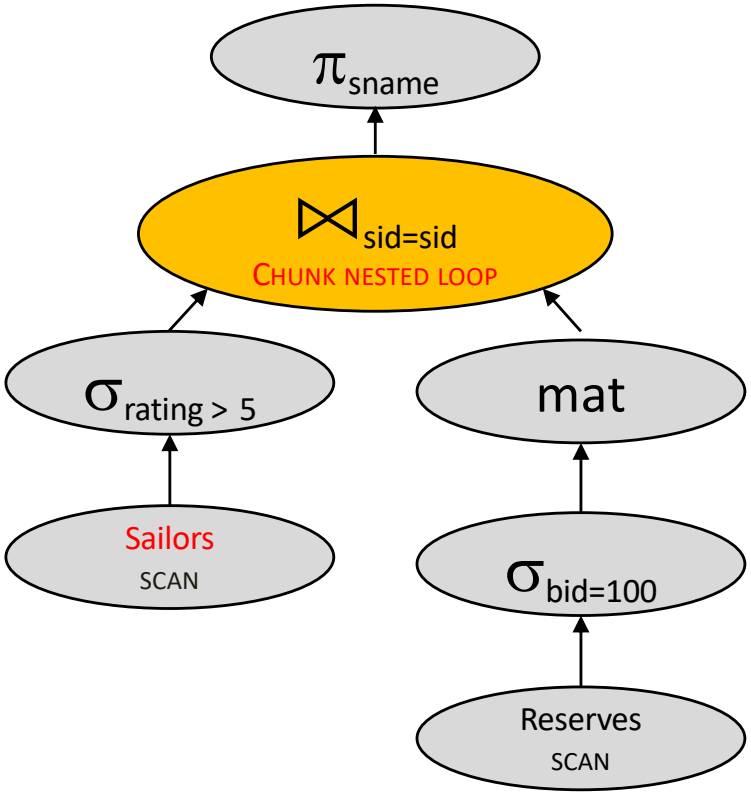
Scan Sailors (500)

Scan Reserves (1000)

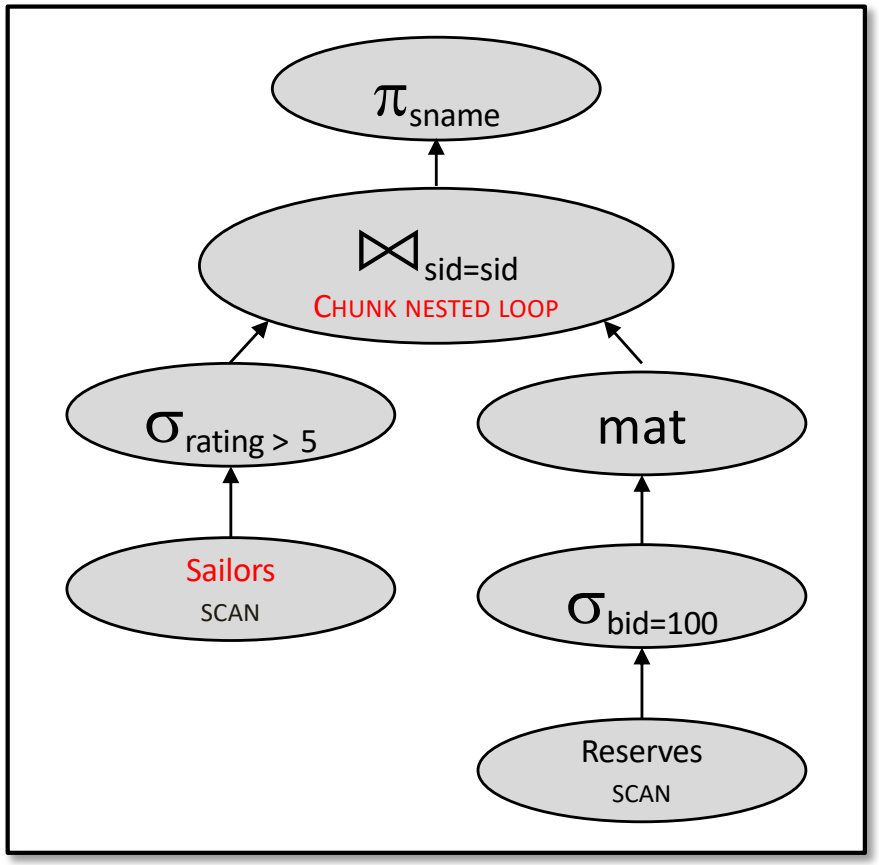
Write Temp T1 (10)

For each blockful of high-rated sailors
    Loop on T1 (??? * 10)

Total: 500 + 1000 + 10 + (??? * 10)

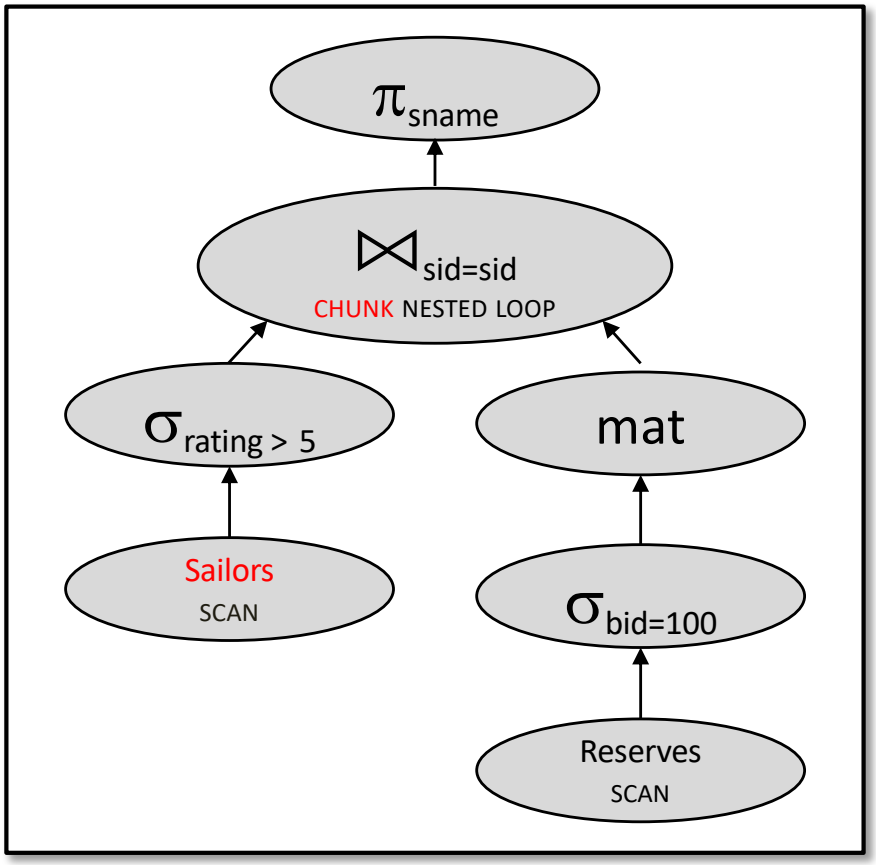


Join algorithm again



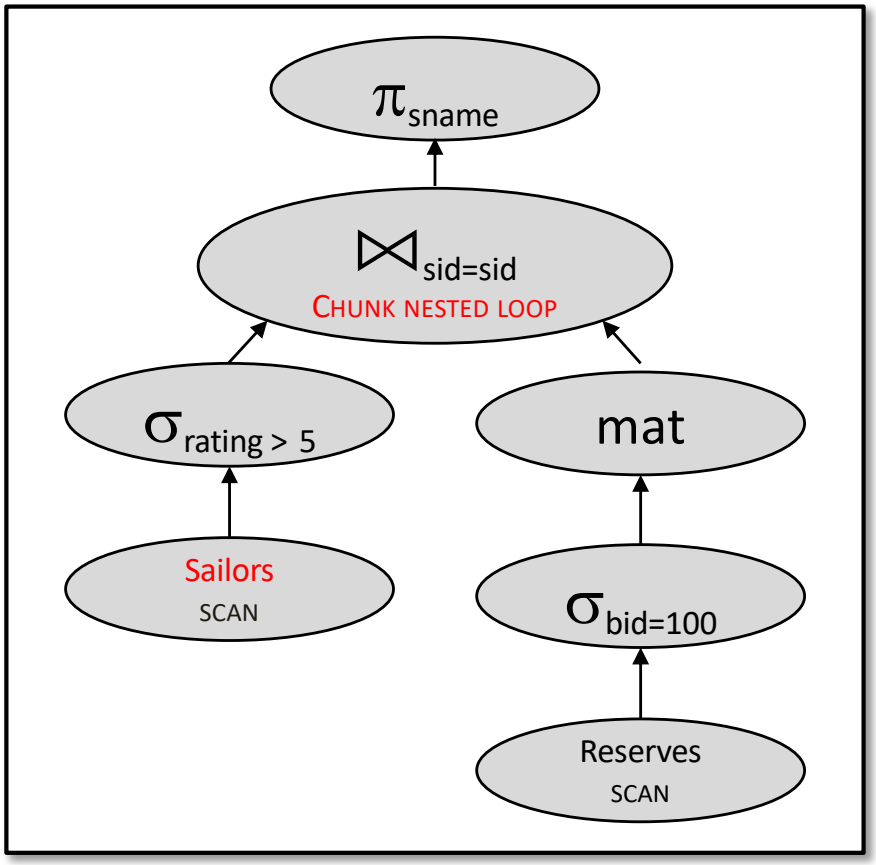
2140 IOs

Join algorithm again

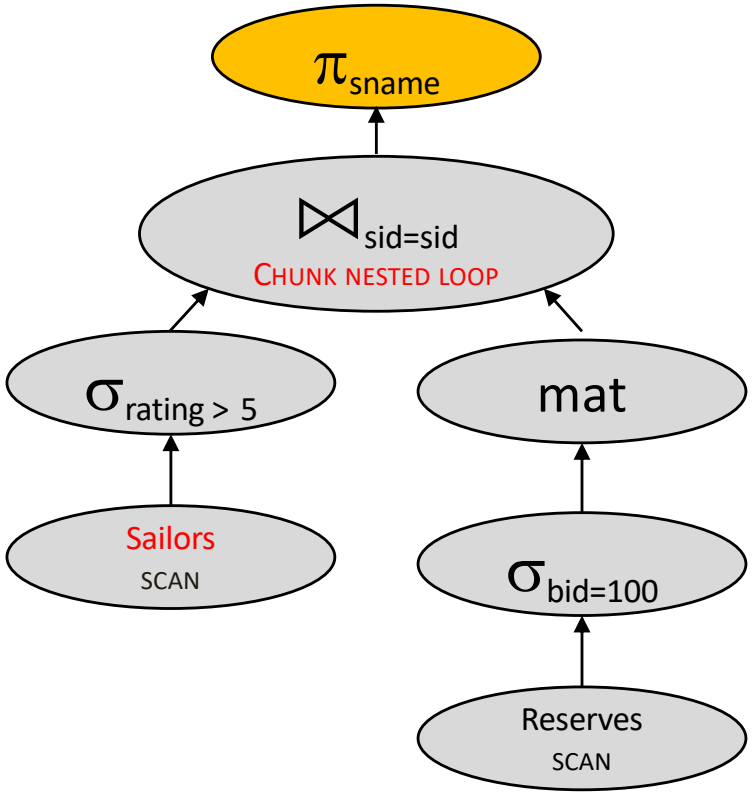


2140 IOs

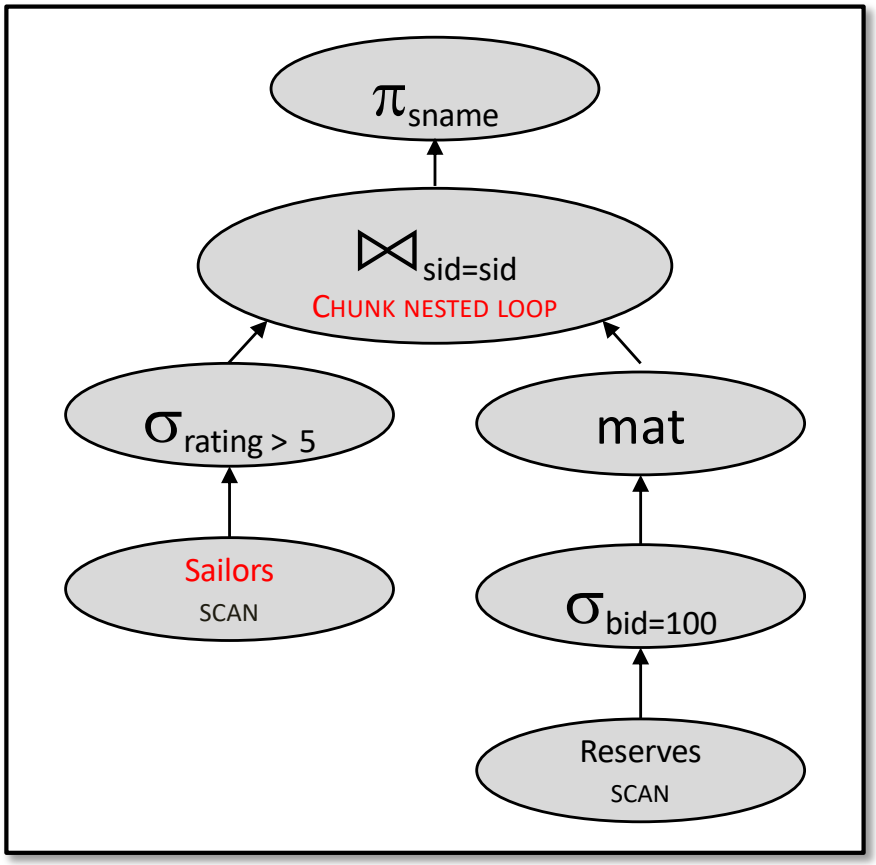
Projection cascade & pushdown



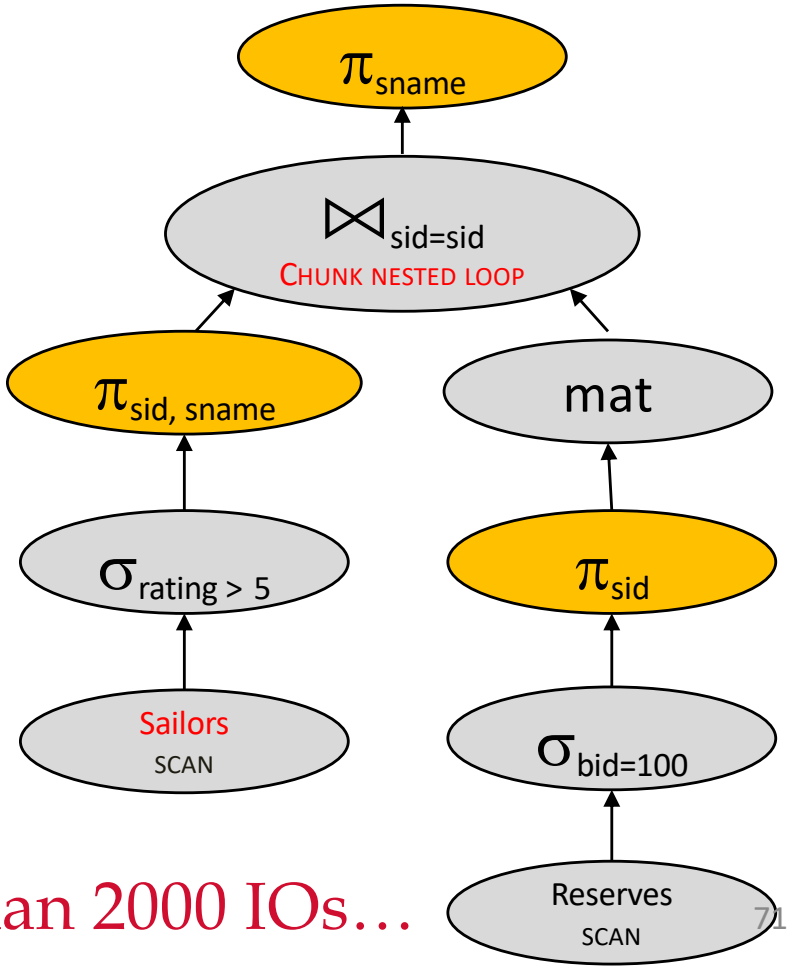
2140 IOs



Projection cascade & pushdown

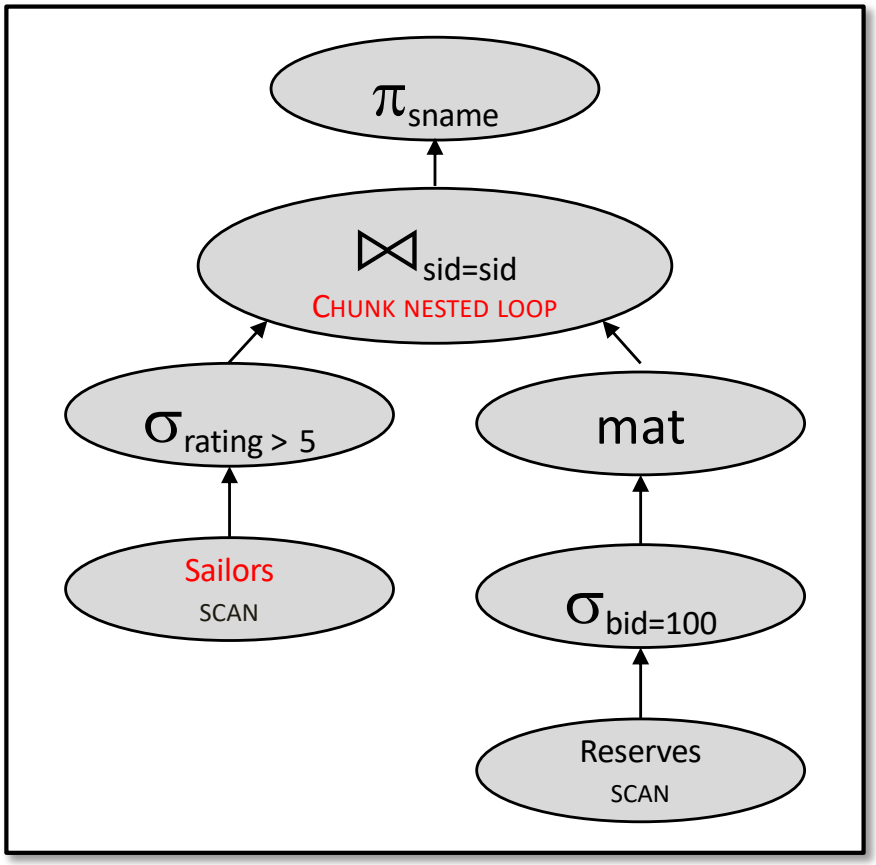


2140 IOs

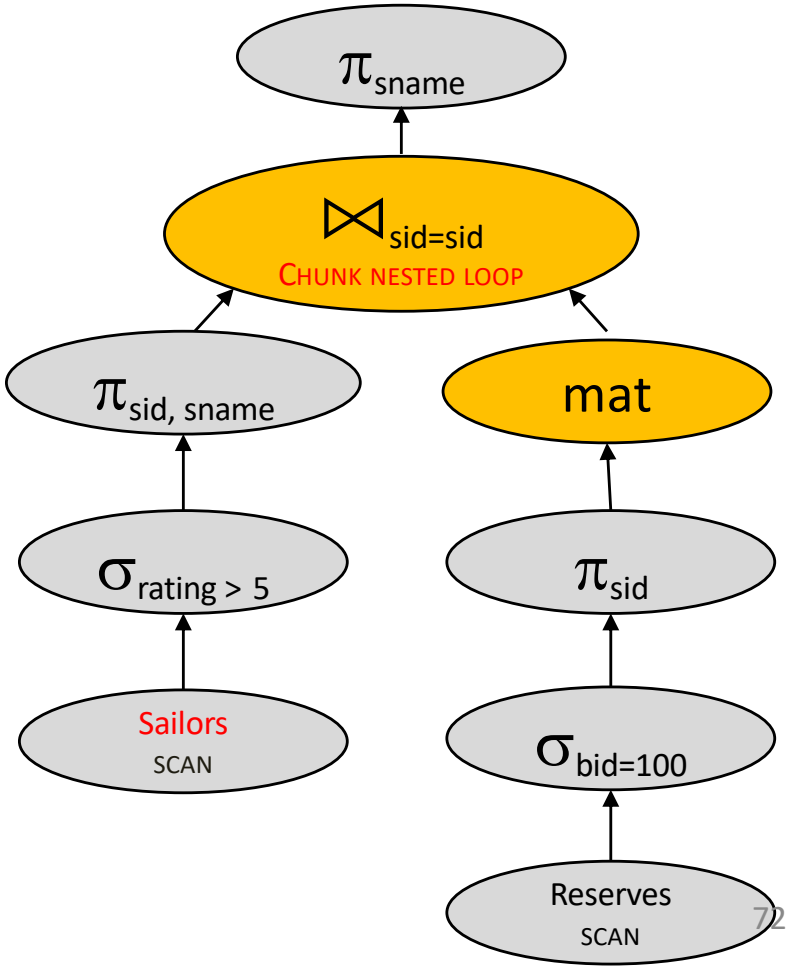


Less than 2000 IOs…

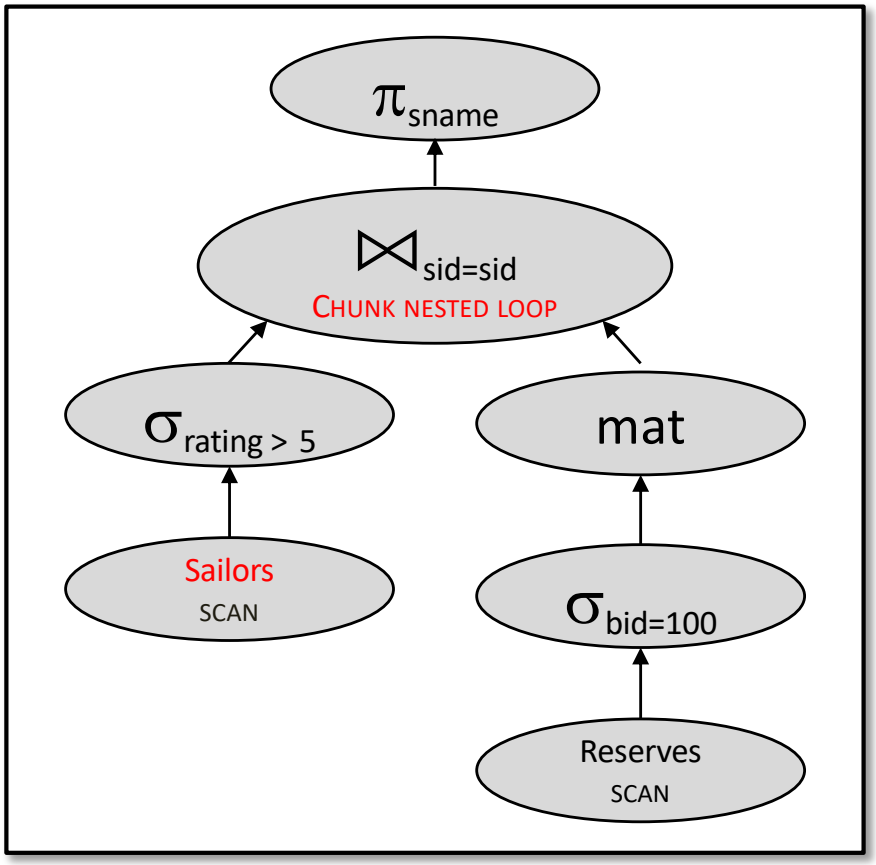
With join reordering, no materialization

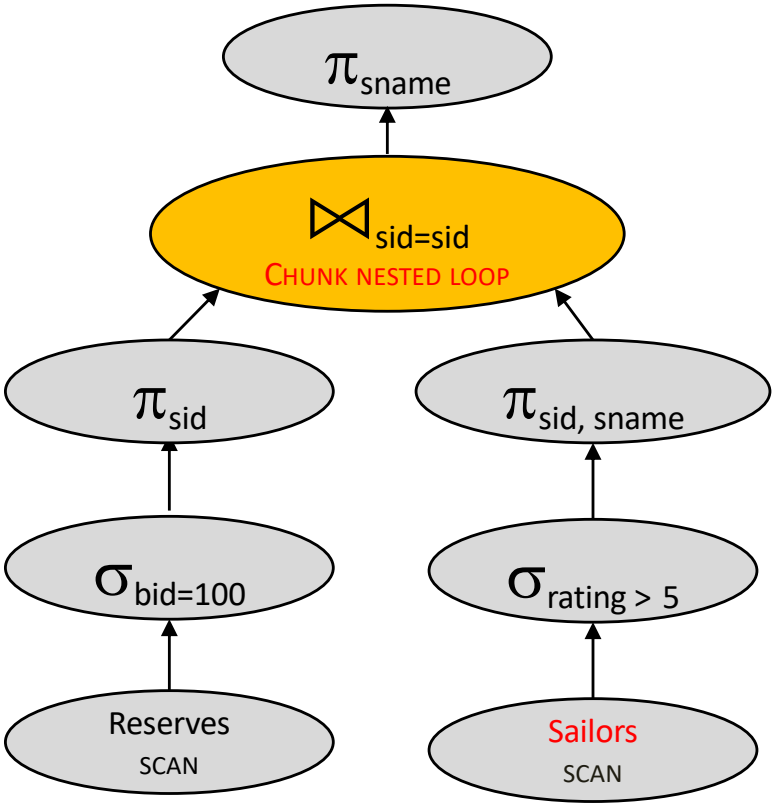


2140 IOs



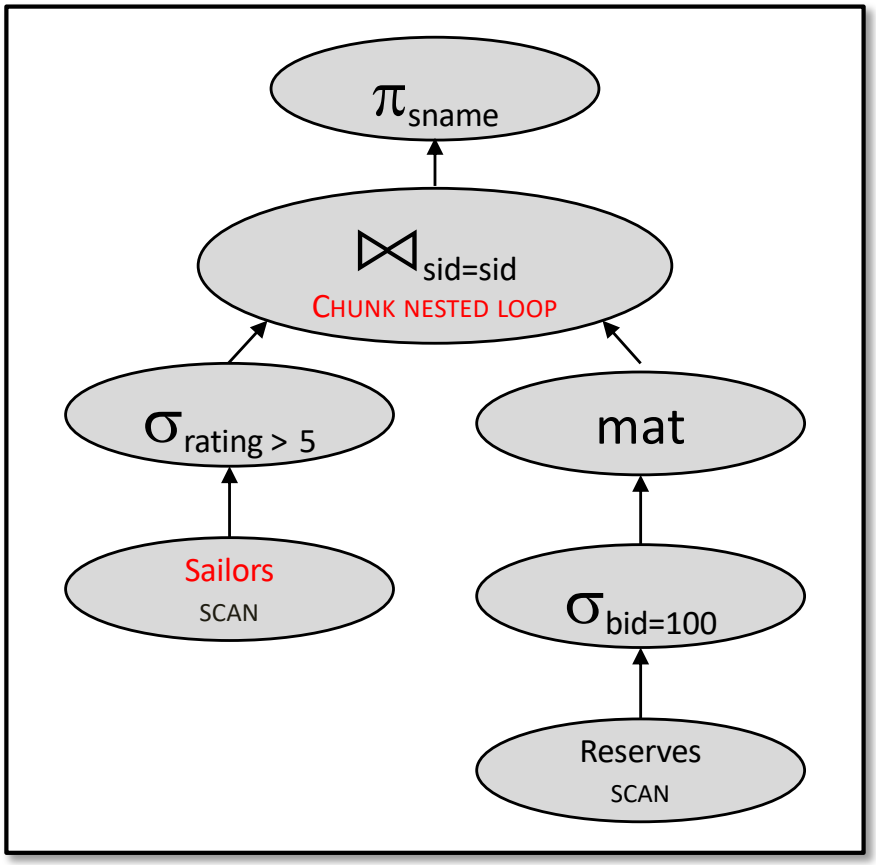
With join reordering, no materialization



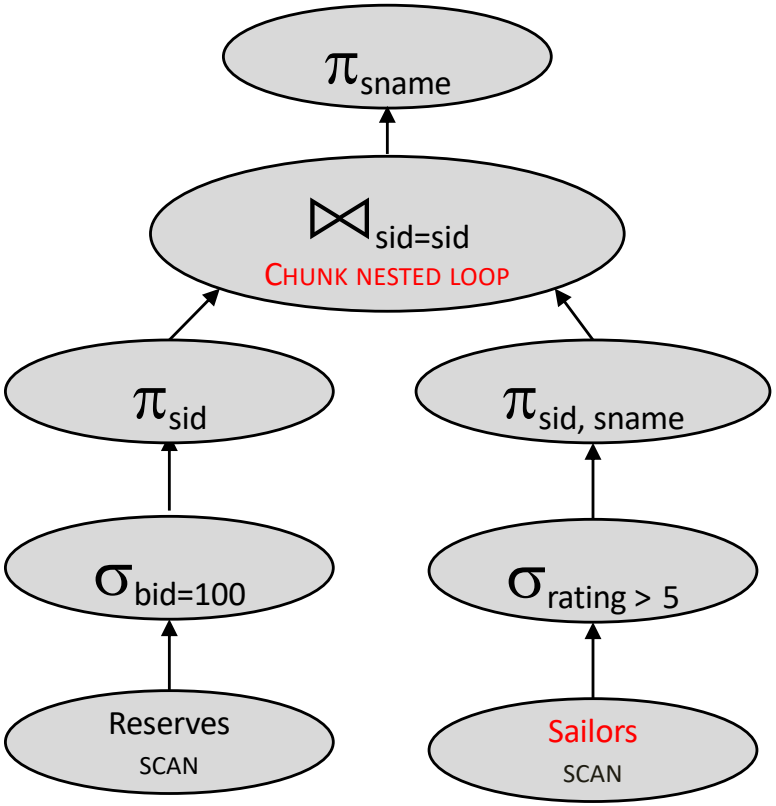


2140 IOs

With join reordering, no materialization



2140 IOs



1500 IOs

Quick quizz

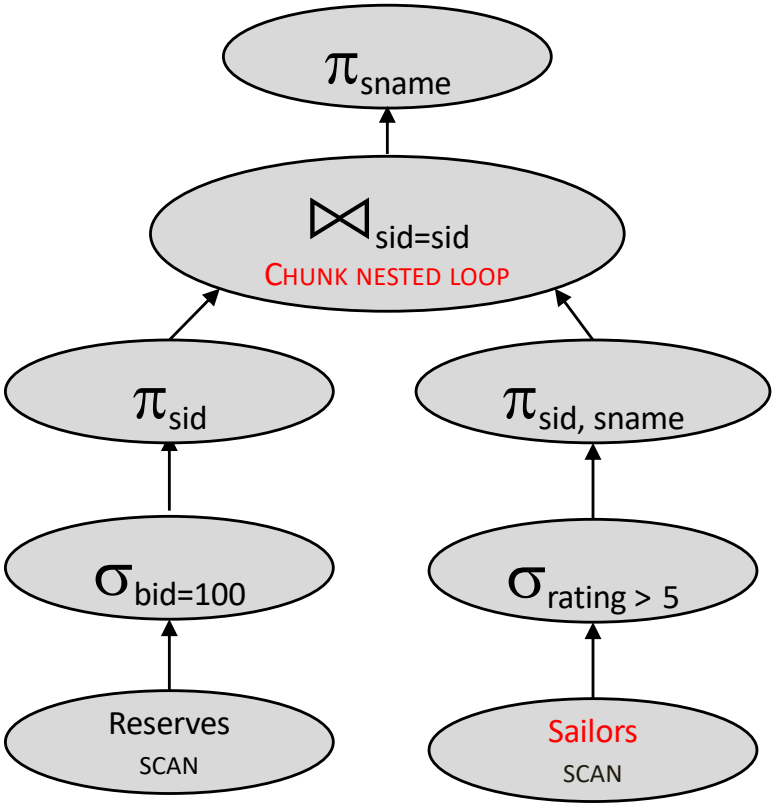
With 5 buffers, cost of plan:

Scan Reserves (1000)

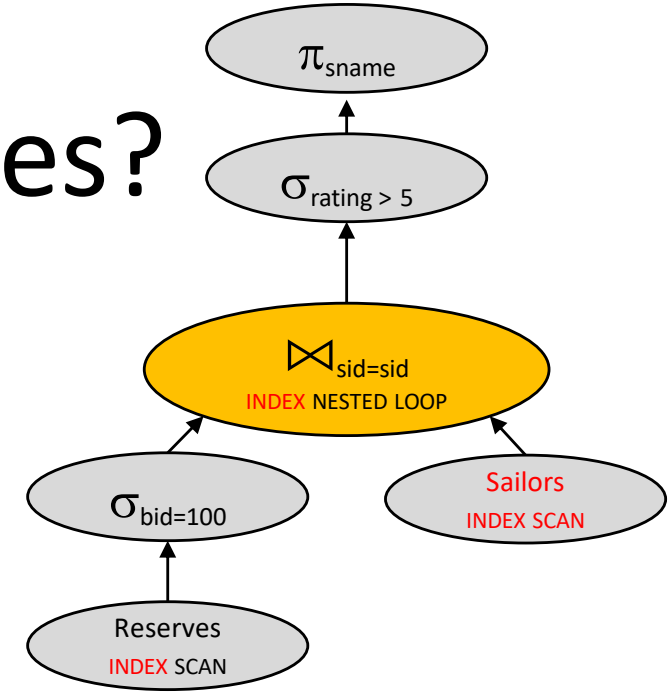
For each blockful of sids that rented boat 100
(recall Reserve tuple is 40 bytes, assume sid is 4 bytes)

Loop on Sailors (??? * 500)

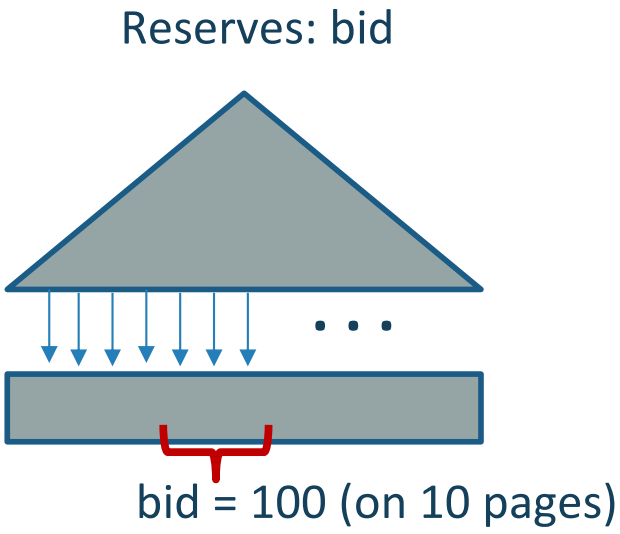
Total: 1000 + (??? * 500)

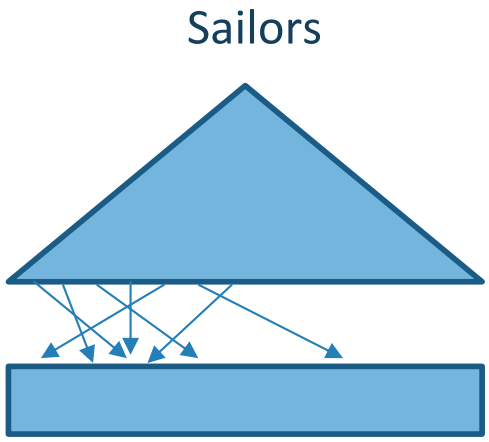


How about indexes?

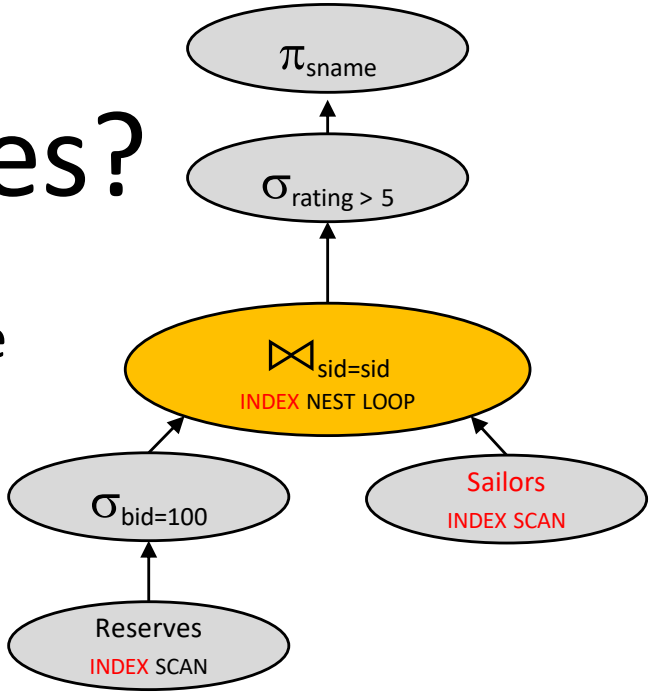


- Indexes:
  - Reserves.bid clustered
  - Sailors.sid unclustered
- Assume indexes fit in memory





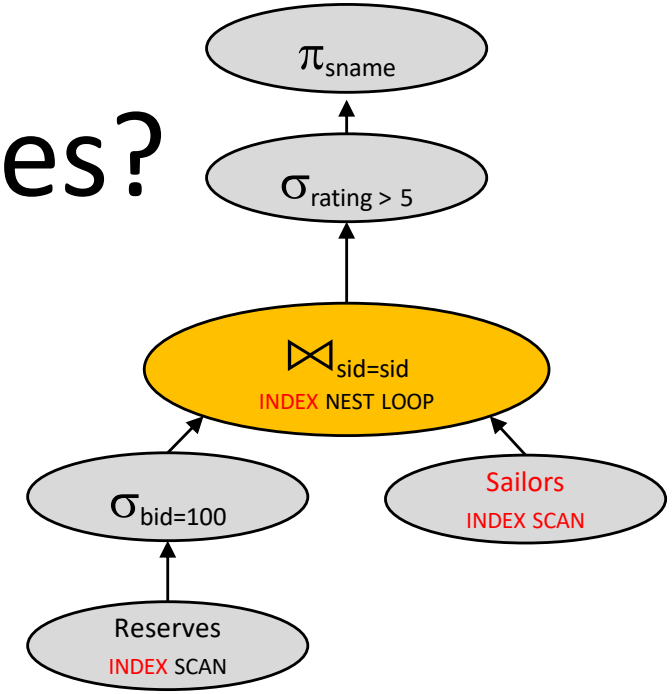
How about indexes?



- With clustered index on bid of Reserves, we access how many pages of Reserves?:
  - 100,000/100 = 1000 tuples on 1000/100 = 10 pages.
- Index Nested Loop; **no projection pushdown**
  - Projecting out unnecessary fields from outer doesn't make an I/O difference.
- Join column sid is a **key** for Sailors.
  - At most one matching tuple, unclustered index on sid OK
- No need to push $\sigma_{rating > 5}$ before the join
  - Does not affect Sailors.sid index lookup

1010 IOs

How about indexes?

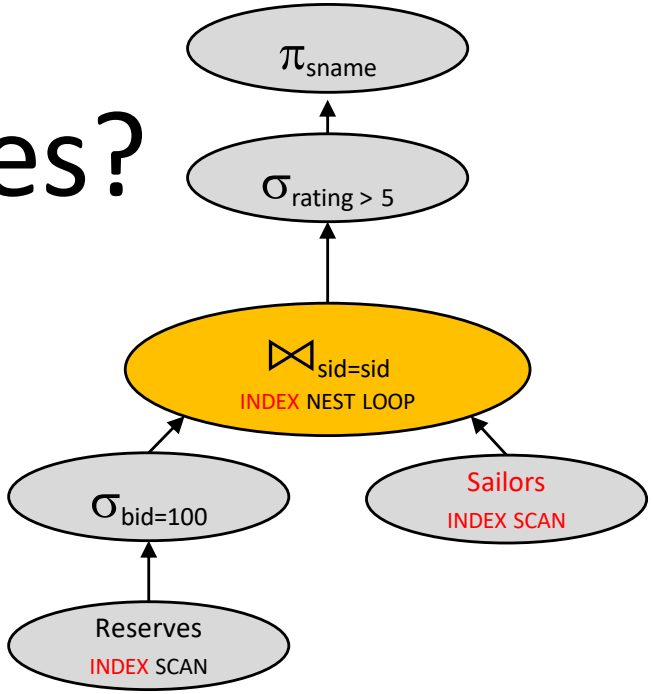


- With clustered index on bid of Reserves, we access how many pages of Reserves?:
  - 100,000/100 = 1000 tuples on 1000/100 = 10 pages.
- for each Reserves tuple (???) get matching Sailors tuple (1 IO)

(recall: 100 Reserves per page, 10 pages)

- 10 + ???*1

How about indexes?



- With clustered index on bid of Reserves, we access:
  - 100,000/100 = 1000 tuples on 1000/100 = 10 pages.
- for each Reserve tuple (1000) get matching Sailors tuple (1 IO)

  (recall:100 Reserves per page, 1000 pages)

1010 IOs

- 10 + 1000*1

Summing up

- There are *lots* of plans
  - Even for a relatively simple query

- Engineers often think they can pick good ones
- Not so clear that is true!
  - Manual query planning can be tedious, technical
  - Machines are better at enumerating options than people
  - We will see soon how optimizers make simplifying assumptions

RELATIONAL QUERY OPTIMIZATION: COST MODELS AND SEARCHING

What is needed for optimization?

- Given: A closed set of operators
  - Relational ops (table(s) in, table out)
  - Physical implementations (of those ops and a few more)

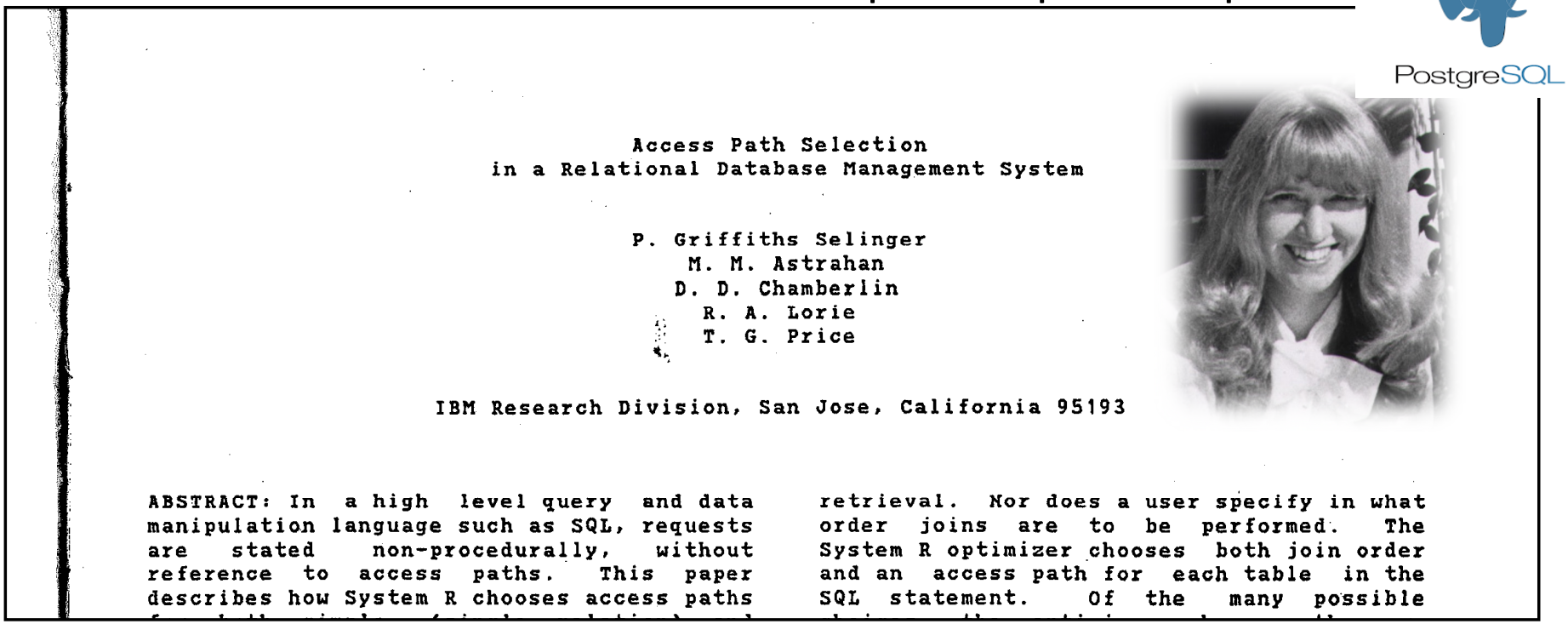
1. Plan space
   - Based on relational equivalences, different implementations

2. Cost Estimation based on
   - Cost formulas
   - Size estimation, in turn based on
     - Catalog information on base tables
     - Selectivity (Reduction Factor) estimation

3. A search algorithm
   - To sift through the plan space and find lowest cost option!

Query Optimization

- We'll focus on "System R" ("Selinger") optimizers (bottom-up)
  - Later Starburst, most open-source DBMSs
- "Cascades" optimizer is the other common one (top-down)
  - Notable differences, but similar big picture
- Randomized: random-walk over space of possible plans



Access Path Selection
in a Relational Database Management System

P. Griffiths Selinger
M. M. Astrahan
D. D. Chamberlin
R. A. Lorie
T. G. Price

IBM Research Division, San Jose, California 95193

ABSTRACT: In a high level query and data manipulation language such as SQL, requests are stated non-procedurally, without reference to access paths. This paper describes how System R chooses access paths

retrieval. Nor does a user specify in what order joins are to be performed. The System R optimizer chooses both join order and an access path for each table in the SQL statement. Of the many possible

Highlights of System R optimizer

Works well for up to 10-15 joins

1. Plan Space: Too large, must be pruned.
   - Potential win:
     - many plans could have the same "overpriced" subtree
     - ignore all those plans!
   - Common heuristic: consider only *left-deep plans*
   - Common heuristic: avoid cartesian products
2. Cost estimation
   - Very inexact, but works in practice
   - Statistics in system catalogs used to estimate sizes & costs
   - Considers combination of CPU and I/O costs
   - System R's scheme has been improved since that time
3. Search Algorithm: dynamic programming

Query optimization

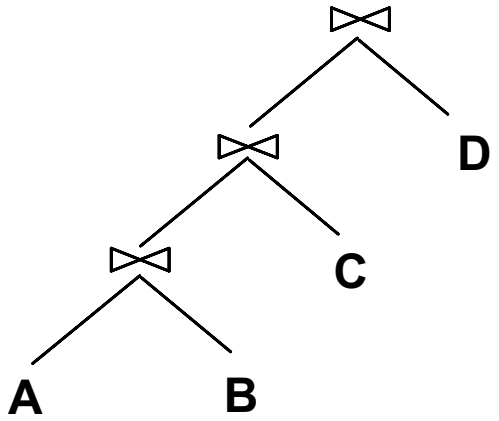
1. Plan Space
2. Cost Estimation
3. Search Algorithm

Query blocks: units of optimization

- Break query into *query blocks*
- Optimize one block at a time
- Uncorrelated nested blocks computed once
- Correlated nested blocks like function calls
  - But sometimes can be "decorrelated" (beyond our scope)

```
SELECT S.sname
  FROM Sailors S
 WHERE S.age IN
 (SELECT MAX (S2.age)
    FROM  Sailors S2
GROUP BY  S2.rating)
```

Outer block *Nested block*



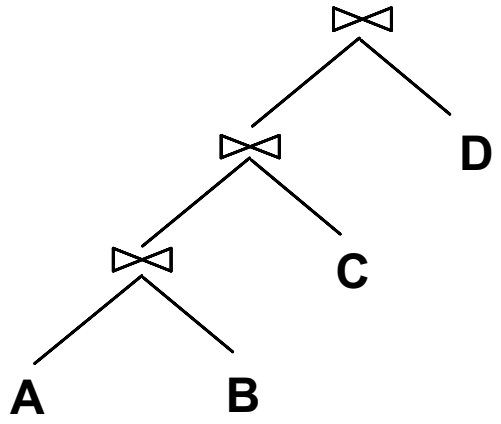
Query blocks: units of optimization

- Break query into *query blocks*
- Optimize one block at a time
- Uncorrelated nested blocks computed once
- Correlated nested blocks like function calls
  - But sometimes can be "decorrelated" (beyond our scope)

```
SELECT S.sname
  FROM Sailors S
 WHERE S.age IN
 (SELECT MAX (S2.age)
    FROM  Sailors S2
GROUP BY  S2.rating)
```

Outer block *Nested block*

- For each block, the plans considered are:
  - All available access methods, for each relation in FROM clause.
  - All *left-deep join trees*
    - right branch always a base table
    - consider all join orders and join methods



Recall algebra equivalences

- Selections:
  - $\sigma_{c1 \wedge \ldots \wedge cn}(R) \equiv \sigma_{c1}(\ldots(\sigma_{cn}(R))\ldots)$ (*cascade*)
  - $\sigma_{c1}(\sigma_{c2}(R)) \equiv \sigma_{c2}(\sigma_{c1}(R))$ (*commute*)
- Projections:
  - $\pi_{a1}(R) \equiv \pi_{a1}(\ldots(\pi_{a1, \ldots, an-1}(R))\ldots)$ (*cascade*)
- Cartesian product
  - $R \times (S \times T) \equiv (R \times S) \times T$ (associative)
  - $R \times S \equiv S \times R$ (commutative)

More RA equivalences

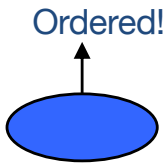
- Eager projection
  - Rule of thumb: can project anything not needed later ("downstream" if you follow the arrows)

- Selection on a cross-product is equivalent to a join
  - If selection is comparing attributes from each side
  - E.g. $\sigma_{R.a=S.b}(R \times S) \equiv R \bowtie_{R.a=S.b} S$

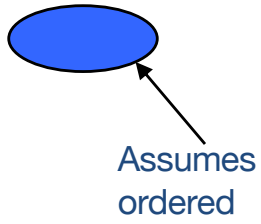
- Selection pushdown: A selection only on attributes of R "commutes" with $R \bowtie S$.
  - i.e., $\sigma(R \bowtie S) \equiv \sigma(R) \bowtie S$
  - but only if the selection doesn't refer to S!

“Physical” properties

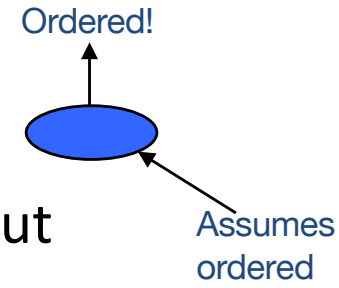
- Two common “physical” properties of an output:
  1. Sort order
  2. Hash grouping
- Certain operators produce these properties in output
  - E.g. index scan (result is sorted)
  - E.g. sort (result is sorted)
  - E.g. hash (result is grouped)



- Certain operators require these properties at input
  - E.g. MergeJoin requires sorted input

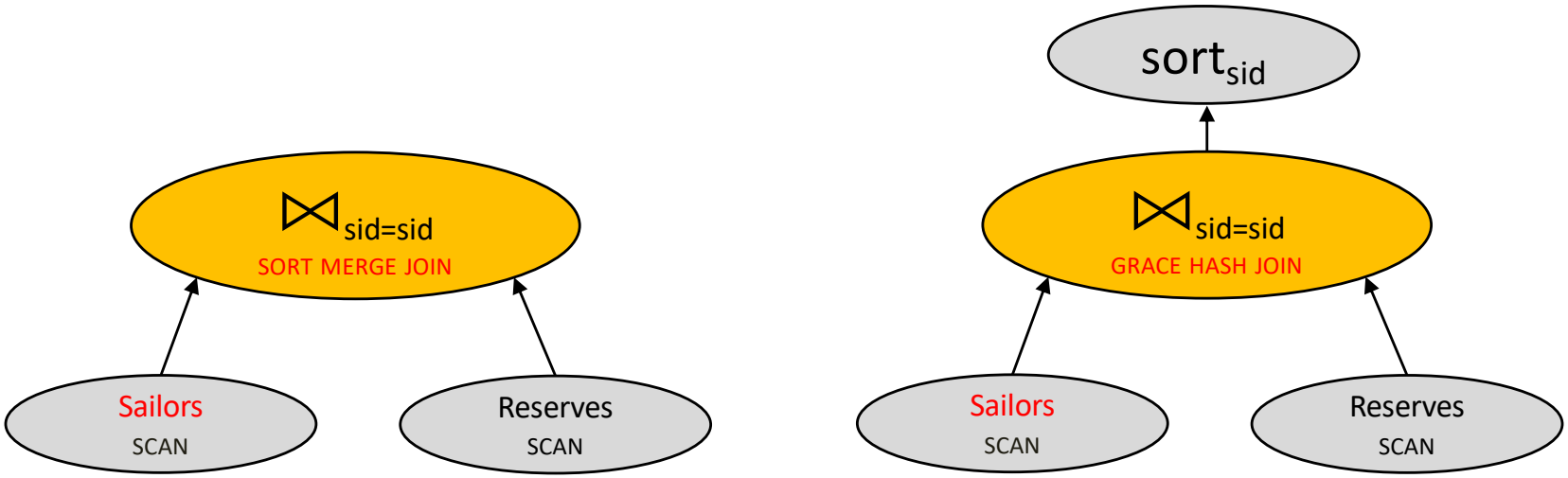


- Certain operators preserve these properties from inputs
  - E.g. MergeJoin preserves sort order of inputs
  - E.g. NestLoop Join preserves sort order of outer (left) input



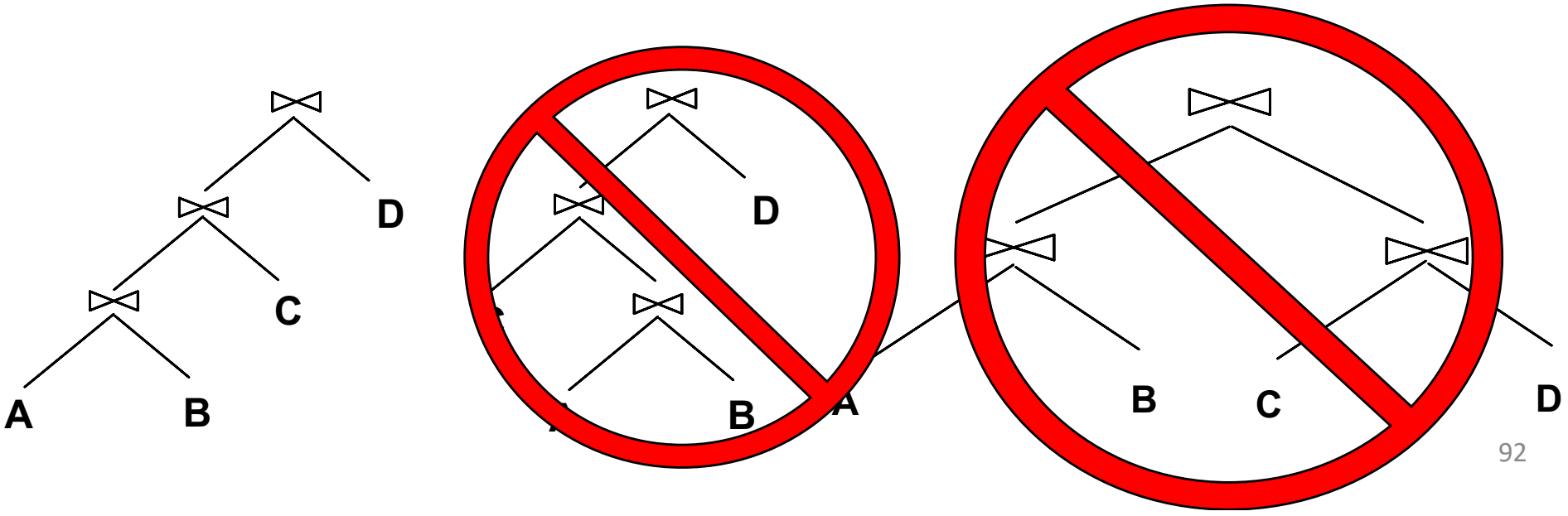
Physically equivalent plans

- Same content *and* same physical properties



Queries over multiple relations

- A System R heuristic: *only left-deep join trees* considered.
  - Restricts the search space
  - Left-deep trees allow us to generate all *fully pipelined* plans
    - Intermediate results not written to temporary files
  - Not all left-deep trees are fully pipelined (e.g., SM join)



Plan space review

- For a SQL query, full plan space:
  - All equivalent relational algebra expressions
    - Based on the equivalence rules we learned
  - All mixes of physical implementations of those algebra expressions
- We might prune this space:
  - Selection/Projection pushdown
  - Left-deep trees only
  - Avoid cartesian products
- Along the way we may care about physical properties like sorting
  - Because downstream ops may depend on them
  - And enforcing them later may be expensive

Plan execution models

- How are plans executed on relations ?
  - How does an operator handle input from the operators below?
  - How does an operator send output to the operator above?

Three execution models:

- Iterator model (pipelined model) – tuple-at-a-time, most common – implements Next()
- Materialization model – sometimes good for OLTP
- Vectorized model – batch-of-tuples-at-a-time, ideal for OLAP
  - Also implements Next()
  - Amortize over fewer function calls
  - SIMD -> batch size

Plan execution direction

- Bottom-up: operator *pushes* data to parent
  - One tuple at a time
  - A batch of tuples at a time
  - All tuples at once
- Top-down: operator *pulls* data from children

- OLAP: vectorized + bottom-up
- OLTP: materialize / iterator + bottom-up

Pipelined execution

For tuple-at-a-time or batch-of-tuples-at-a-time execution.

A plan can be pipelined if

- For each unary operator O: output of the child operator can be streamed as input to O one tuple (or batch of tuples) at a time
  - Example: selection, projection
- For each binary operator O: output of the *outer branch* child operator can be streamed as input to O one tuple (or batch of tuples) at a time
  - Example: INLJ

Pipeline breakers: operators that block until children emit all their tuples

- Sort-merge join , Grace-hash join
- Rule: eliminate redundant attributes before pipeline breakers, to reduce materialization cost

Query optimization

1. Plan Space
2. Cost Estimation
3. Search Algorithm

Cost estimation

- For each plan considered, must estimate total cost:
  - Must estimate *cost* of each operation in plan tree.
    - Depends on input cardinalities.
    - We've already discussed this for various operators
      - sequential scan, index scan, joins, etc.
  - Must estimate *size of result* for each operation in tree!
    - Because it determines downstream input cardinalities!
    - Use information about the input relations.
    - For selections and joins, assume independence of predicates.
  - In System R, cost is boiled down to a single number consisting of #I/O + *CPU-factor* * #tuples
- Q: Is "cost" the same as estimated "run time"?

Statistics and catalogs

- Need information on the relations and indexes involved
- *Catalogs* typically contain at least:

Statistic	Meaning
NTuples	# of tuples in a table (cardinality)
NPages	# of disk pages in a table
Low/High	min/max value in a column
Nkeys	# of distinct values in a column
IHeight	the height of an index
INPages	# of disk pages in an index

- Catalogs updated periodically.
  - Too expensive to do continuously
  - Lots of approximation anyway, so a little sloppiness here is ok…
- Modern systems do more
  - Especially keep more detailed statistical information on data values
    - e.g., histograms

Size estimation and selectivity

```
SELECT  attribute list
  FROM  relation list
 WHERE  term1 AND ... AND termk
```

- Max output cardinality = product of input cardinalities

- *Selectivity (sel)* associated with each *term*
  - reflects the impact of the *term* in reducing result size.
  - selectivity = |output| / |input|
  - *Cow book calls selectivity "Reduction Factor" (RF)*

- Avoid confusion:
  - "highly selective" in common English is opposite of a high selectivity value (|output|/|input| high!)

Result size estimation

- *Result cardinality* = Max # tuples * product of all RF's.

- Term col=value (given Nkeys(I) on *col*)
  RF = *1/NKeys(I)*

- Term col1=col2 (handy for joins too…)
  RF = *1/MAX(NKeys(I1), NKeys(I2))*
  Why MAX? See bunnies in 2 slides…
- Term col>value
  RF = *(High(I)-value)/(High(I)-Low(I) + 1)*

- *Note, if missing the needed stats, assume 1/10!!!*

P(leftEar = rightEar)

- 100 bunnies
- Tables:
  - LeftEar(BunnyID, color)
  - RightEar(BunnyID,color)
- 2 distinct LeftEar colors $\{C_1, C_2\}$
- 10 distinct RightEar colors $\{C_1..C_{10}\}$
- Independent ears
- What's the probability of matching ears (size of the natural join!)?

$$
\begin{aligned}
&P(L = R) \\
&= \sum_i P(C_i, C_i) \\
&= P(C_1, C_1) + P(C_2, C_2) + P(C_3, C_3) + \dots \\
&= (\frac{1}{2} \cdot \frac{1}{10}) + (\frac{1}{2} \cdot \frac{1}{10}) + (0 \cdot \frac{1}{10}) + \dots \\
&= 1/10 \\
&= 1/\mathrm{MAX}(2, 10)
\end{aligned}
$$

```
SELECT  *
FROM  LeftEar L, RightEar R
WHERE  R.color=L.color;
```

Postgres 10.0: src/include/utils/selfuncs.h

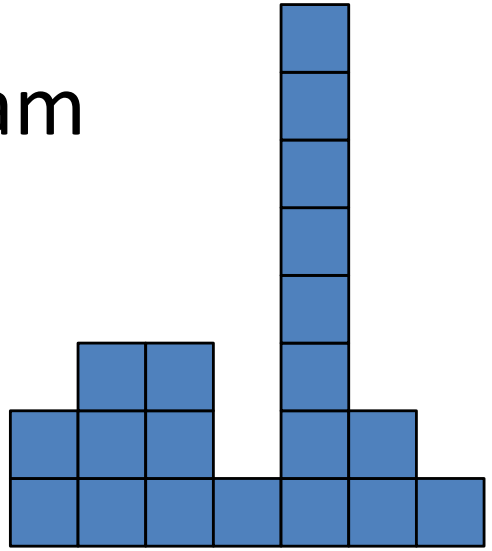
```
/* default selectivity estimate for equalities such as "A = b" */
   #define DEFAULT_EQ_SEL  0.005

   /* default selectivity estimate for inequalities such as "A < b" */
   #define DEFAULT_INEQ_SEL  0.3333333333333333

   /* default selectivity estimate for range inequalities "A > b AND A < c"
   */
   #define DEFAULT_RANGE_INEQ_SEL  0.005


  /* default selectivity estimate for pattern-match operators such as LIKE */
     #define DEFAULT_MATCH_SEL 0.005

     /* default number of distinct values in a table */
     #define DEFAULT_NUM_DISTINCT  200

     /* default selectivity estimate for boolean and null test nodes */
     #define DEFAULT_UNK_SEL     0.005
     #define DEFAULT_NOT_UNK_SEL   (1.0 - DEFAULT_UNK_SEL)
```

Reduction factors & histograms

- For better estimation, use a histogram

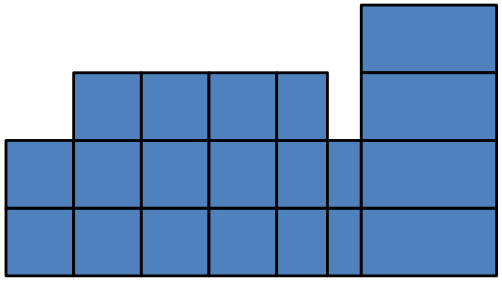
equiwidth

No. of Values	2	3	3	1	8	2	1
Value	0-.99	1-1.99	2-2.99	3-3.99	4-4.99	5-5.99	6-6.99



equidepth

No. of Values	2	3	3	3	3	2	4
Value	0-.99	1-1.99	2-2.99	3-4.05	4.06-4.67	4.68-4.99	5-6.99



Note: 10-bucket equidepth histogram divides the data into *deciles*

- akin to quantiles, median, etc.

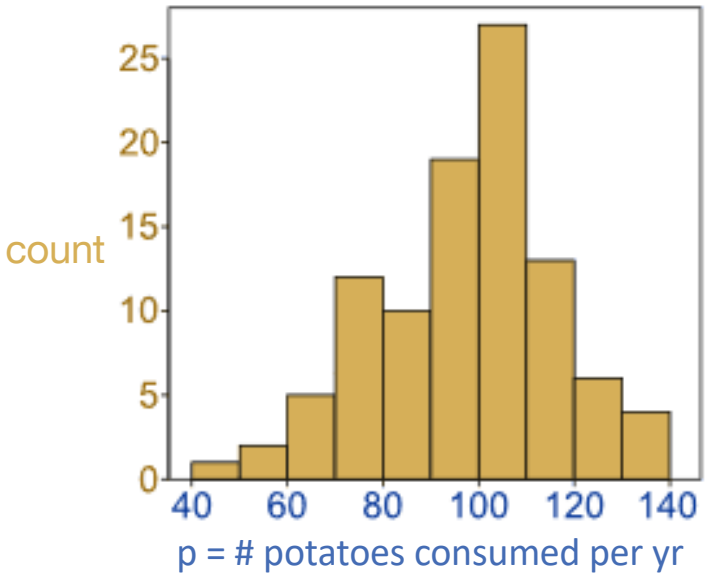
Common trick: "end-biased" histogram

- very frequent values in their own buckets

See also V-Optimal histograms on Wikipedia

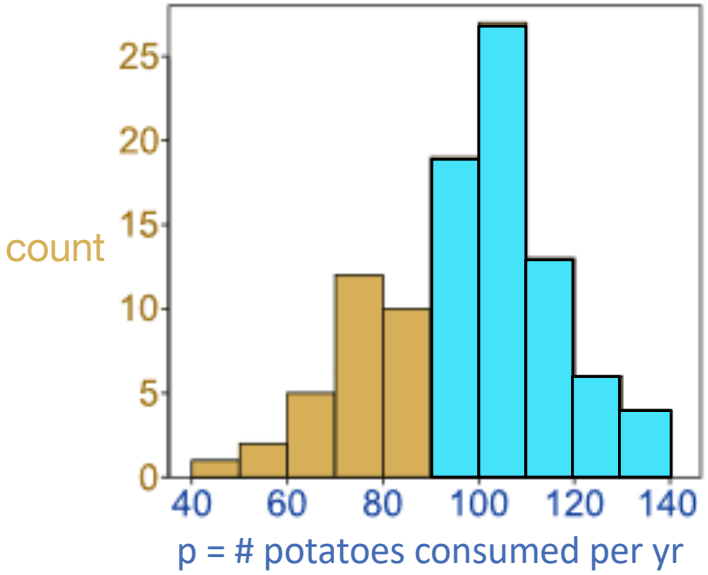
Computing selectivity with histograms

- 100 rows
- $\sigma_{p > 89}$?



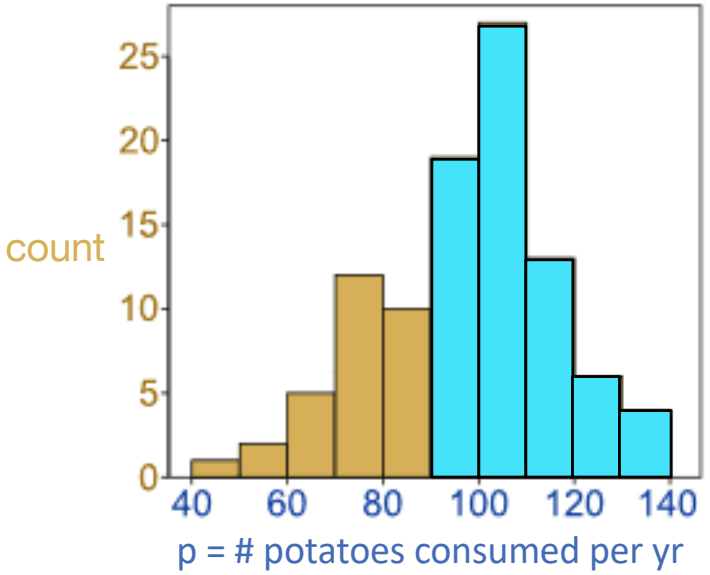
Computing selectivity with histograms

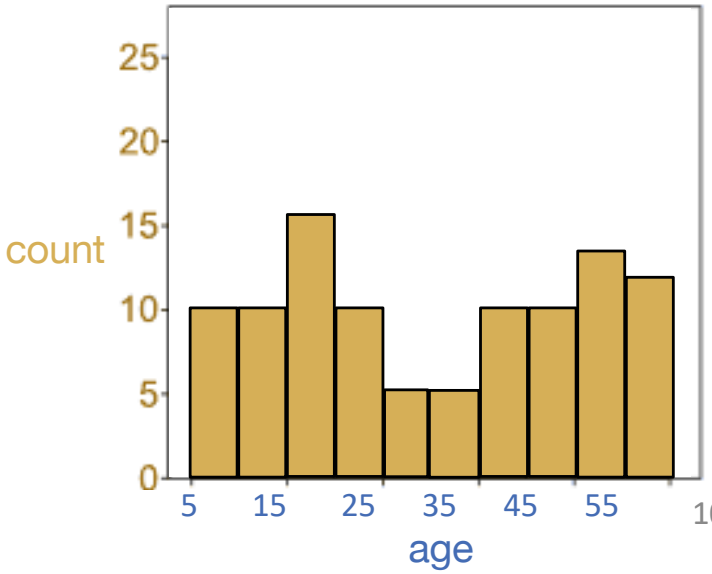
- 100 rows
- $\sigma_{p > 89}$?



Computing selectivity with histograms

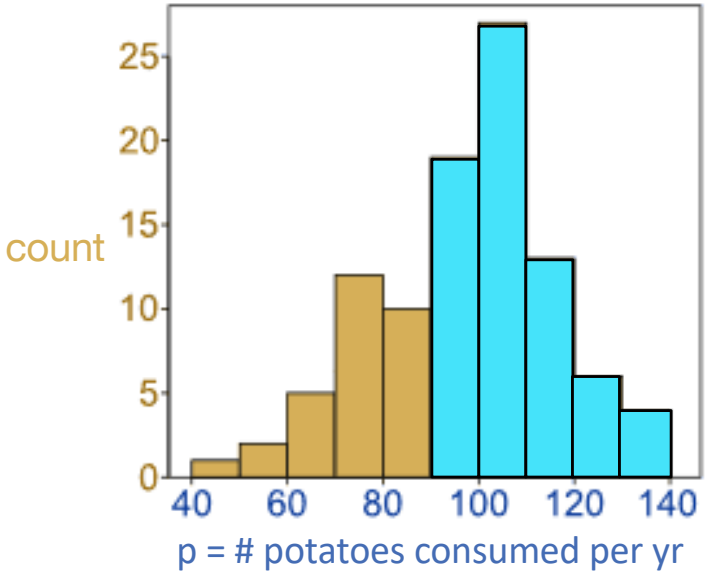
- 100 rows
- $\sigma_{p > 89}$? 50/100 = 50%.

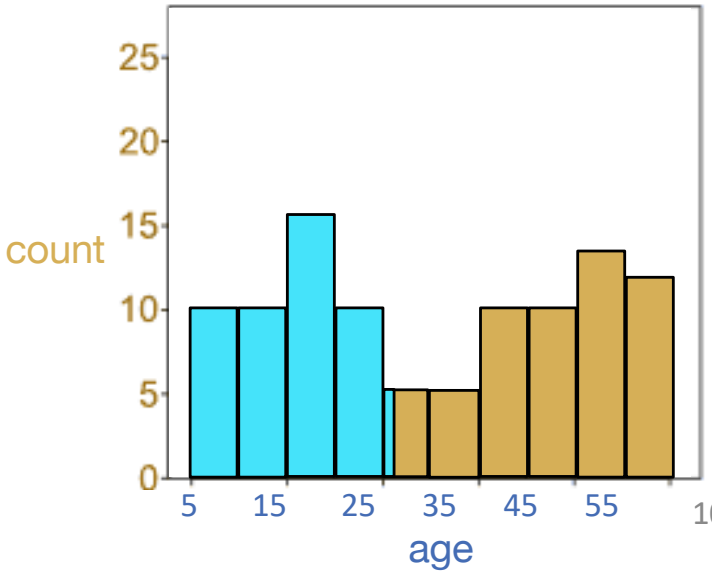




Computing selectivity with histograms

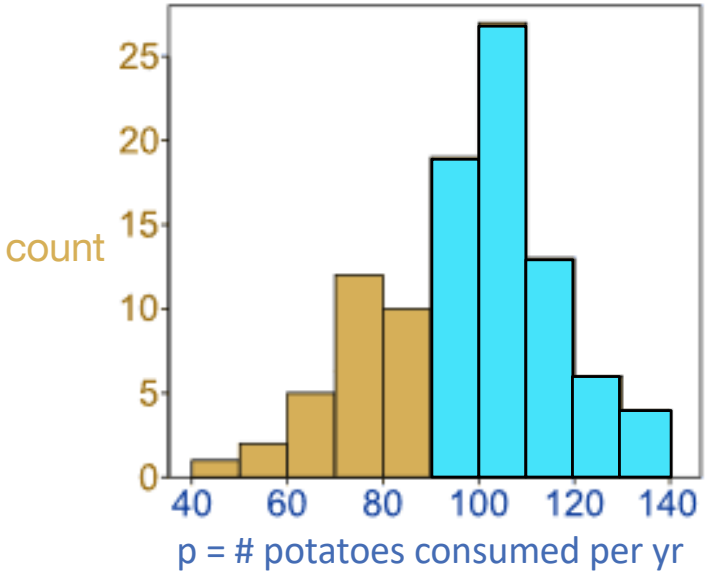
- 100 rows
- $\sigma_{age < 26}$?

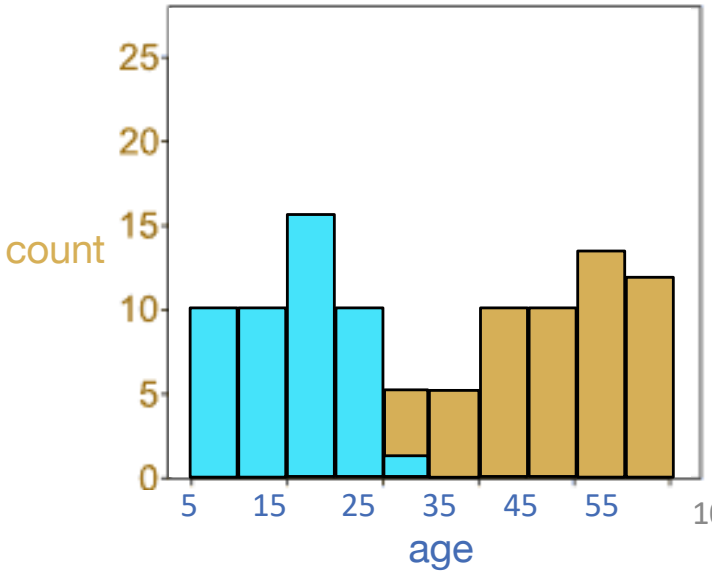




Computing selectivity with histograms

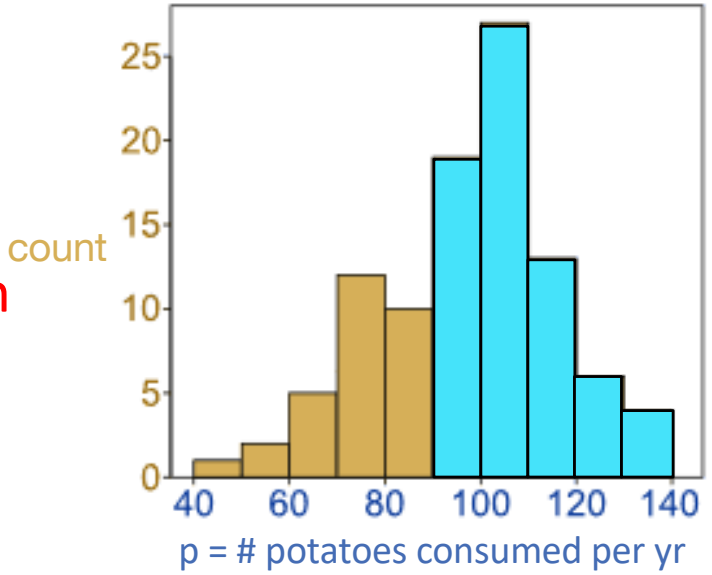
- 100 rows
- $\sigma_{age < 26}$?

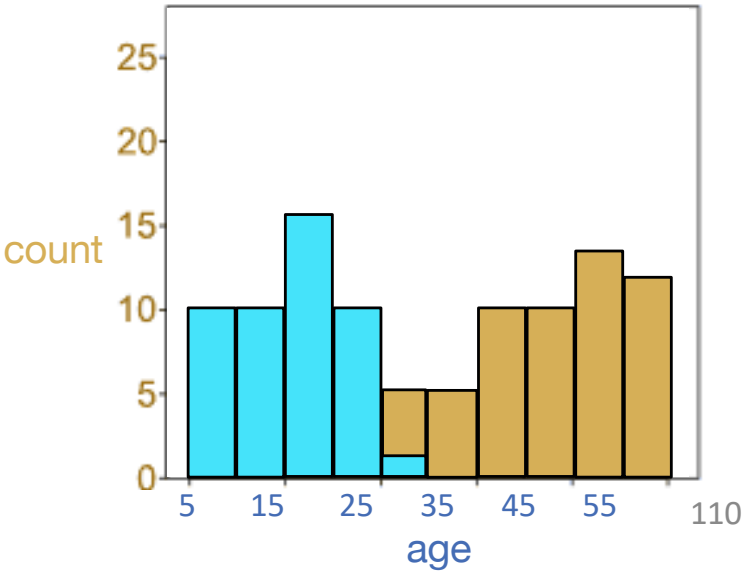




Computing selectivity with histograms

- 100 rows
- $\sigma_{age < 26}$?
  - Assumption: uniform distribution within each bin!
  - Hence ⅕ of the population of bin [25,30) has age < 26.
  - 10 + 10 + 15 + 10 + (⅕ * 5) = 46/100 = 46%

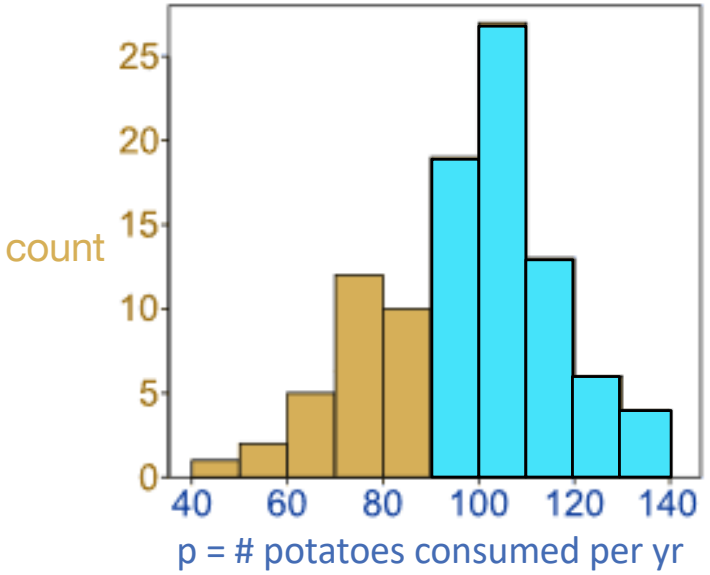


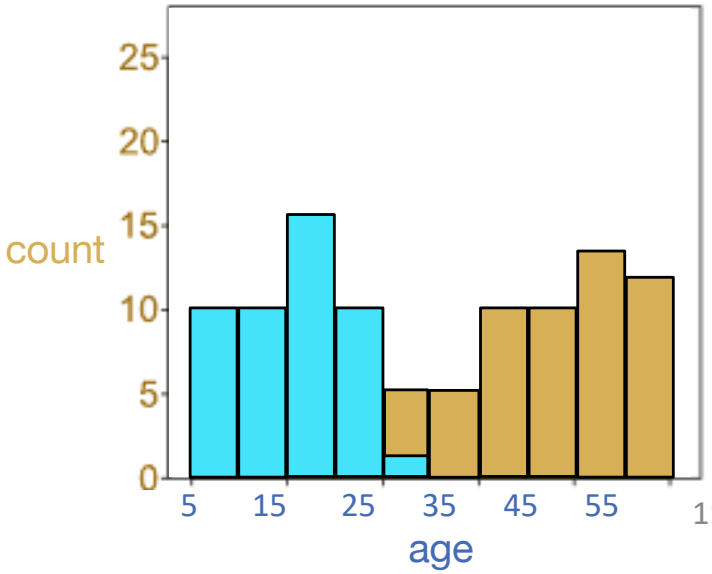


Selectivity of conjunction

- 100 rows
- $\sigma_{p > 89} \wedge \sigma_{age < 26}$?

50% 46%



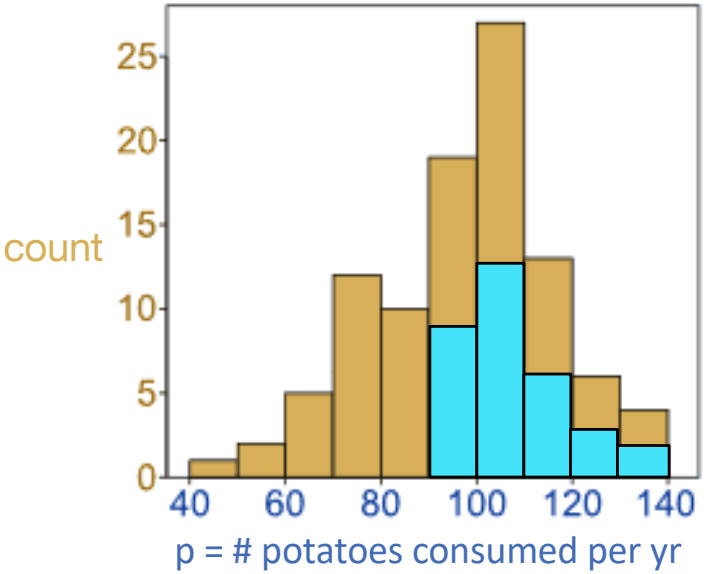


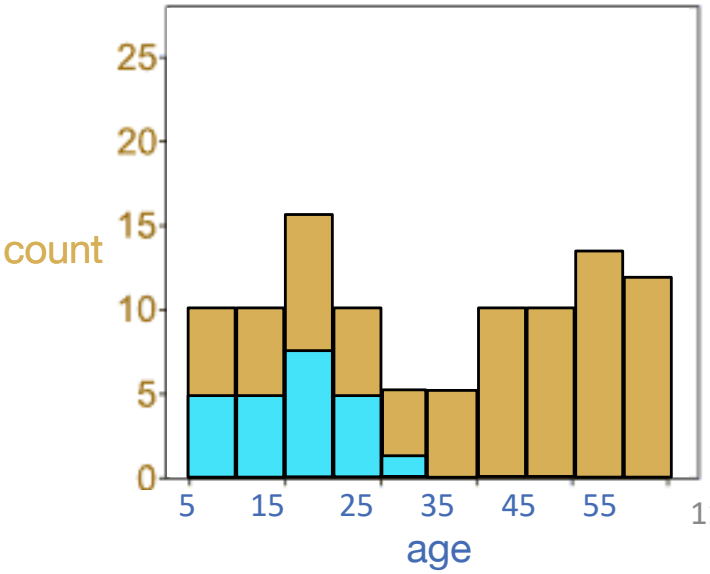
Selectivity of conjunction

- 100 rows
- $\sigma_{p > 89}^{50\%} \wedge \sigma_{age < 26}^{46\%}$?

- *Independence assumption:*
  - *Age and potato consumption are independent*
  - *Hence* p *bins all shrink by 46%.*
  - *Hence* age *bins all shrink by 50%.*

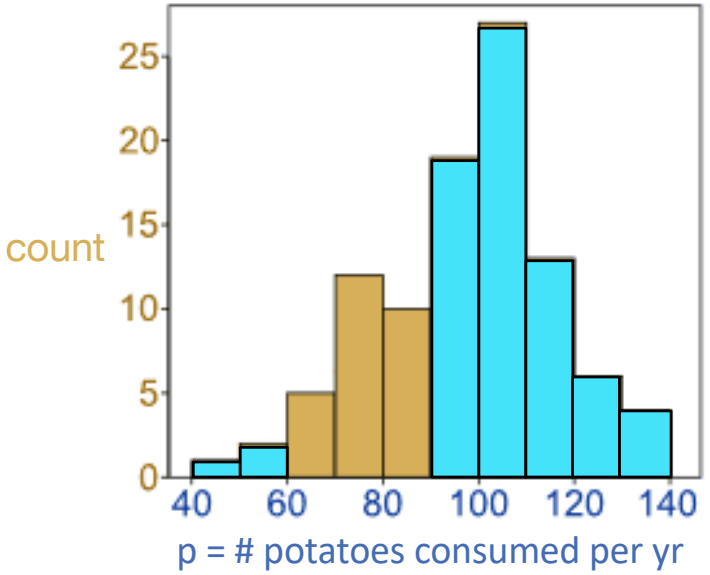
- *Selectivity: 50%* × 46% = 23%

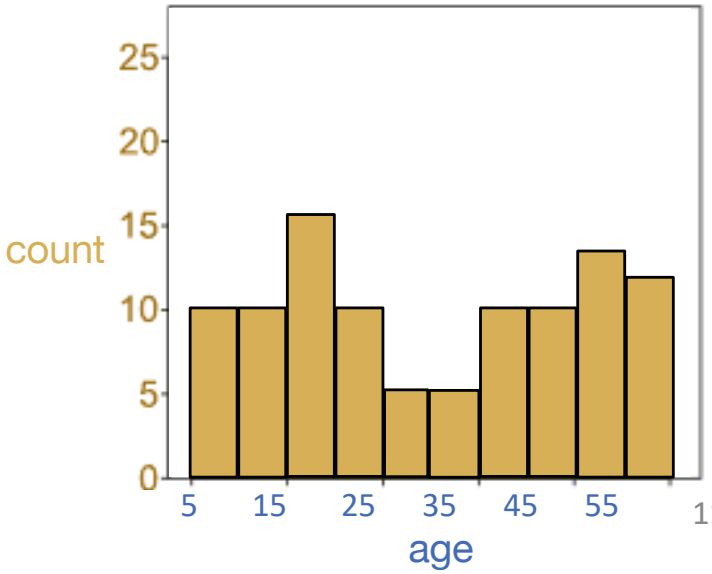




Selectivity of disjunction

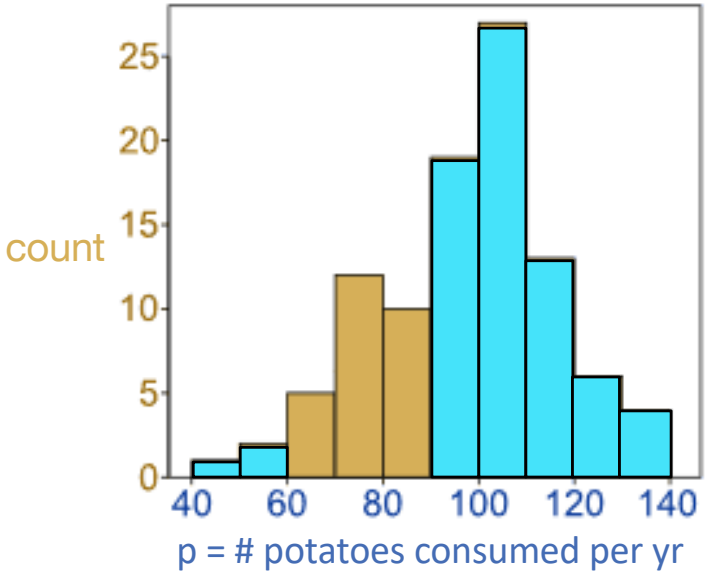
- 100 rows
- $\sigma_{p > 89}^{50\%} \vee \sigma_{p < 60}^{3\%}$?

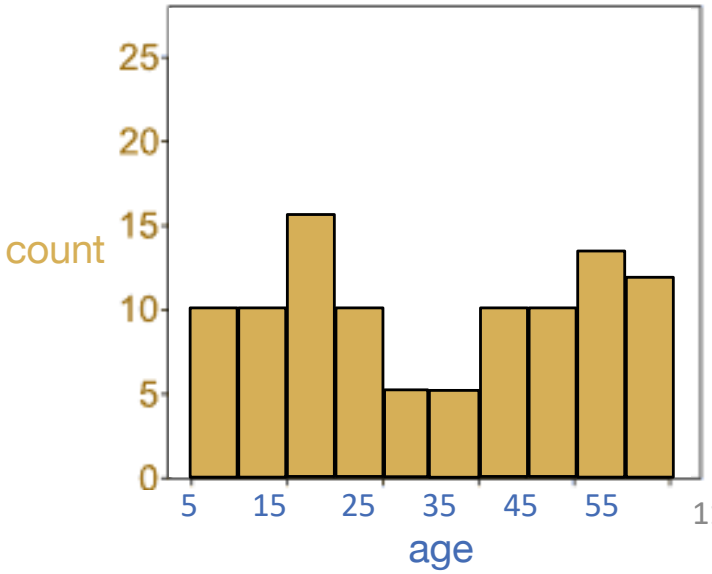




Selectivity of disjunction

- 100 rows
- $\sigma_{p > 89}^{50\%} \vee \sigma_{p < 60}^{3\%}$?
- *Selectivity: 50%* + 3% = 53%





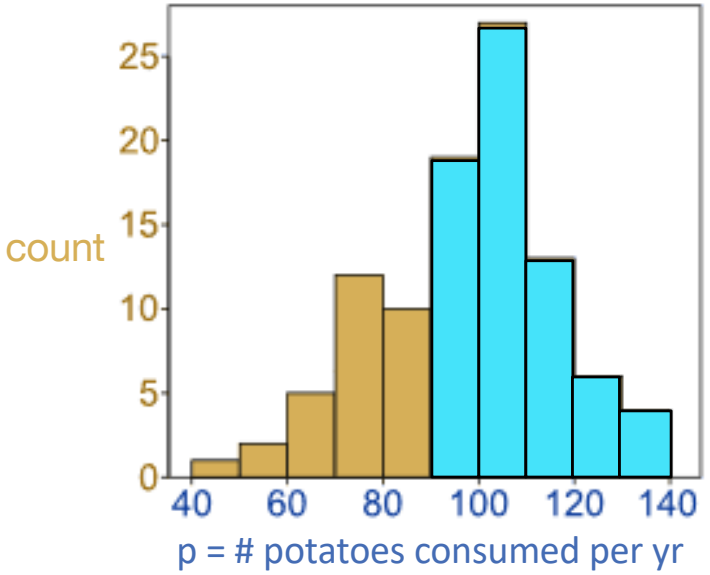
Selectivity of disjunction

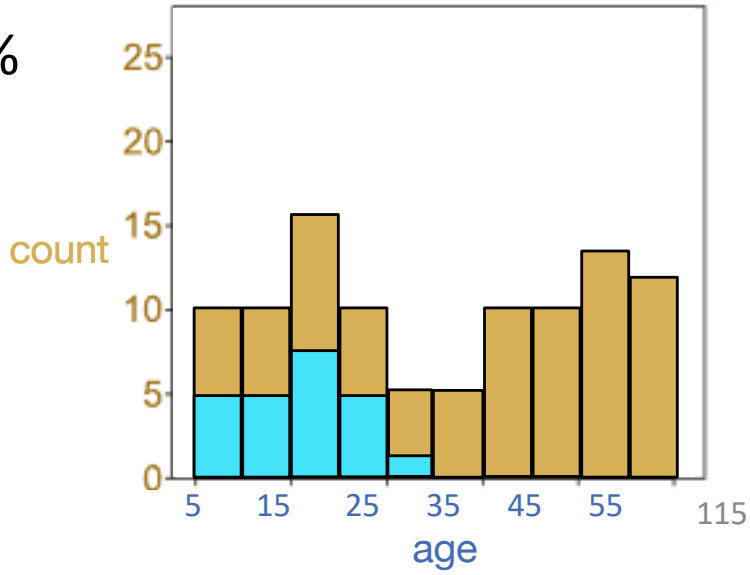
- 100 rows
- $\sigma_{p > 89}^{50\%} \vee \sigma_{age < 26}^{46\%}$?

- Answer tuples satisfy one or both predicates
- *By independence assumption:*
  - Satisfy the first predicate: 50%
  - Satisfy the second predicate: 46%
  - Satisfy both: 50% × 46%
    - Don't double-count!

- *Selectivity:*

  50% + 46% - (50% × 46%) = 73%





Selectivity for join queries?

- $R \bowtie_p \sigma_q(S)$
  - Selectivity of join predicate p is s_p
  - Selectivity of selection predicate q is s_q
  - How to think about overall selectivity?

Selectivity for join queries?

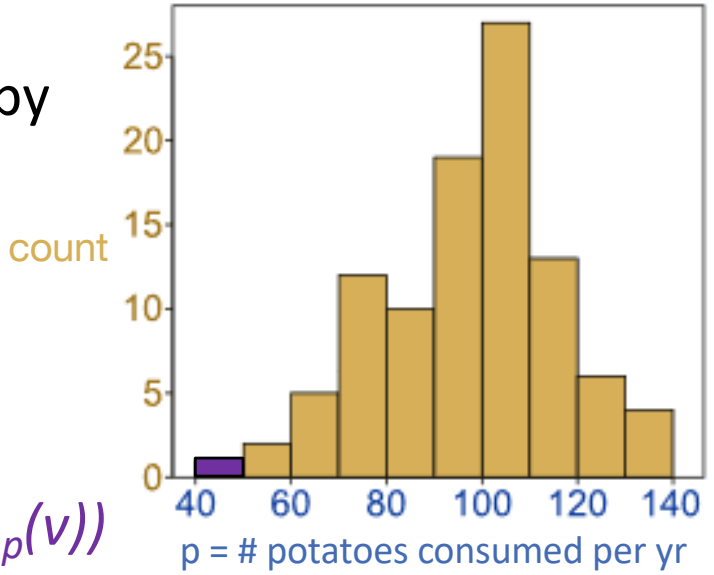
- $R \bowtie_p \sigma_q(S)$
  - Selectivity of join predicate p is s_p
  - Selectivity of selection predicate q is s_q
  - How to think about overall selectivity?

- Note: $R \bowtie_p \sigma_q(S) \equiv \sigma_p(R \times \sigma_q(S)) \equiv \sigma_{p \wedge q}(R \times S))$
  - So selectivity just $s_p s_q$
    - Applied to $|R| \times |S|$!

- Total number of rows: $s_p s_q |R||S|$

Column equality?

- T.p = T.age ??
  Intuition: similar to bunny ears but weighted by the histogram bins.



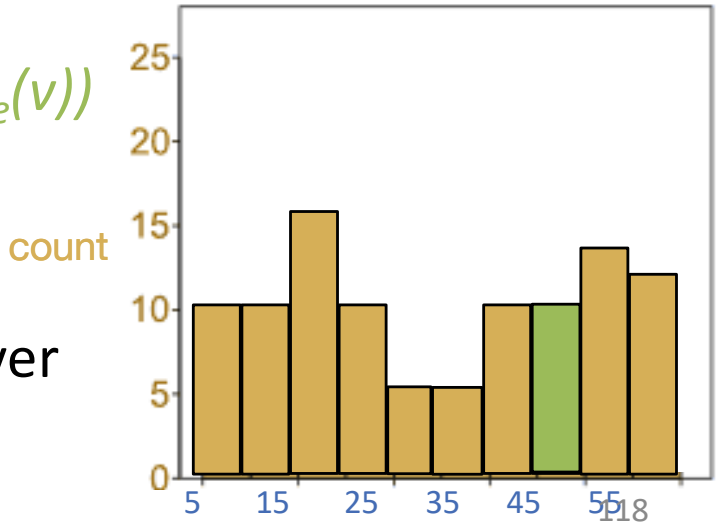
- For each value v covered in either histogram:

  // uniformity assumption within bins:

  *// P(T.p = v) = height($bin_p(v)$)/n * 1/width($bin_p(v)$)*

  // P(T.age = v) =

  *height($bin_{age}(v)$)/n * 1/width($bin_{age}(v)$)*



Idea: make this more efficient by iterating over bin boundaries rather than values!

What to remember

- How to compute selectivities for basic predicates
  - The original Selinger version
  - The histogram version
- Assumption 1: uniform distribution within histogram bins
  - Within a bin, **fraction of range = fraction of count**

- Assumption 2: independent predicates
  - Selectivity of AND = *product* of selectivities of conjuncts
  - Selectivity of OR = *sum* - *product* of selectivities of disjuncts
  - Selectivity of NOT = 1 – selectivity of operand
- Fact: joins are not a special case
  - Simply compute the selectivity of all predicates
  - And multiply by the product of the table sizes

Query optimization

1. Plan Space
2. Cost Estimation
3. Search Algorithm

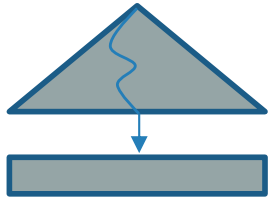
Enumeration of alternative plans

- There are two main cases:
  - Single-table plans (base case)
  - Multiple-table plans (induction)

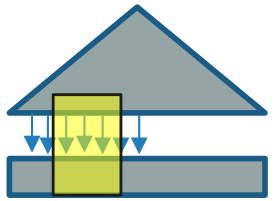
- Single-table queries include selects, projects, and groupby / aggregates:
  - Consider each available access path (file scan / index)
    - Choose the one with the least estimated cost
  - Selection/Projection done on the fly
  - Result pipelined into grouping/aggregation

Cost estimates for single-relation plans

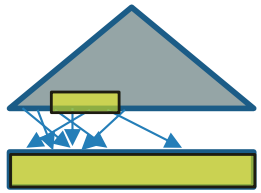
According to [Selinger79]



- Index I on primary key matches selection:
  - *Cost is Height(I)+1 for a B+ tree.*



- Clustered index I matching one or more selects:
  - *(NPages(I)+NPages(R)) * product of RF's of matching selects.*



- Non-clustered index I matching one or more selects:
  - *(NPages(I)+NTuples(R)) * product of RF's of matching selects.*
- Sequential scan of file:
  - *NPages(R).*

☞ *Recall: Must also charge for duplicate elimination if required*

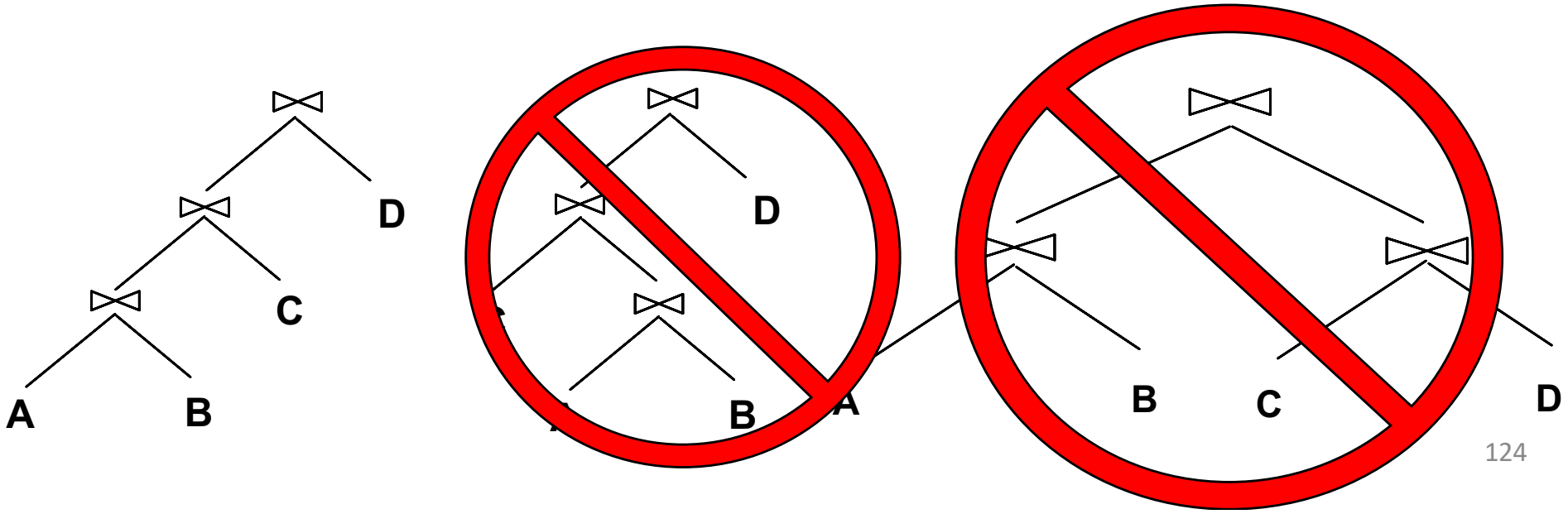
Example

```
SELECT S.sid
  FROM Sailors S
 WHERE S.rating=8
```

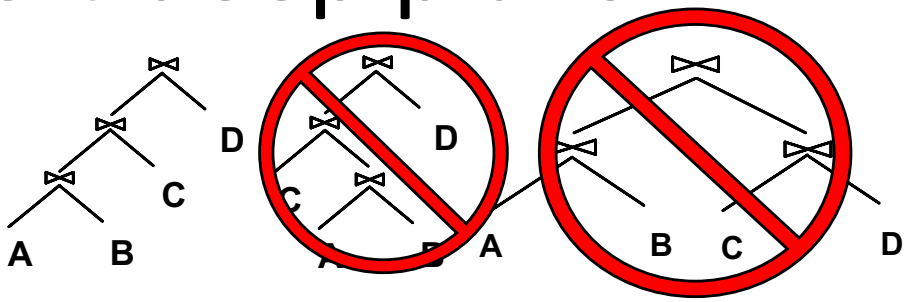
- If we have an index on *rating*:
  - Cardinality = (1/NKeys(I)) * NTuples(R) = (1/10) * 40000 tuples
  - Clustered index: (1/NKeys(I)) * (NPages(I)+NPages(R))
    = (1/10) * (50+500) = 55 pages are retrieved. (This is the ***cost***.)
  - Unclustered index: (1/NKeys(I)) * (NPages(I)+NTuples(R))
    = (1/10) * (50+40000) = 4005 pages are retrieved.
- If we have an index on *sid*:
  - Would have to retrieve all tuples/pages. With a clustered index, the cost is 50+500, with unclustered index, 50+40000.
- Doing a file scan:
  - We retrieve all file pages (500).

Queries over multiple tables

- Recall System R heuristic: *only left-deep join trees* considered.
  - Restricts the search space
  - Left-deep trees allow us to generate all *fully pipelined* plans.
    - Intermediate results not written to temporary files.
  - But note: not all left-deep trees are fully pipelined (e.g., SM join).



Enumeration of left-deep plans



- Left-deep plans differ in
  - the order of relations / joins
  - the access method for each relation
  - the join method for each join
- Enumerated using N passes (if N relations joined):
  - Pass 1: Find best 1-relation plan for each relation
  - Pass i: Find best way to join result of an (*i* -1)-relation plan (as outer) to the *i*' th relation. (*i* between 2 and N)
- For each subset of relations, retain only:
  - Cheapest plan overall, plus
  - Cheapest plan for each *interesting order* of the tuples

Dynamic programming table

Subset of tables in FROM clause	Interesting-order columns	Best plan	Cost
{R, S}	<none>	hashjoin(R,S)	1000
{R, S}	<R.a, S.b>	sortmerge(R,S)	1500

A note on “interesting orders”

Physical property: Order
When should we care?

An intermediate result has an “interesting order” if it is sorted by anything we can use later in the query (“downstream” the arrows):

- ORDER BY attributes
- GROUP BY attributes
- Join attributes of *yet-to-be-added* joins
  - subsequent merge join might be good

Enumeration of plans (continued)

- Match an i -1 way plan with another table *only if*
  a) there is a join condition between them, *or*
  b) all predicates in WHERE have been "used up".
     - i.e., avoid cartesian products if possible.
- ORDER BY, GROUP BY, aggregates etc. handled as a final step
  - via `interestingly ordered' plan if chosen (free!)
  - or via an additional sort/hash operator
- Despite pruning, this System R search algorithm is exponential in #tables

Heuristic!

Example

Sailors:
Hash, B+ on *sid*
Reserves:
Clustered B+ tree on *bid*
B+ on *sid*
Boats
B+ on *color*

```
SELECT S.sid, COUNT(*) AS number
  FROM Sailors S, Reserves R, Boats B
 WHERE S.sid = R.sid
   AND R.bid = B.bid
   AND B.color = "red"
 GROUP BY S.sid
```

- Pass 1: Best plan(s) for each relation
  - Sailors, Reserves: file scan
    - Also B+ tree on Reserves.bid as interesting order
    - Also B+ tree on Sailors.sid as interesting order
  - Boats: B+ tree on color

Subset of tables in FROM clause	Interesting-order columns	Best plan	Cost
{Sailors}	--	filescan	
{Reserves}	--	filescan	
{Boats}	--	B-tree on color	
{Reserves}	(bid)	B-tree on bid	
{Sailors}	(sid)	B-tree on sid	

Pass 2

- for each plan P in pass 1
  for each FROM table T not in P
    for each access method M on T
      generate P ⋈ M(T) for each join methods
  - File Scan Reserves (outer) with Boats (inner)
  - File Scan Reserves (outer) with Sailors (inner)
  - Reserves Btree on bid (outer) with Boats (inner)
  - Reserves Btree on bid (outer) with Sailors (inner)
  - File Scan Sailors (outer) with Boats (inner)
  - File Scan Sailors (outer) with Reserves (inner)
  - Boats Btree on color with Sailors (inner)
  - Boats Btree on color with Reserves (inner)
- Retain cheapest plan for each (pair of relations, order)

Subset of tables in FROM clause	Interesting-order columns	Best plan	Cost
{Sailors}	--	filescan	
{Reserves}	--	Filescan	
{Boats}	--	B-tree on color	
{Reserves}	(bid)	B-tree on bid	
{Sailors}	(sid)	B-tree on sid	
{Boats, Reserves}	(B.bid) (R.bid)	SortMerge(B-tree on Boats.color, filescan Reserves)	
Etc...			

Pass 3 and beyond

- Using pass 2 plans as outer relations, generate plans for the next join
  - E.g. Boats B+-tree on color with Reserves (bid) (sort-merge)
    inner Sailors (B-tree sid) sort-merge
- Then, add cost for groupby / aggregate:
  - This is the cost to sort the result by sid, *unless it has already been sorted by a previous operator.*
- Then, choose the cheapest plan

Now you understand the optimizer!

- Benefits: You can influence the system's optimizer
  - People who write non-trivial SQL often get frustrated with optimizer
    - It picked a bad plan
    - It didn't use the index I built
    - …
  - Understanding the optimizer can lead you to:
    - Design your DB & indexes better
    - Avoid "weak spots" in your optimizer's implementation
    - Guide your optimizer to do what you want

Points to remember (1)

- Three parts to optimizing a query:
  - Plan space
    - E.g., left-deep plans only
    - Avoid cartesian products
    - Prune plans with *interesting orders* separate from unordered plans
  - Cost estimation
    - Output cardinality and cost for each plan node
    - *Key issues*: Statistics, indexes, operator implementations
  - Search strategy
    - we learned “bottom-up” dynamic programming

Points to remember (2)

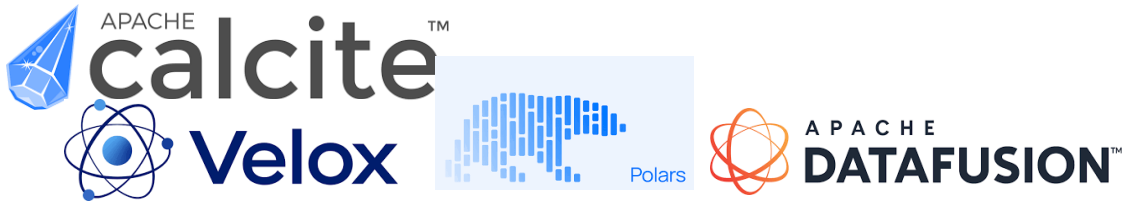
- Single-relation queries:
  - All access paths considered, cheapest is chosen
  - *Issues*:
    - Selections that *match* index
    - Whether index key has all needed fields
    - Whether index provides tuples in an interesting order

Points to remember (3)

- Multiple-relation queries:
  - All single-relation plans are first enumerated
    - Selections/projections considered as early as possible
  - Use best 1-way plans to form 2-way plans. Prune losers.
  - Use best (i-1)-way plans and best 1-way plans to form i-way plans
  - At each level, for each subset of relations, retain:
    - best plan for each interesting order (including no order)

Conclusion

- Optimization is the reason for the lasting power of the RDBMS → more important than a fast execution engine
  - real queries have outer-joins, semi-joins, anti-joins, k-way joins, etc
- But it may still be primitive in some SQL databases
- Many new areas!
  - Smarter statistics (fancy histograms, "sketches", ML models for selectivity estimation)
  - Auto-tuning statistics
  - Adaptive runtime re-optimization vs. plan first execute later
  - Multi-query optimization
  - Parallel scheduling issues
  - Optimizer as a service
  - Query engine as a service
  - AI-based query optimization / learned optimization



Much more to learn about

- Query execution & compilation
- Query scheduling – where, when, how to execute
  - Fairness, throughput, latency